

Processes

Abhilash Jindal

Agenda

- Vocabulary (OSTEP Ch. 4)
 - What is a process? System calls, Scheduler, Address space, Process control block
- Memory isolation (OSTEP Ch. 13, 15, 16)
 - How to isolate memory into address spaces?
- Processes in action (xv6 Ch. 3: system calls, x86 protection, trap handlers)
 - User stack<>kernel stack, sys call handling
- Scheduling (xv6 Ch5: context switching, OSTEP Ch. 6-9)
 - Response time, throughput, fairness

Process is a running program

- Load program from disk to memory
 - Exactly how we loaded OS
- Give control to the process. Jump cs, eip

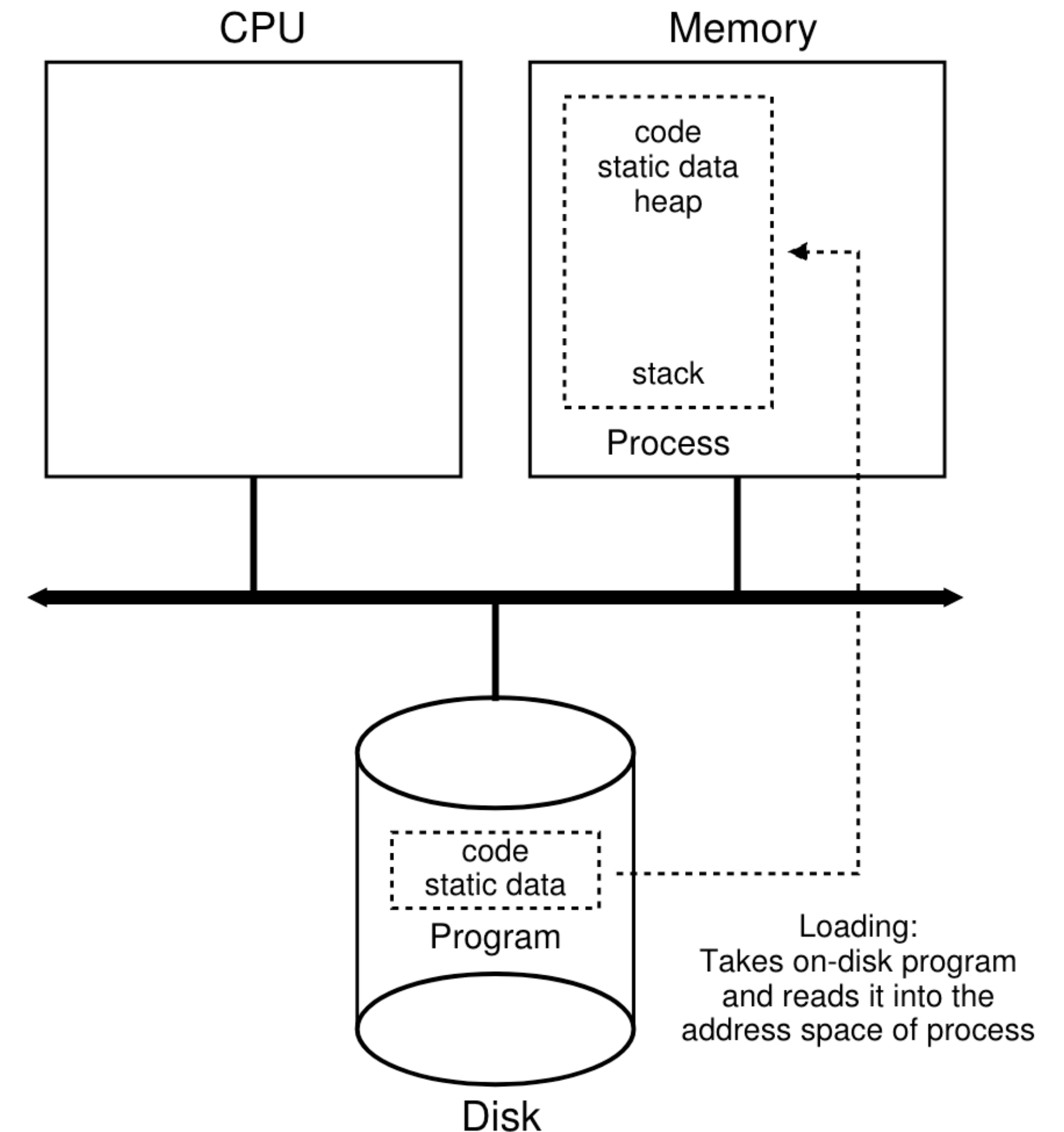


Figure 4.1: Loading: From Program To Process

Processes can ask OS to do work for them

System calls

```
$ strace cat /tmp/foo
...
openat(AT_FDCWD, "/tmp/foo", O_RDONLY) = 3
read(3, "hi\n", 131072) = 3
write(1, "hi\n", 3) = 3
...
```

OS maintains process states

Scheduler switches between processes

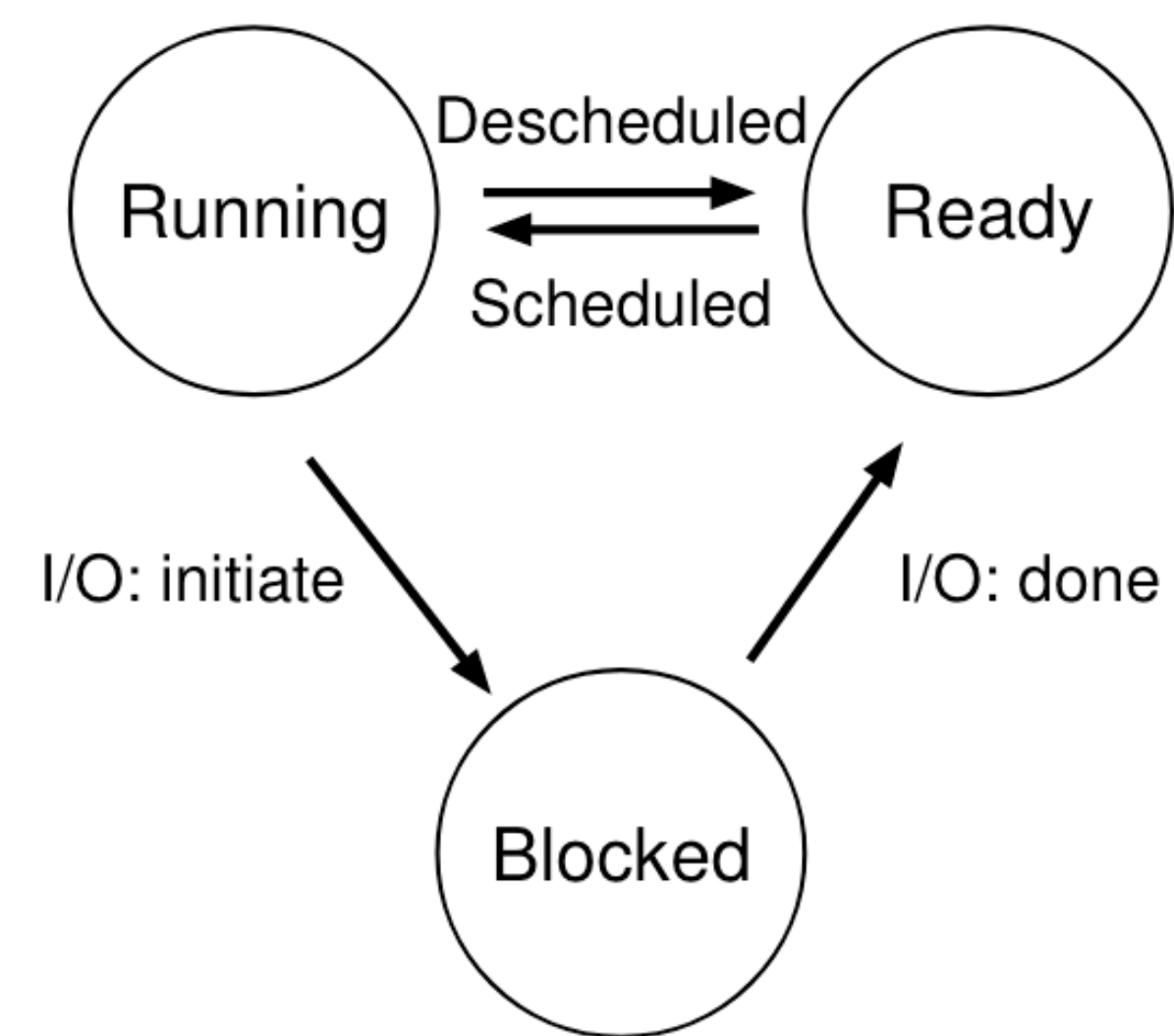


Figure 4.2: Process: State Transitions

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Calculator analogy: A process is computing long sum



20
10
30
50
30
10
20
10

- 2 0 = (move pointer to 10)
- + 1 0 = (move pointer to 30)
- + 3 0 = (move pointer to 50)
- + 5 0 = (move pointer to 30)
- + 3 0 = (move pointer to 10)
- + 1 0 = (move pointer to 20)
- + 2 0 = (move pointer to 10)

Context switch snatches the calculator



20
10
30
50
30
10
20
10



10
70
20
40
20
10
50
10

- Steps to share the calculator:
 - $20 + 10 = 30 + 30 = 60$
 - Write 60 in notebook, remember that we were done till 30, give calculator
 - $10 + 70 = 80$
 - Write 80 in notebook, remember that we were done till 70, give the calculator back

OS maintains list of Process Control Blocks

xv6 PCB

```
struct {
    struct proc proc[NPROC];
} ptable;

enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

struct proc {
    ...
    int pid;                // Process ID
    char name[16];          // Process name (debugging)
    struct proc *parent;    // Parent process
    enum procstate state;   // Process state
    struct file *ofile[Nofile]; // Open files
    struct inode *cwd;      // Current directory
    struct context *context; // Switch here to run process
};
```


Address space

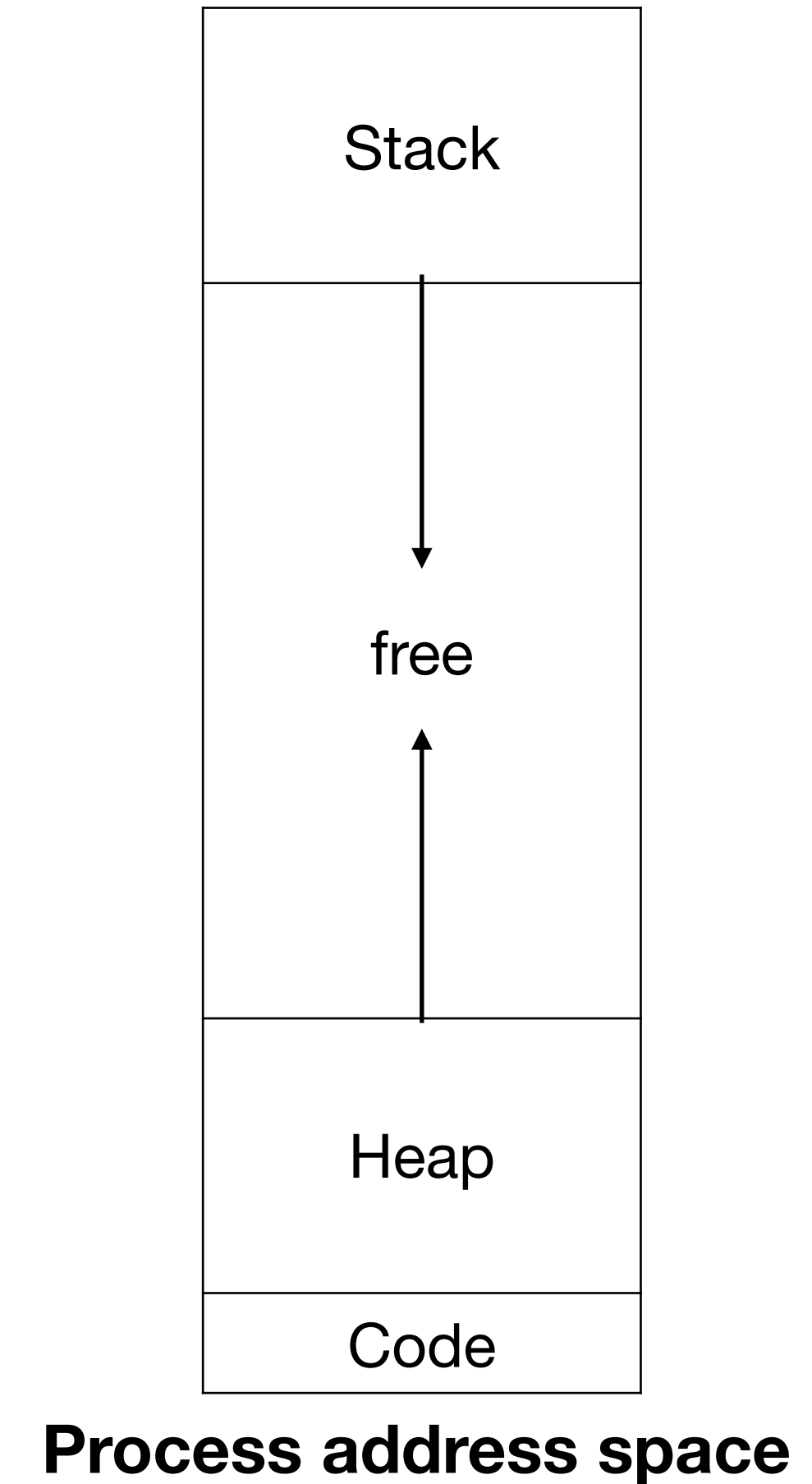
OSTEP Ch. 14,17

Abhilash Jindal

Process Address Space

Code, Heap, Stack

- Example: 01/mem.c
 - Multiple processes access the same pointer location but the OS and hardware isolated process to its own *address space*. Each process thinks they own the memory
- Code: Instructions, global variables. Loader reserved this space when loading from ELF.
- Stack: Local variables
- Heap: Dynamically allocated variables. malloc/free



Function calling in action

Stack

02.s

eip

→

_foo:

pushl %ebp

movl %esp, %ebp

movl 8(%ebp), %eax

addl 12(%ebp), %eax

popl %ebp

retl

.globl _main

.p2align 4, 0x90

_main:

pushl %ebp

movl %esp, %ebp

subl \$24, %esp

movl \$0, -4(%ebp)

movl \$41, (%esp)

movl \$42, 4(%esp)

calll _foo

addl \$24, %esp

popl %ebp

retl

Save caller's base pointer

ebp = esp

eax = *(ebp + 8)

eax = eax + *(ebp + 12)

Restore caller's base pointer

change eip to return address

-- Begin function main

Save caller's base pointer

ebp = esp

esp = esp - 0x18

*(ebp-4)=0

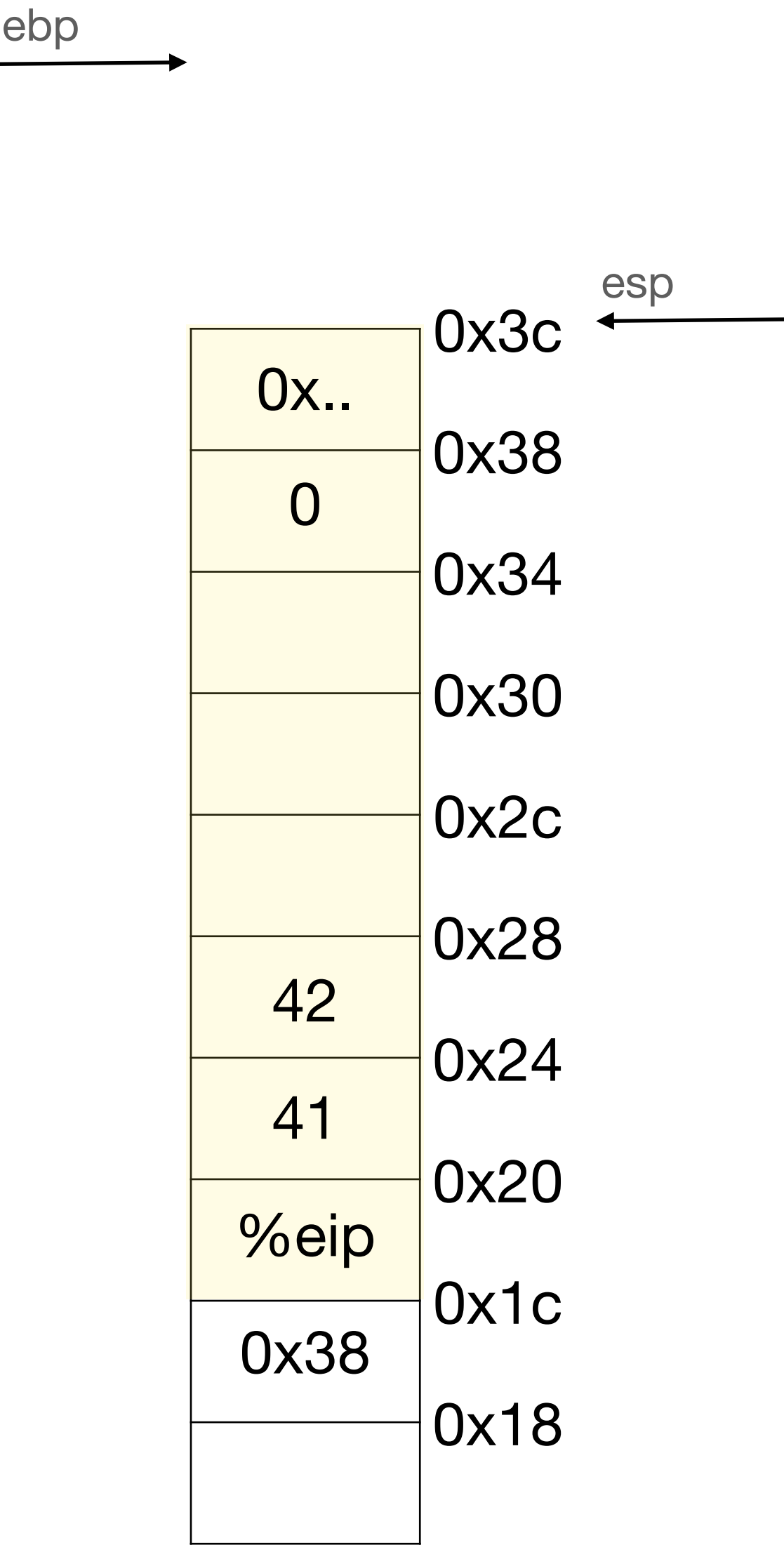
*(esp) = 41

*(esp+4) = 42

Push current eip on to stack, jump to foo

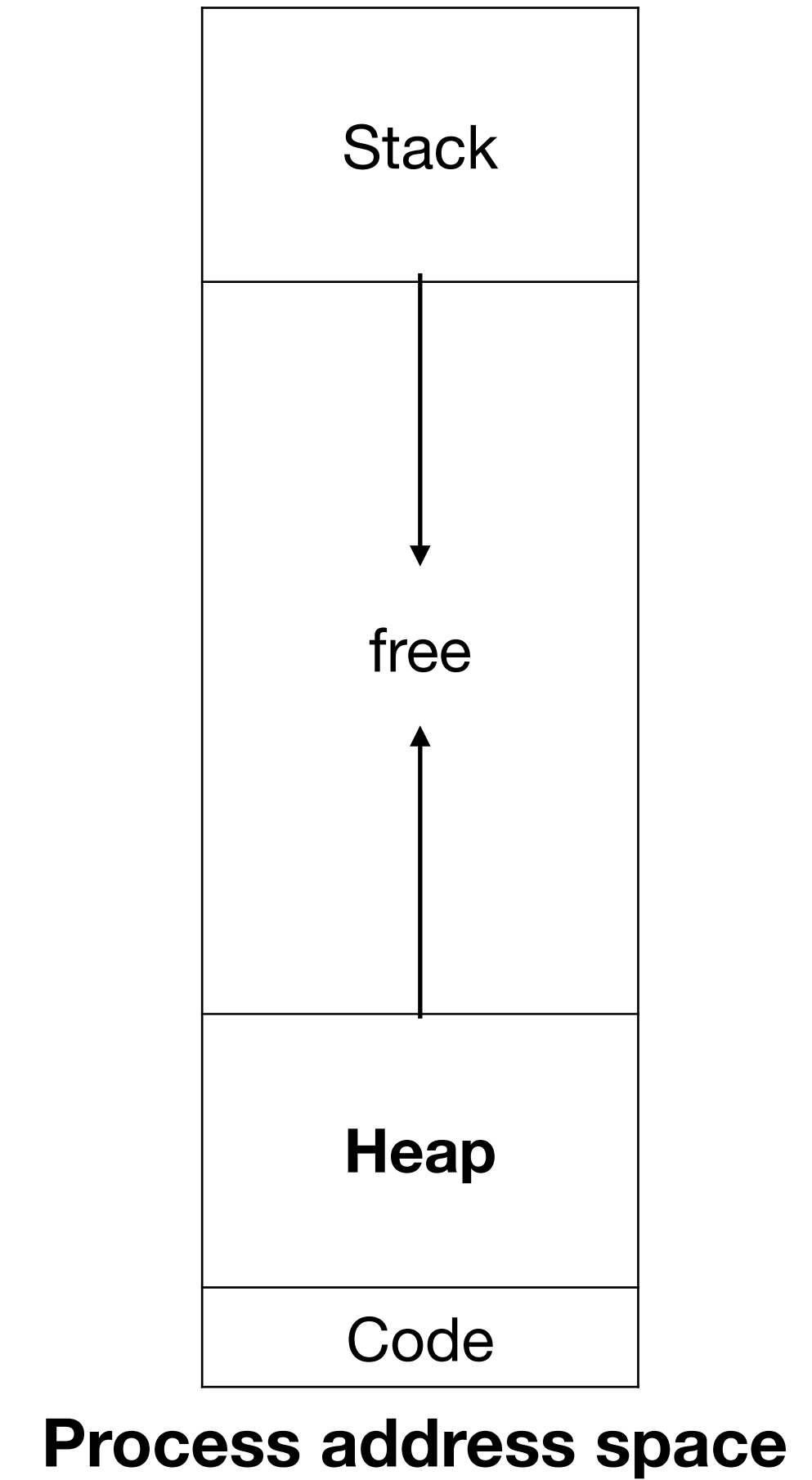
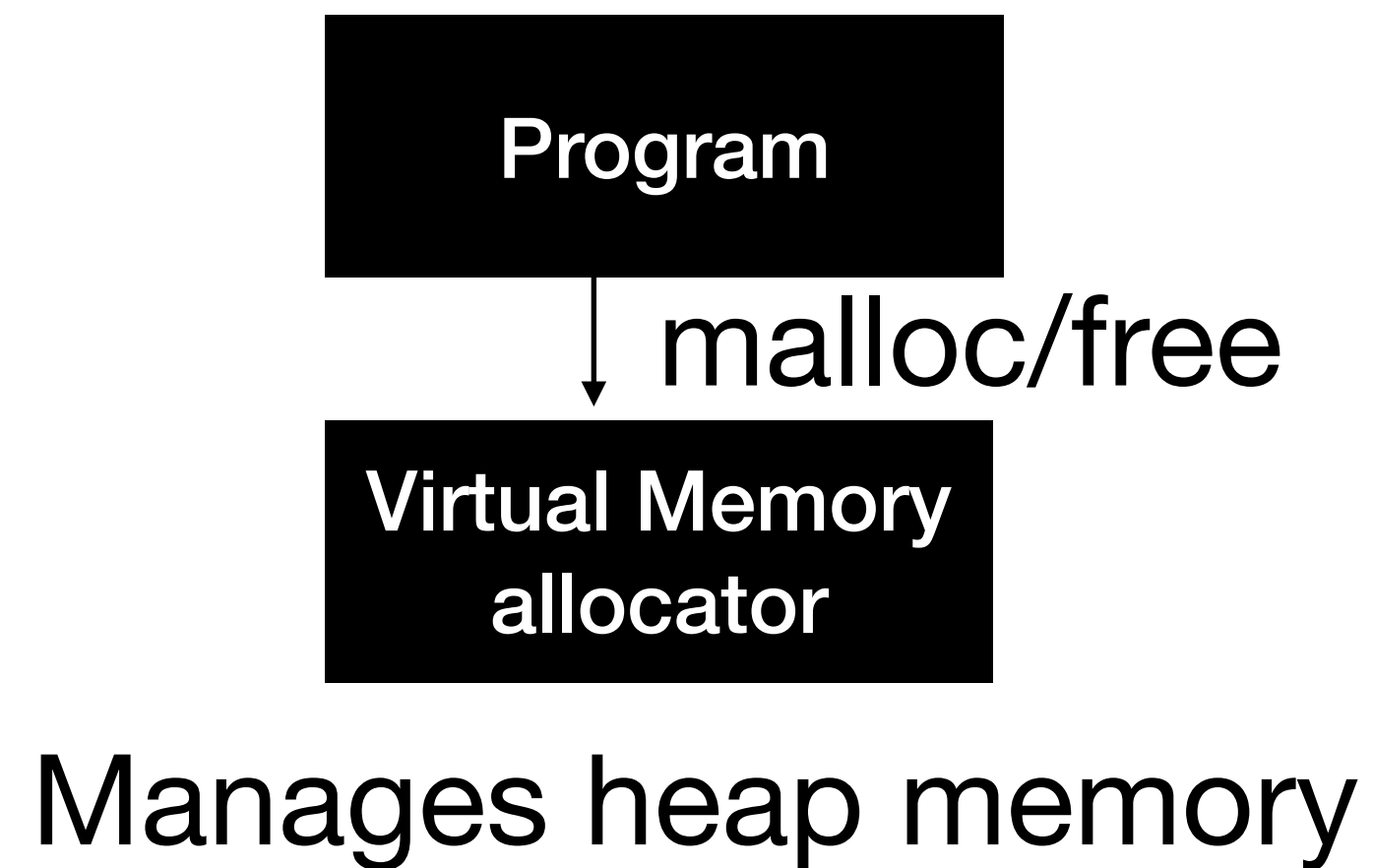
esp = esp + 24 (Restore caller's esp)

Restore caller's ebp



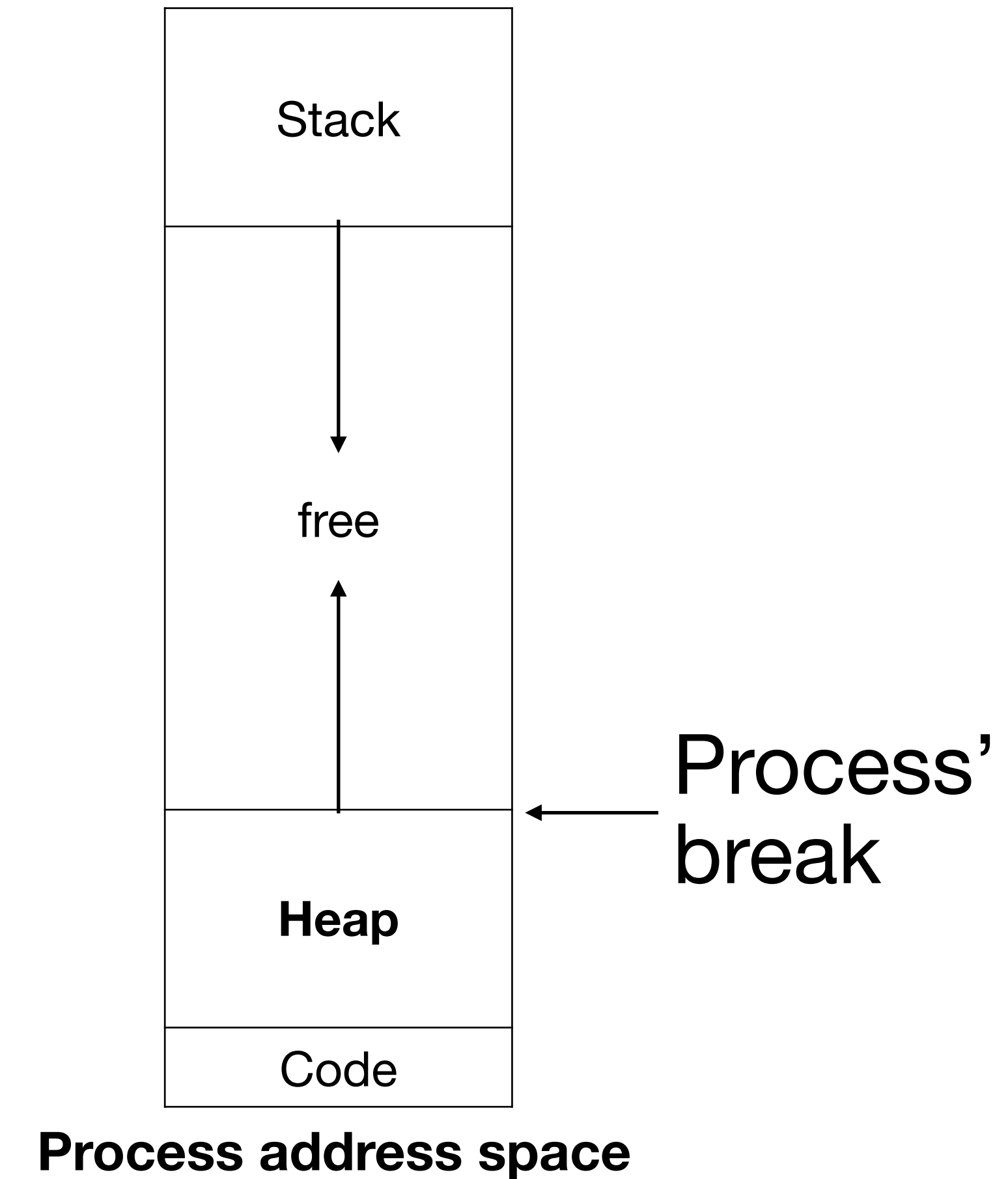
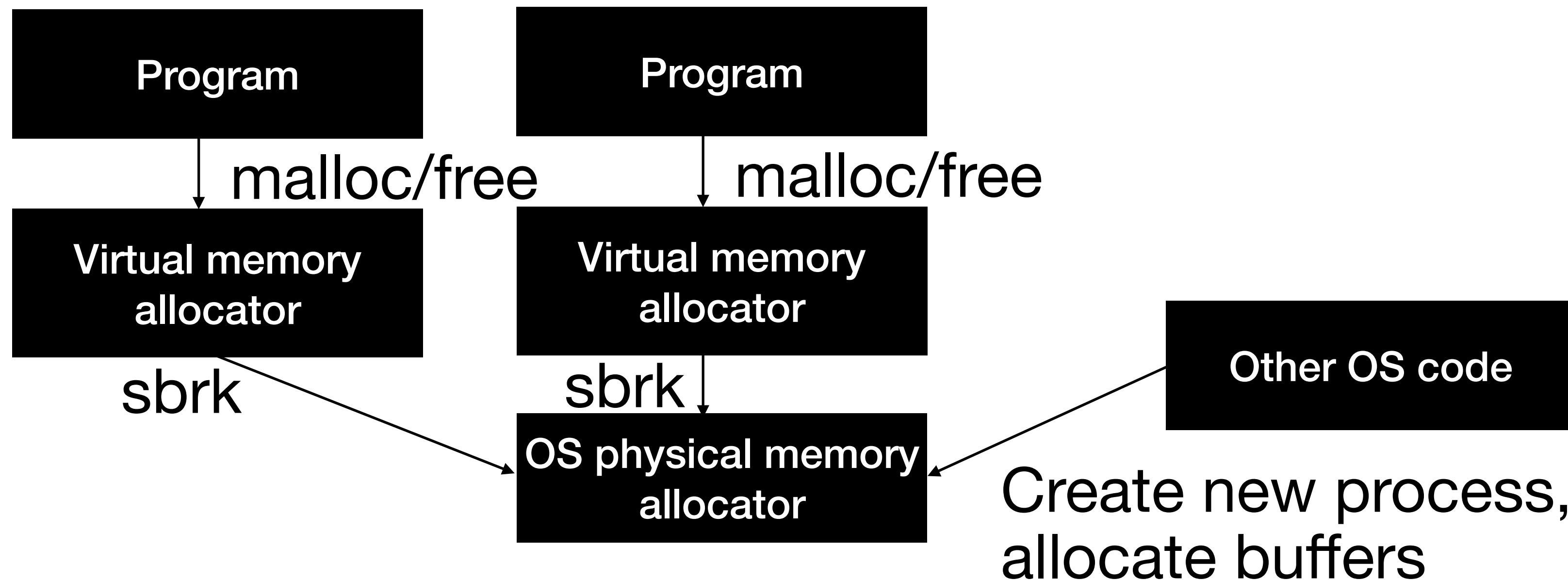
Memory allocator

Works with virtual memory



OS memory allocator

- `sbrk(int increment)` increments process' break. *increment* can be negative.



Heap memory APIs and bugs

- malloc, free. va.c.
 - malloc is for dynamic allocation. Allocation size may not be known at compile time. Slower than stack allocations. Memory allocator needs to find free space on heap.
 - Size of the allocated region is not passed in free. It must be tracked by the memory-allocation library itself.
- Null pointer dereference. null.c
- Memory leak. leak.c
- Buffer overflow. overflow.c
- Use after free. useafterfree.c
- Invalid free. invalidfree.c
- Double free. doublefree.c
- Uninitialised read. uninitread.c

Memory allocation

How to check for invalid frees? How much memory needs to be freed?

```
ptr=malloc(size_t size = 20);  
free(ptr);
```

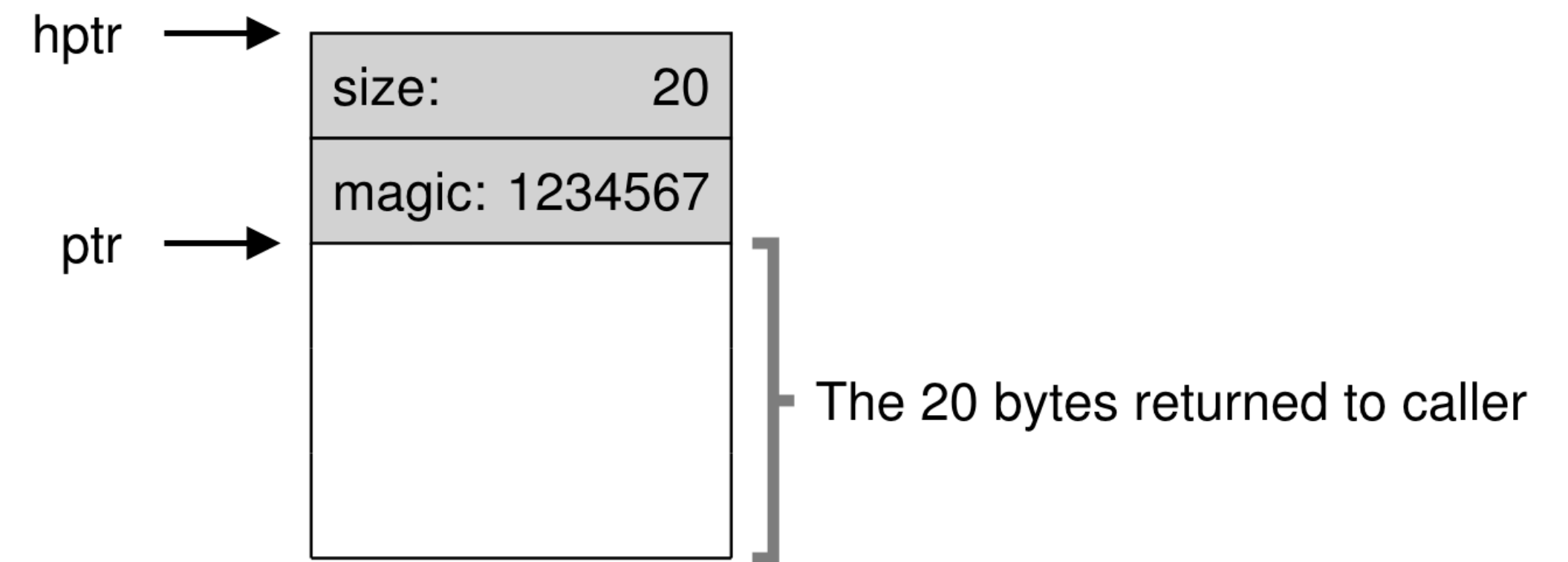
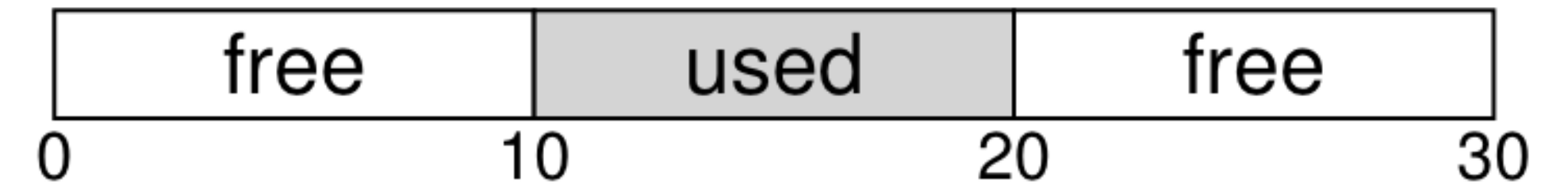


Figure 17.2: Specific Contents Of The Header

Stack allocations are faster than heap allocations. No need to find space.

Memory allocator



Fragmented heap over time

- Assumptions
 - Do not apriori know allocation size and order
 - Cannot move memory once it is allocated. Program might have the pointer to it.
- Goals
 - Quickly satisfy variable-sized memory allocation requests. How to track free memory?
 - Minimize fragmentation
 - All allocators are *bad* for *some programs*. Understand pros and cons of allocator designs

Memory (de)allocation patterns

- Small mallocs can be frequent. Large mallocs are usually infrequent.
 - After malloc, program will initialise the memory area.
- “Clustered deaths”: Objects allocated together die together.

Free list splitting and coalescing

```
ptr = malloc(100)
sptr = malloc(100)
optr = malloc(100)
free(sptr)
free(ptr)
free(optr)
```

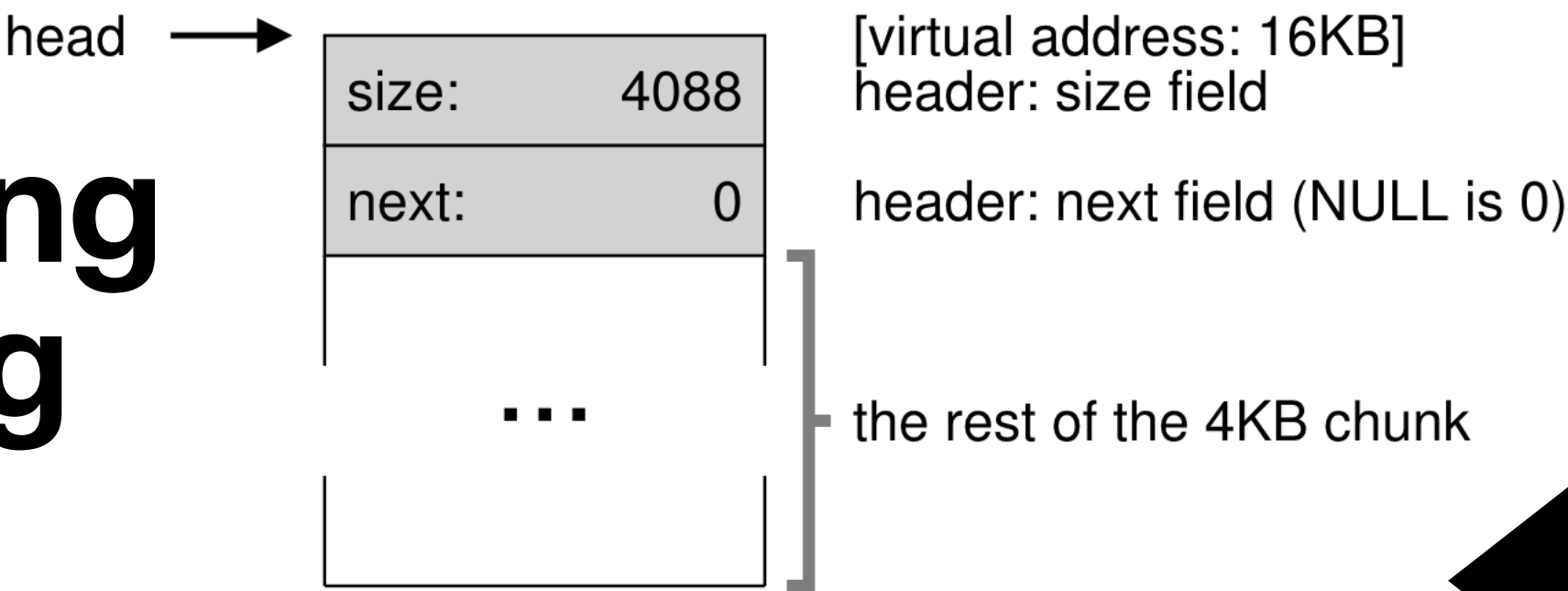


Figure 17.3: A Heap With One Free Chunk

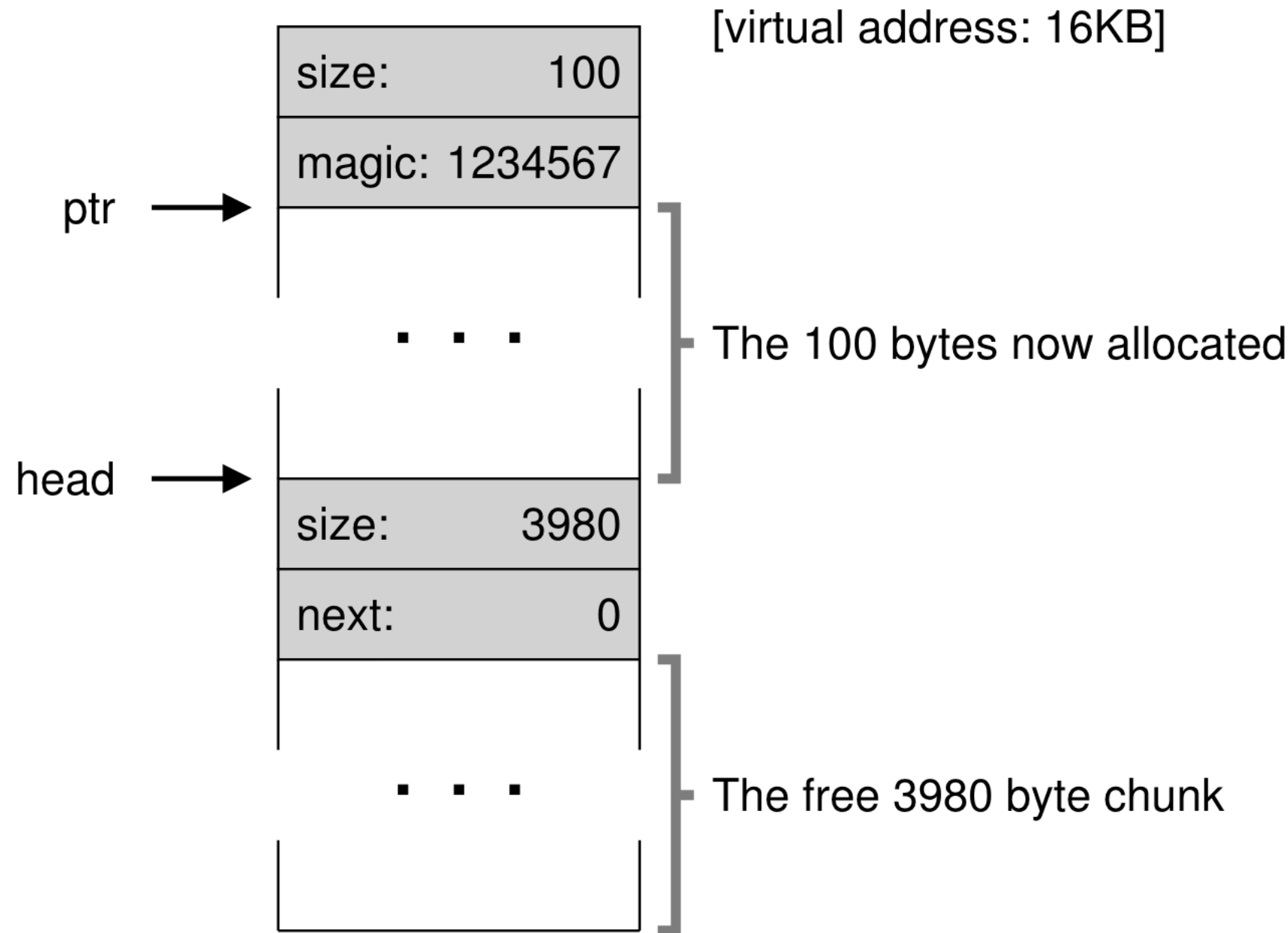


Figure 17.4: A Heap: After One Allocation

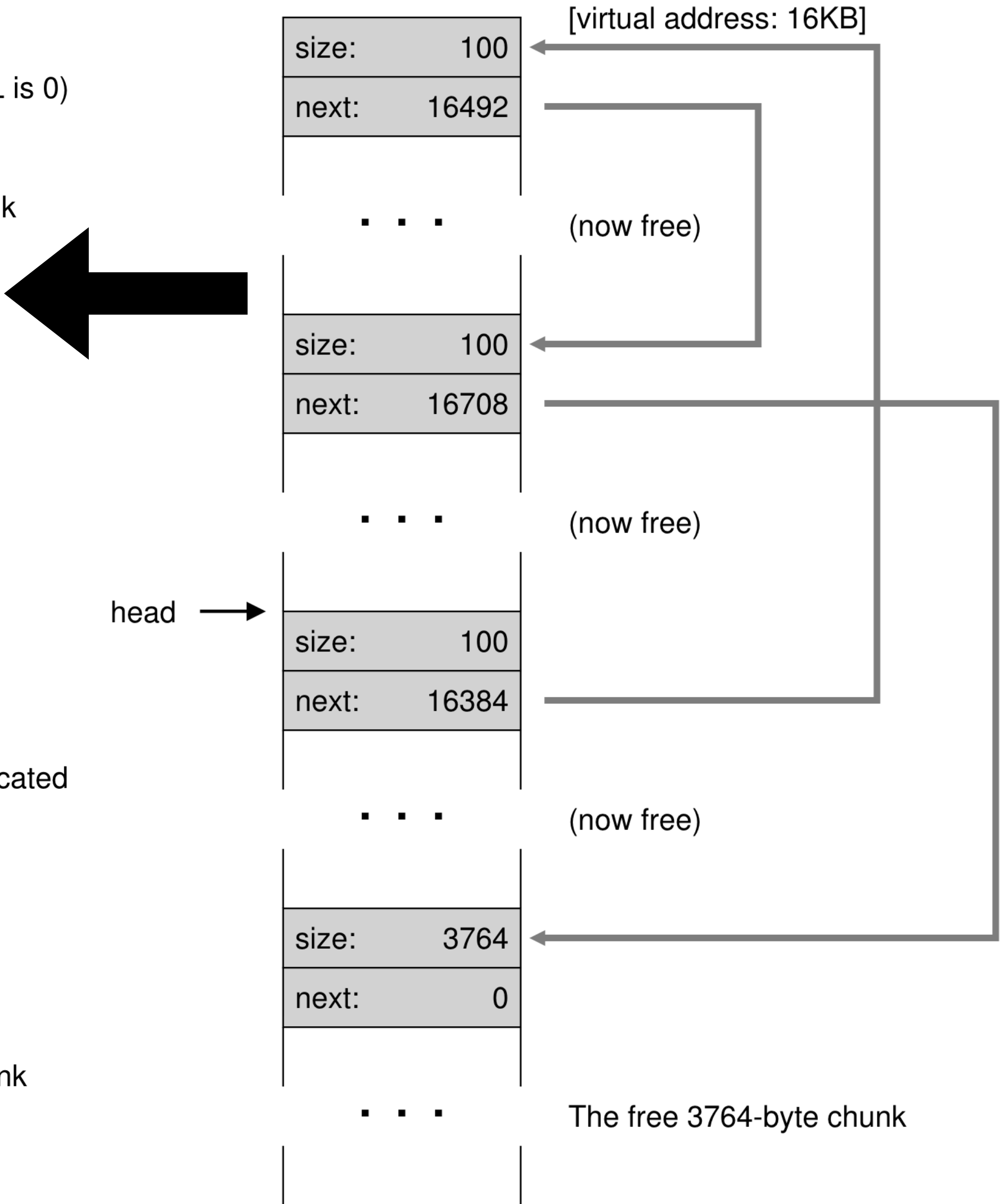


Figure 17.7: A Non-Coalesced Free List

Which block to allocate?

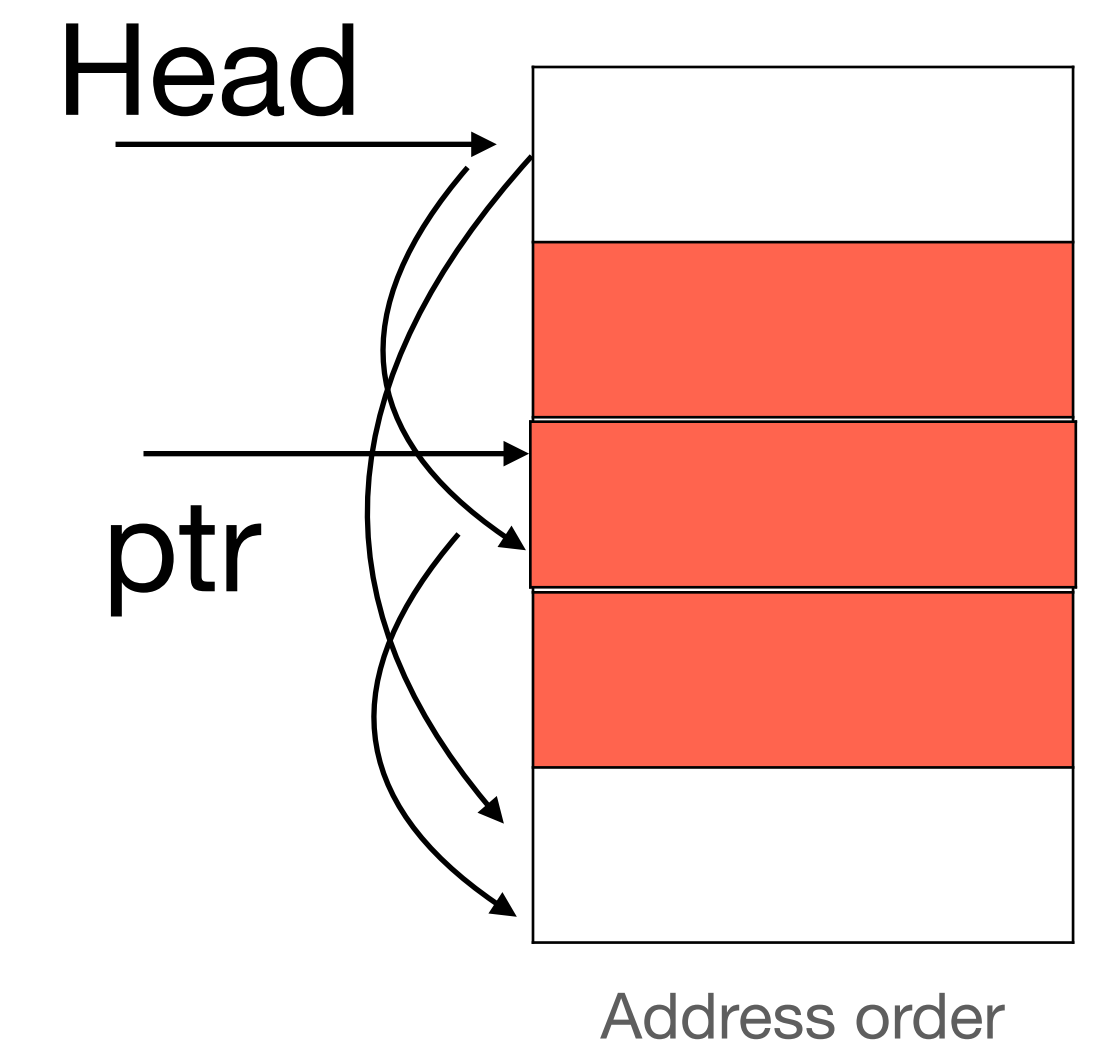
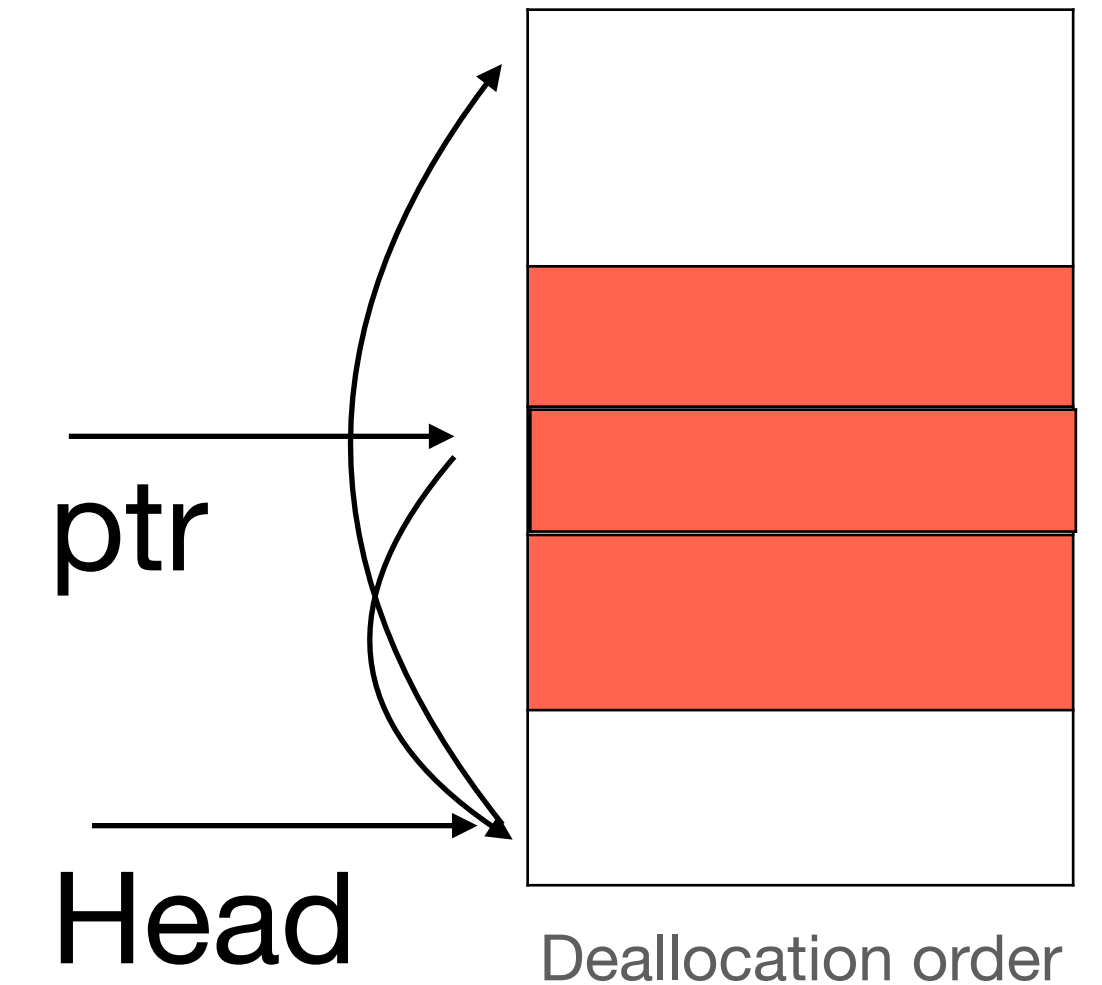
Example: malloc(15)

- Best fit
 - Slow. need to search the whole list
- First fit
 - Faster. (xv6: umalloc.c)
 - May cause more fragmentation by breaking up larger free blocks
 - Example: malloc(25)



In which order should we maintain the free list?

- `free(ptr)`
- Slow frees in address order: need to traverse the free list
- `umalloc.c` in `xv6` implements address order

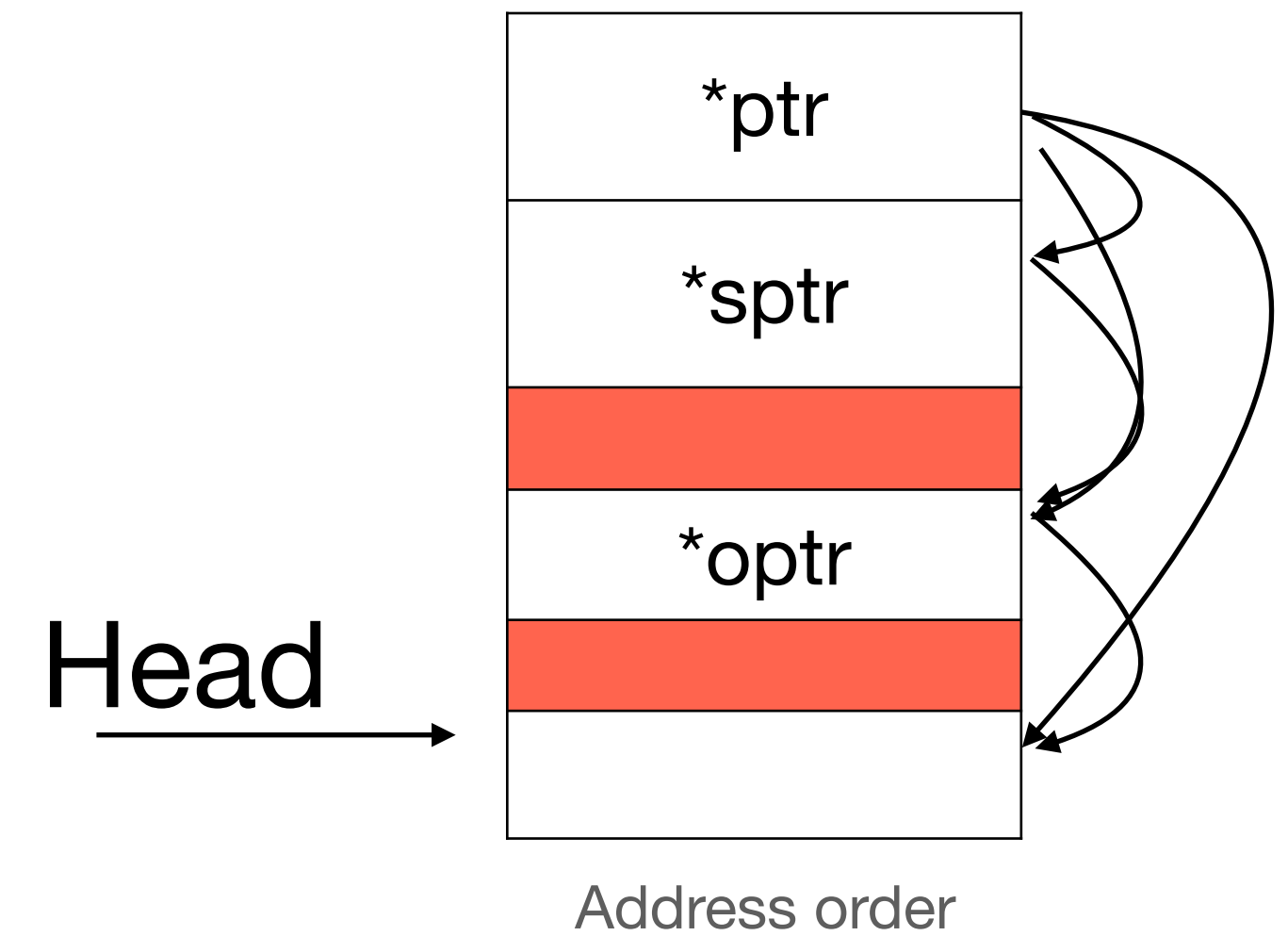
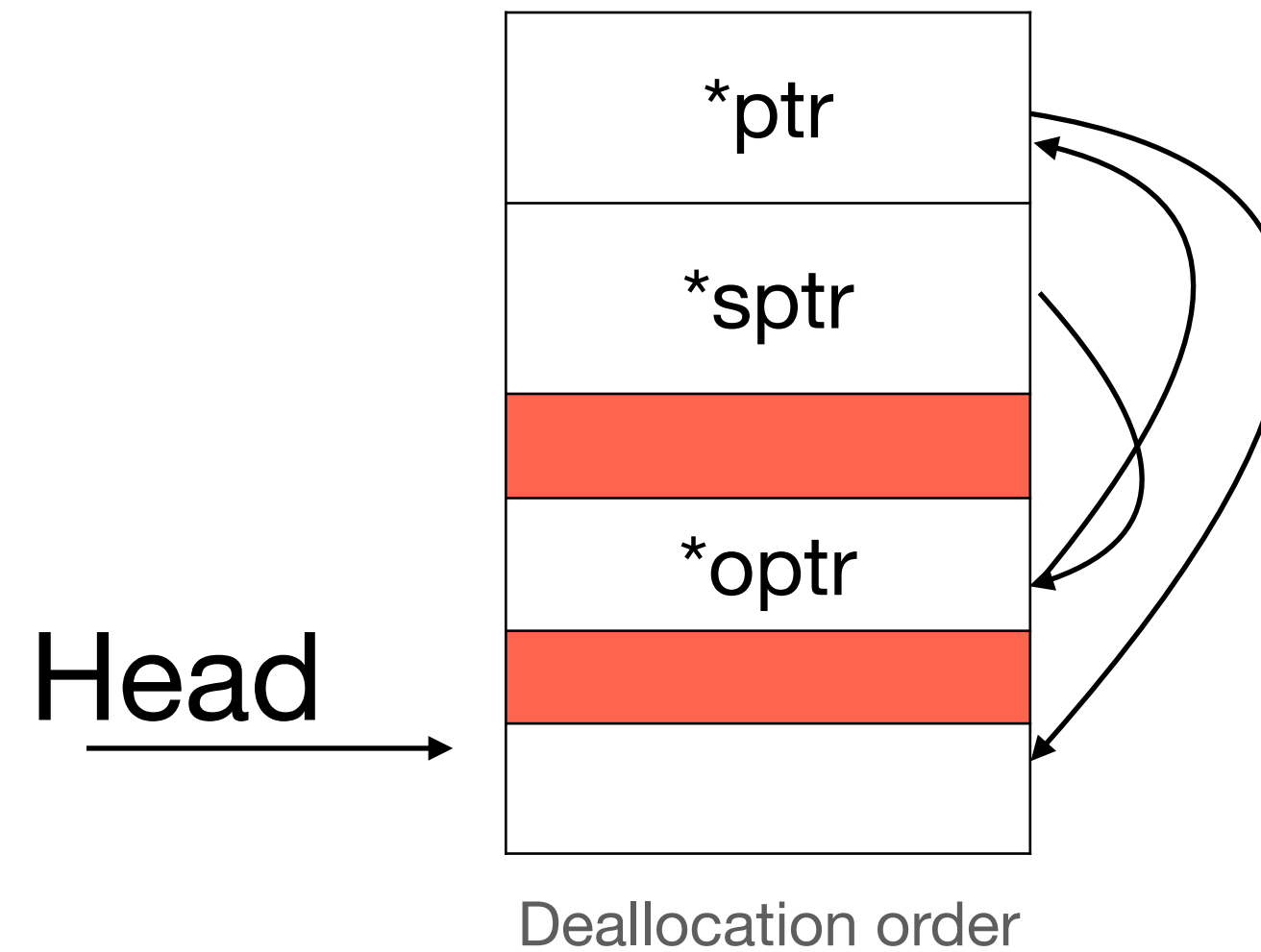


How to do coalescing?

`free(ptr)`

`free(optr)`

`free(sptr)`



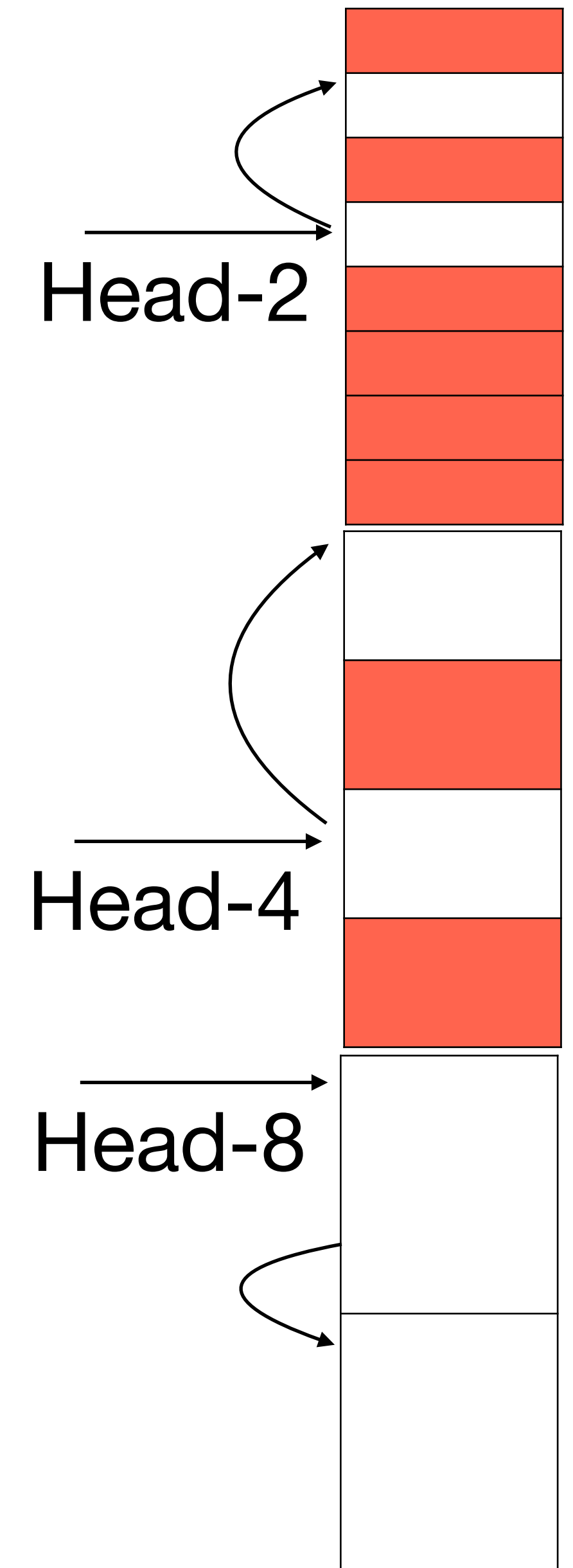
Easier to coalesce in an address-order free-list

Tradeoffs

- Allocation
 - First fit: faster search, leads to fragmentation
 - Best fit: slower search, less fragmentation
- Free list order
 - Address order: Slow frees, easier to coalesce
 - Deallocation order: Fast frees, difficult to coalesce

Segregated lists

- Separate lists for each size.
- “Segregated fit”: *First fit* in the smallest object list that can fit the object. Approximates *best fit*.
 - No splitting, coalescing
- Internal fragmentation: allocates more than asked
- Wastes memory: Reserving space in a segregated list is wasteful if allocations are not requested for that object size



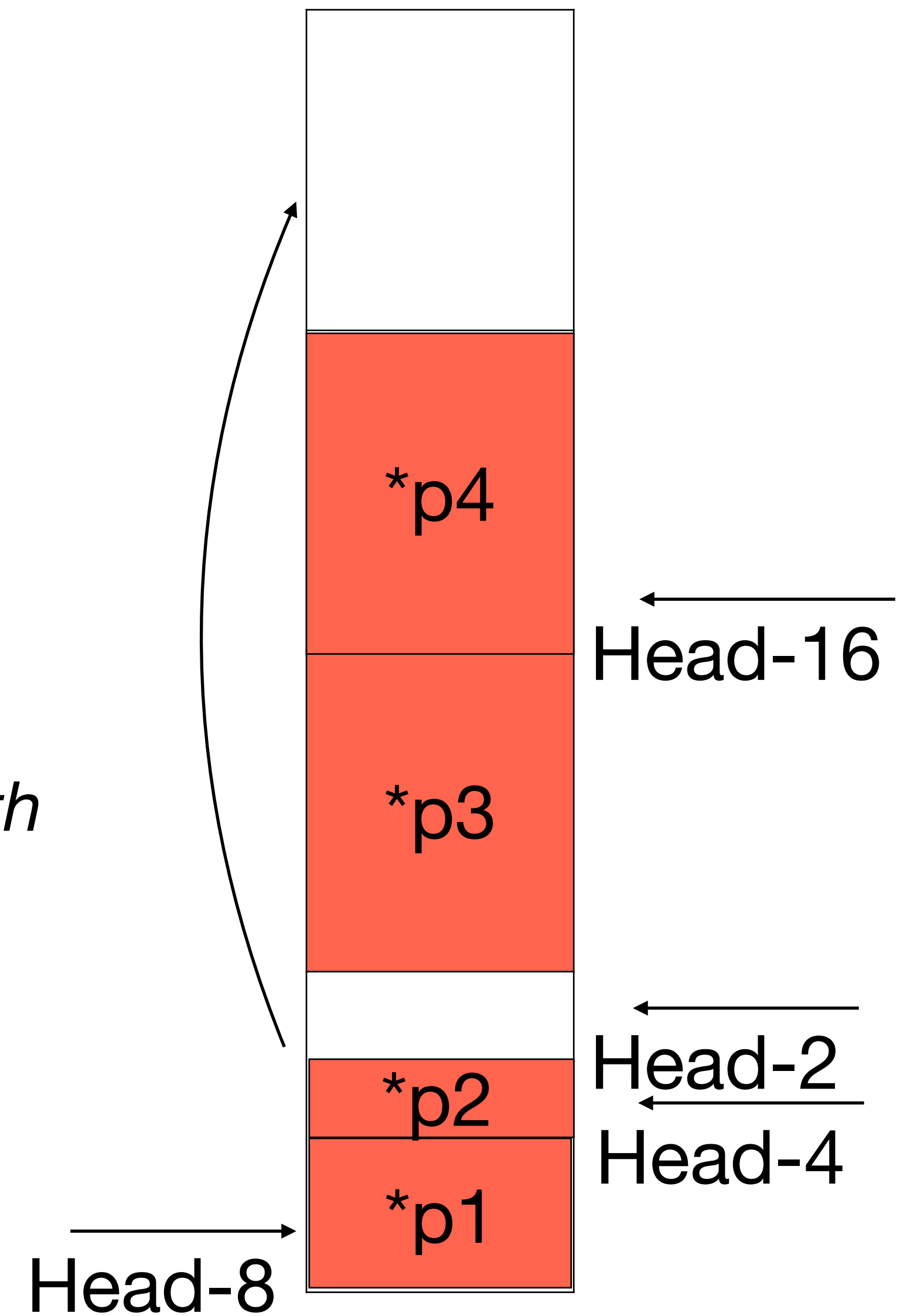
Slab allocator

Used in Linux Kernel

- Knows the size of allocations (process control blocks, buffer cache block, etc.) made by the Linux kernel. Keep segregated lists for each such struct.
 - No internal fragmentation: Exactly the size of the struct
 - Hierarchical allocator: Return unused lists back to global allocator

Buddy allocator

- Example:
 - `p1 = malloc(3)`
 - `p2 = malloc(2)`
- On-the-fly segregated lists
- Straightforward splitting and *coalescing* (*coalesce with my buddy*)
 - `free(p4)`
- *Deallocation order*: fast frees
- Used in Linux kernel



Aside: automatic memory management

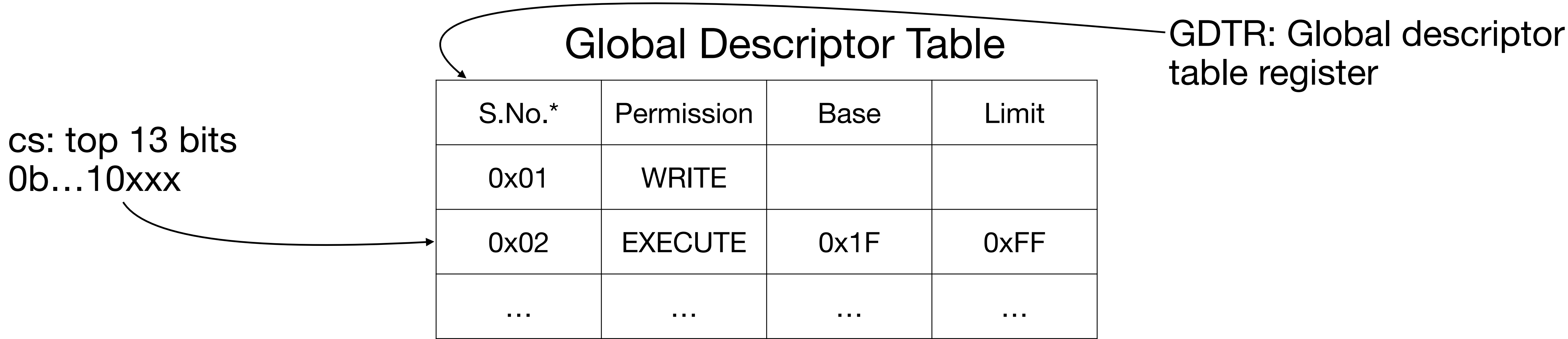
- “Higher level” languages do not expose raw pointers to programmers.
Example: Java, Python, Go
- The language runtime manages memory. Programmer need not call free.
- Largely prevents memory leaks, dangling references, double free, null pointer dereference
- Mark and sweep garbage collector, reference counting based garbage collector
- Copying GC can do compaction to defragment heap. Will rewrite pointers.
- Can incur heavy performance penalty

Memory isolation

OSTEP Ch. 13,15,16

Abhilash Jindal

Many segments can be initialised in GDT



*: S.No. added only for illustration

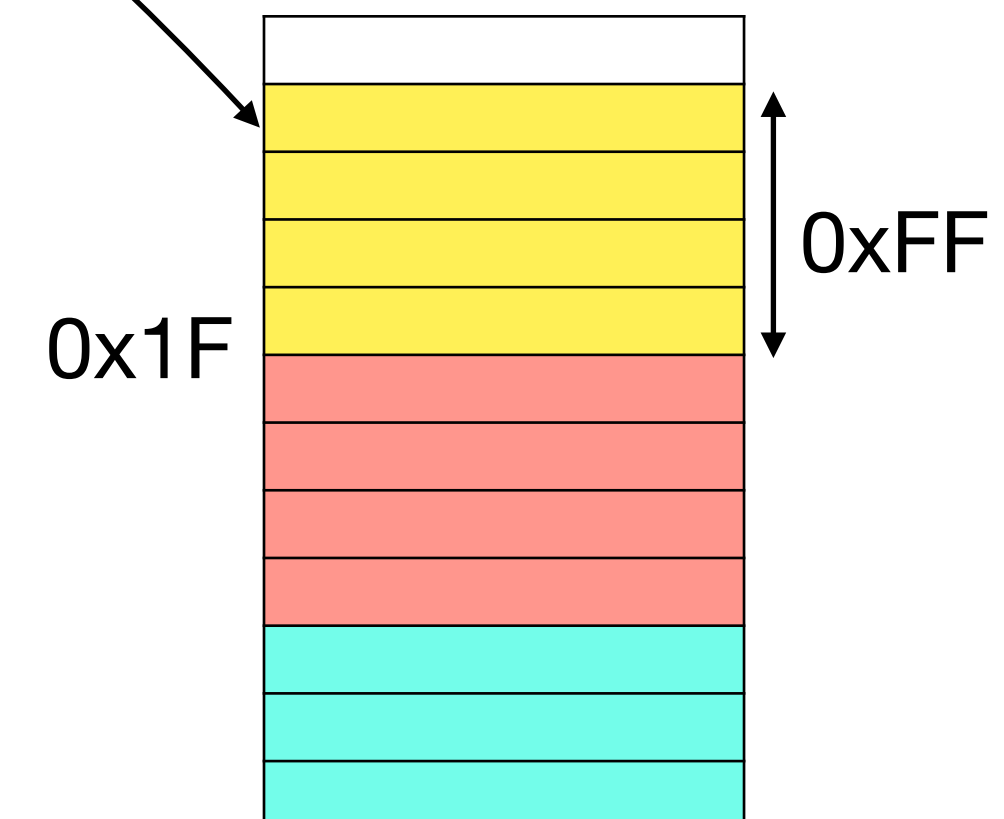
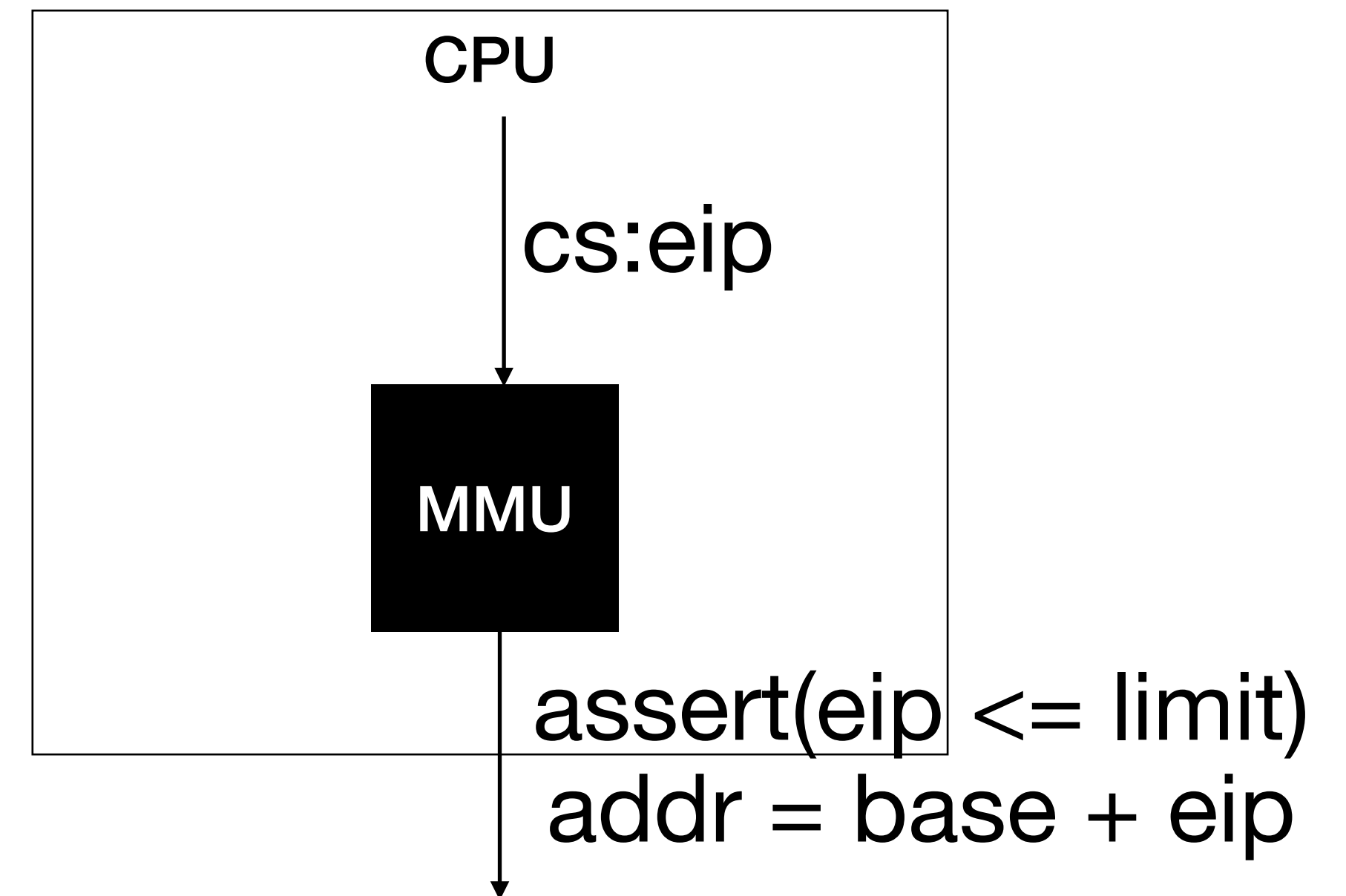
Address translation

Global Descriptor Table

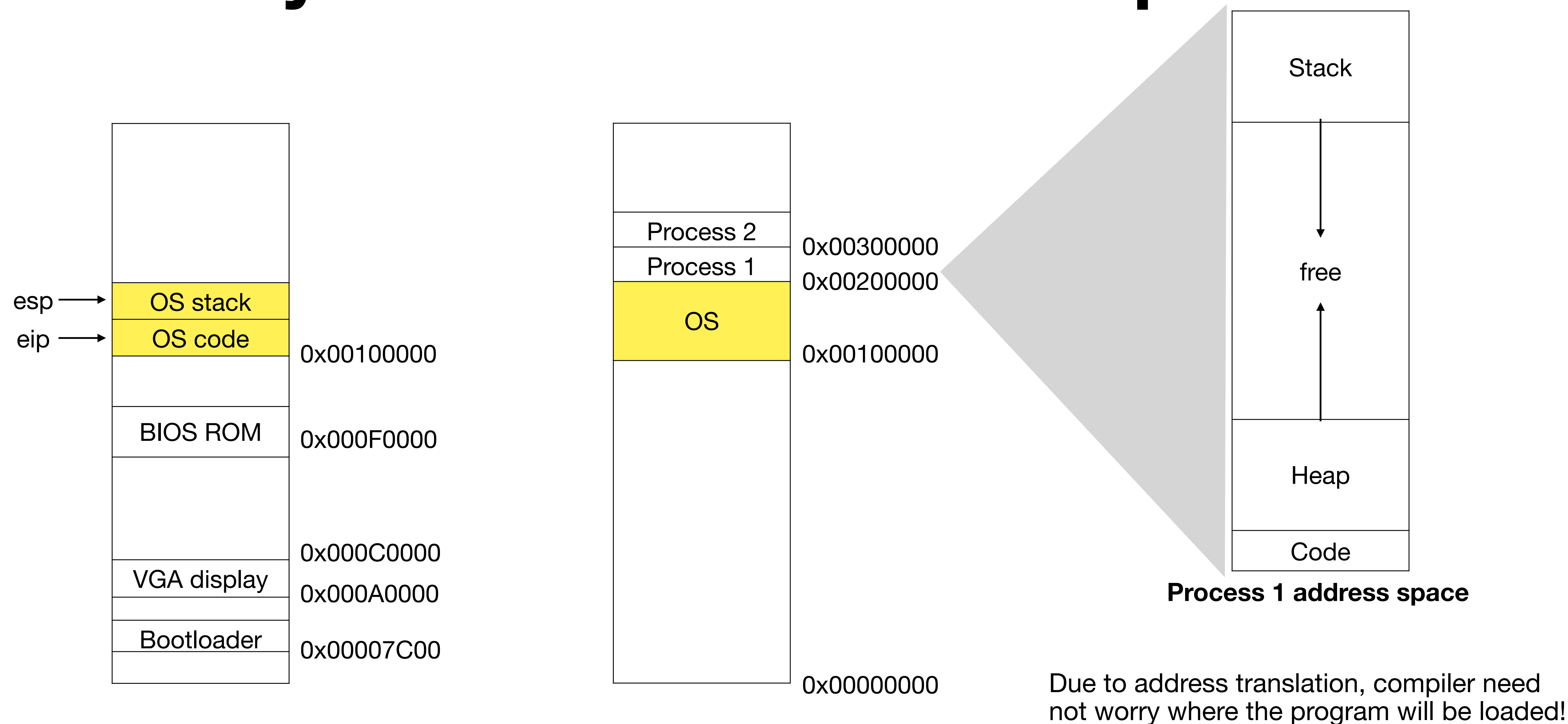
cs: top 13 bits
0b...10xxx

S.No.*	Permission	Base	Limit
0x01	WRITE		
0x02	EXECUTE	0x1F	0xFF
...	...	...	...

- Can “isolate” different segments from each other



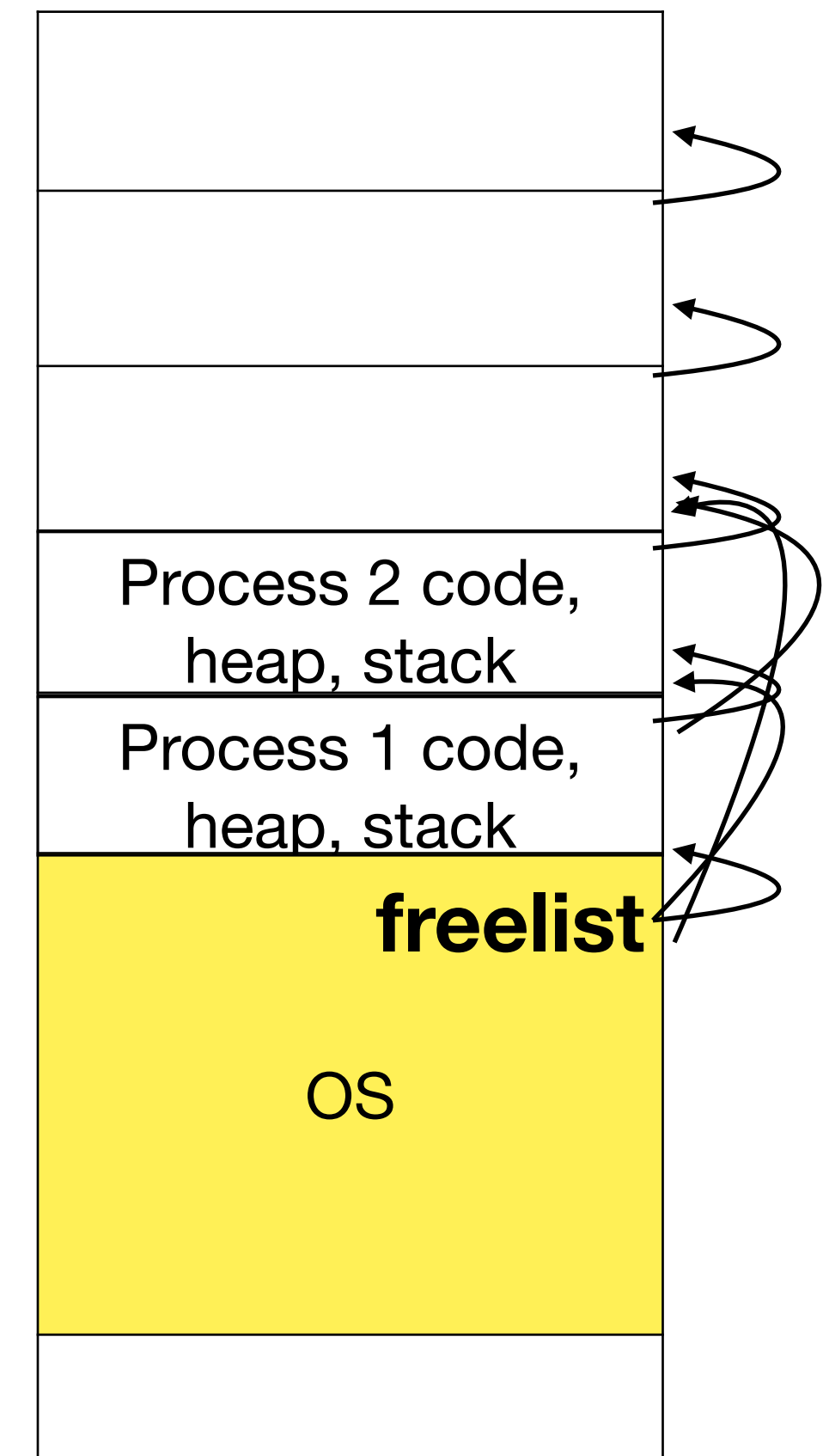
Memory isolation and address space



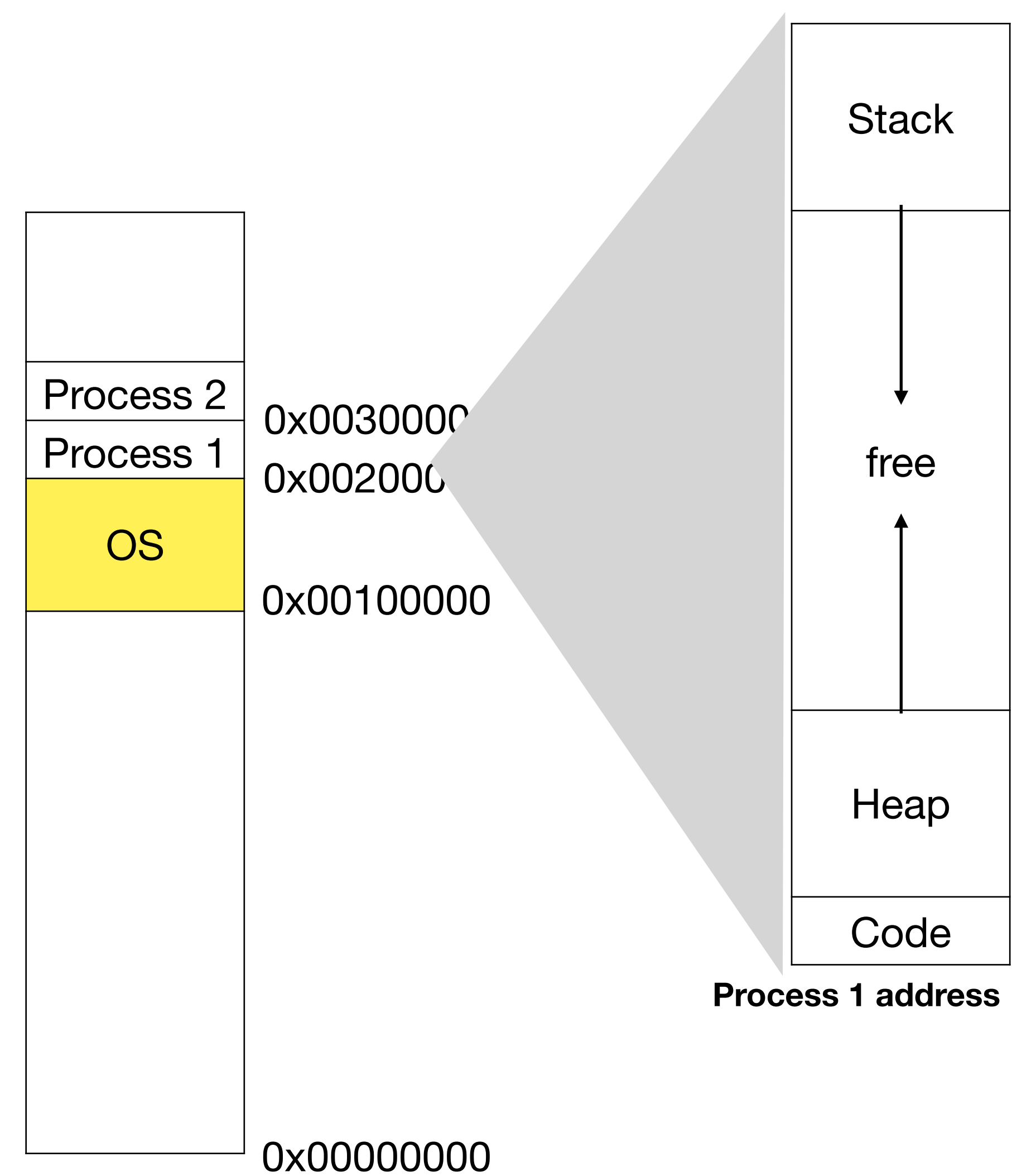
Allocating memory to processes

p17-free-space

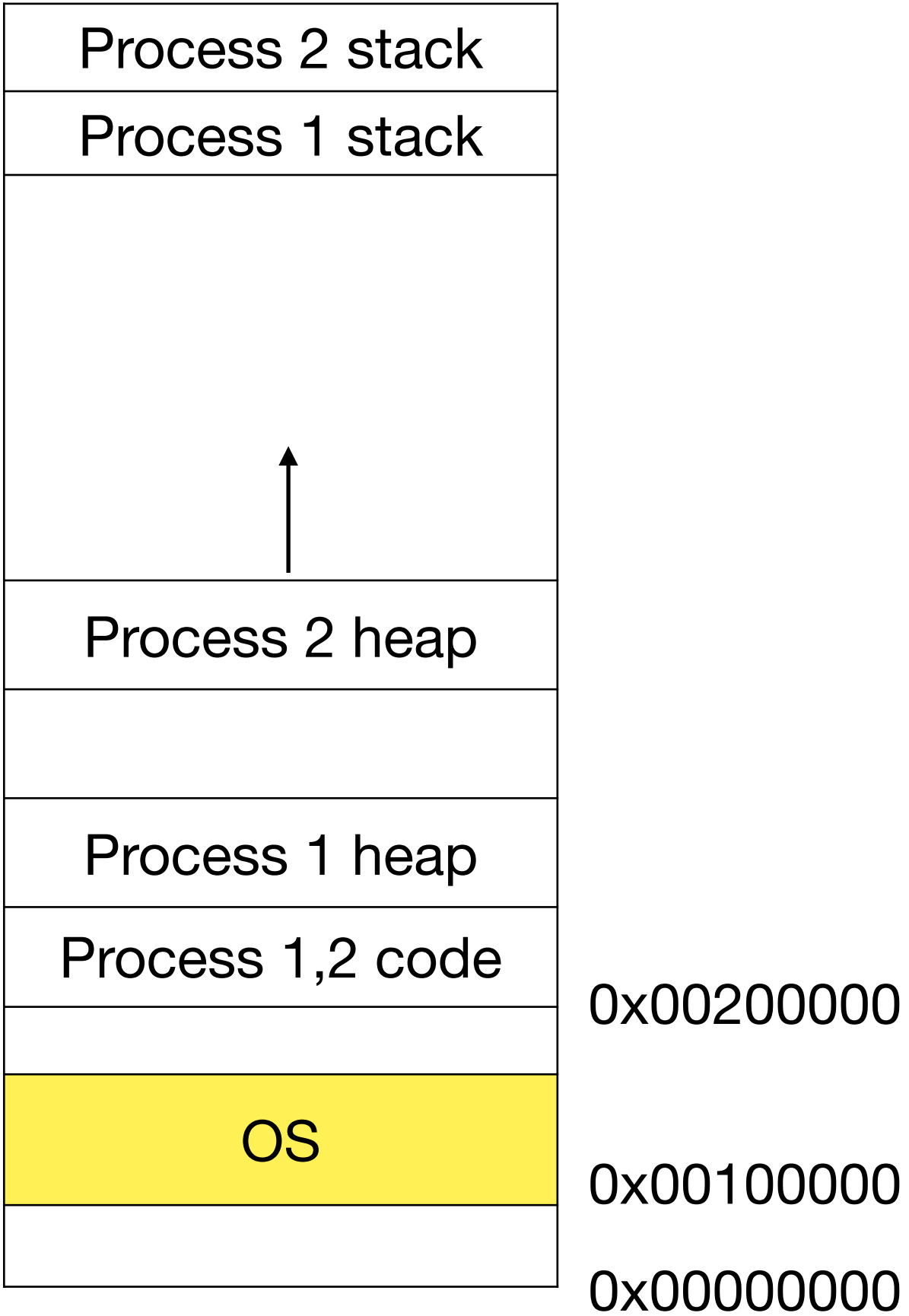
- Flat memory model: cs, ss, ds all point to the same memory segment
- Break memory into equal sized parts (say 1MB). OS allocates/deallocates at this granularity by maintaining a free list
- Create (delete) entries in GDT for the new process, finished process
- What if process needed more/less memory?



Segmentation hardware supports multi-segment model



- Place each segment independently
- Share code segments to save space
- Mark non-writeable
- Process can call `sbrk(N bytes)` system call to grow heap



External fragmentation

- After many processes start and exit, memory might become “fragmented” (similar to disk)
 - Example: cannot allocate 20 KB segment
- Unlike heap memory allocator, OS can do compaction: copy all allocated regions contiguously, update segment base and bound registers
 - Copying is expensive
 - Growing heap becomes not possible

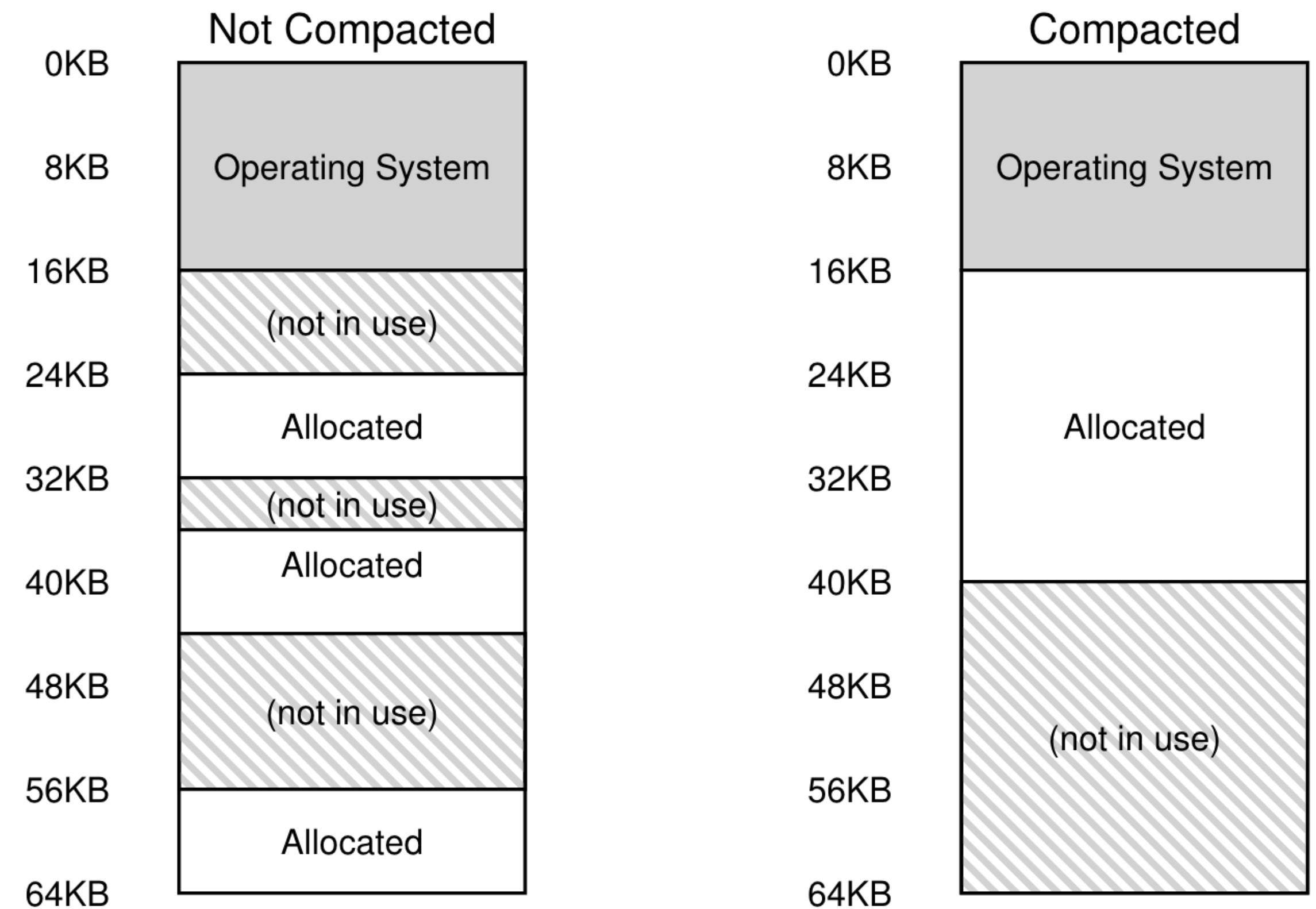


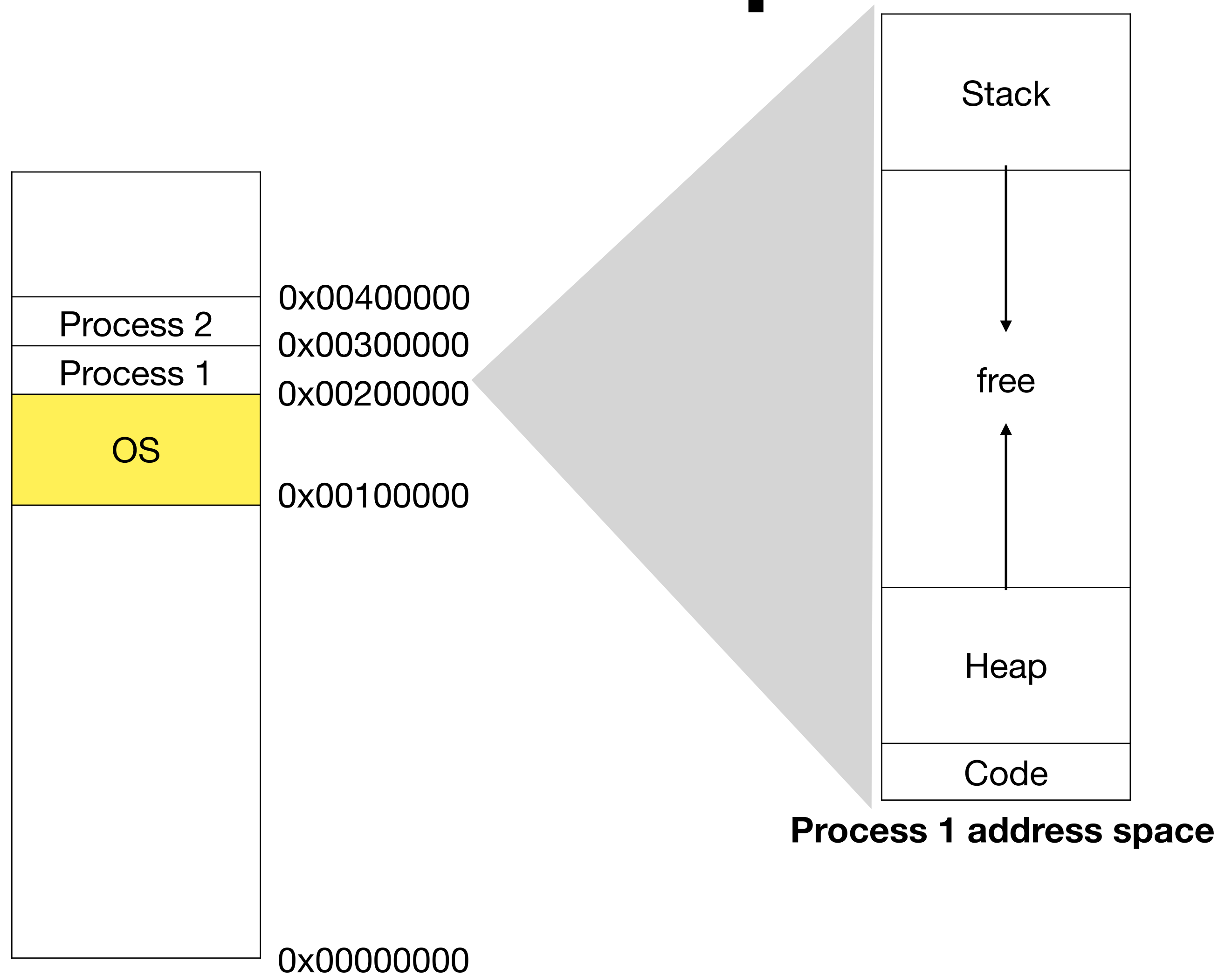
Figure 16.6: Non-compacted and Compacted Memory

Processes in action

xv6 Ch. 3: system calls, x86 protection, trap handlers

Memory isolation and address space

- Flat memory model



Protection

- Process cannot modify OS code since it is not in process' *address space*
- Process cannot modify GDT and IDT entries since GDT, IDT are not in its address space
- What stops process from calling lgdt and lidt instructions?
 - Ring 0: kernel mode. Ring 3: user mode
 - lgdt and lidt are *privileged instructions* only callable from “ring 0”

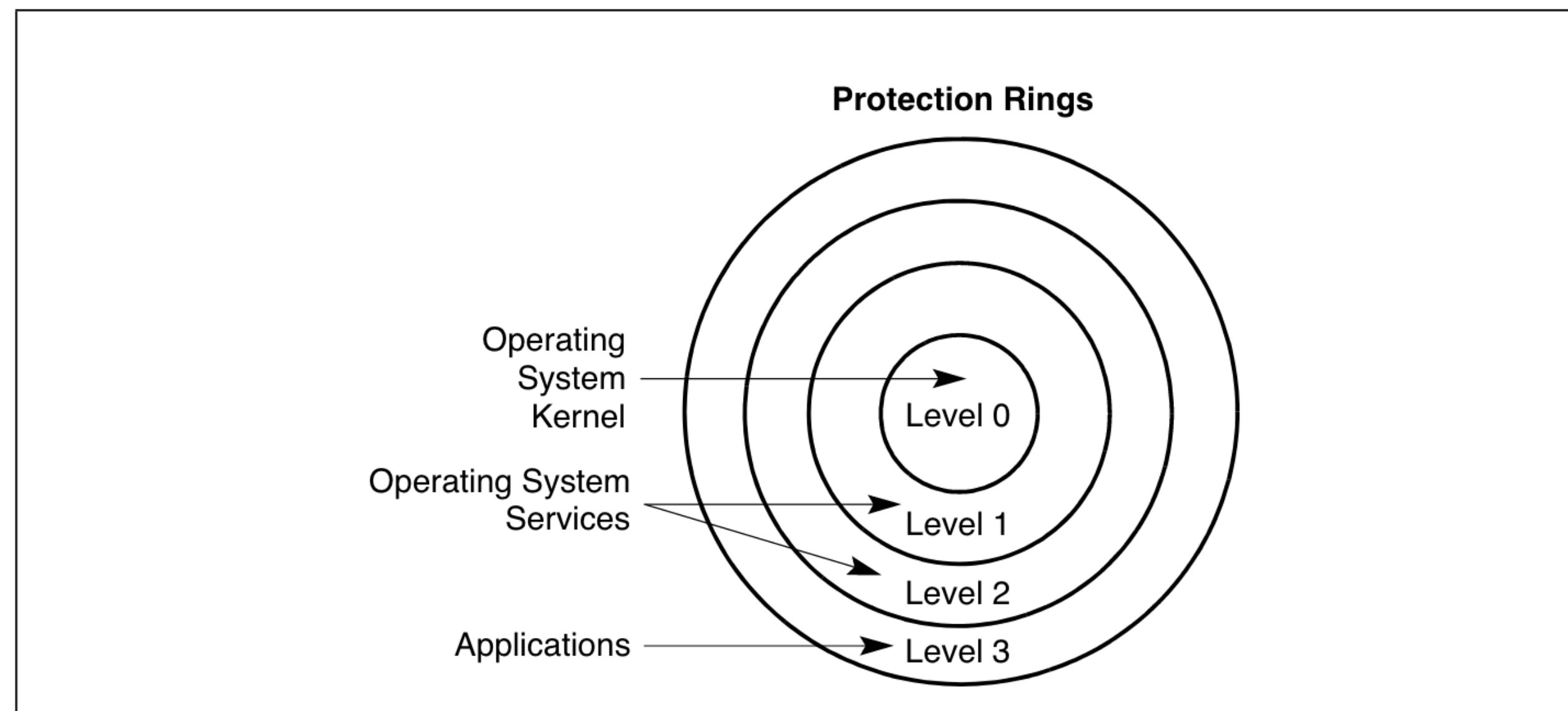
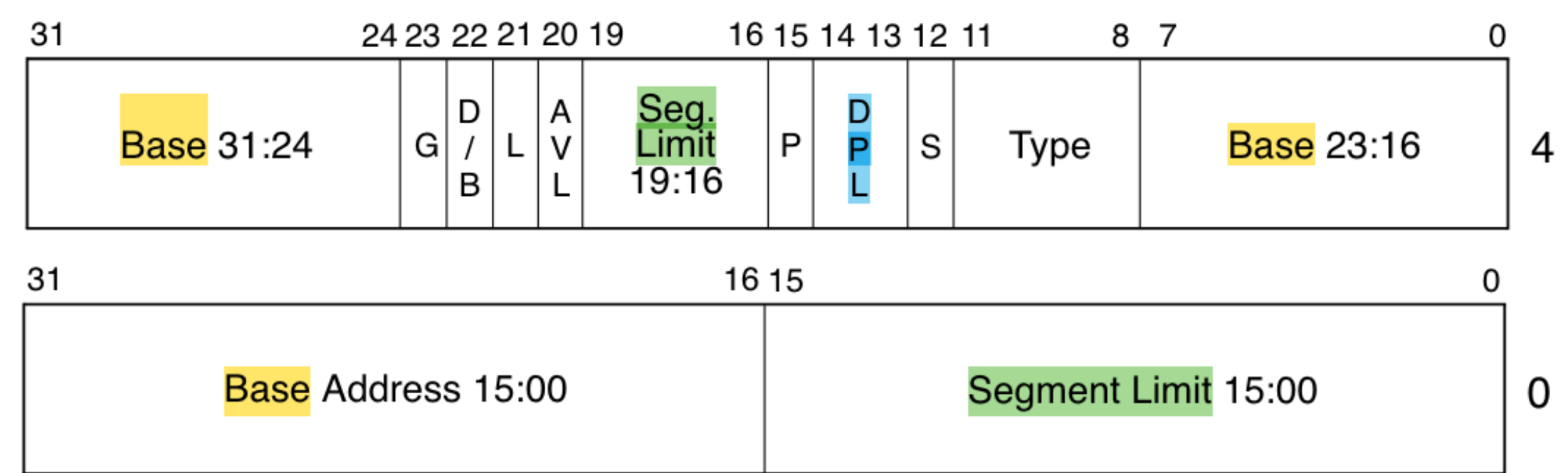


Figure 5-3. Protection Rings

How does hardware know the current privilege level?

- Two LSBs of %cs determine “current privilege level” (CPL)
- Two LSBs of other segment selectors specify “requested privilege level” (RPL)
- Segment descriptor in GDT specifies “descriptor privilege level” (DPL)



L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

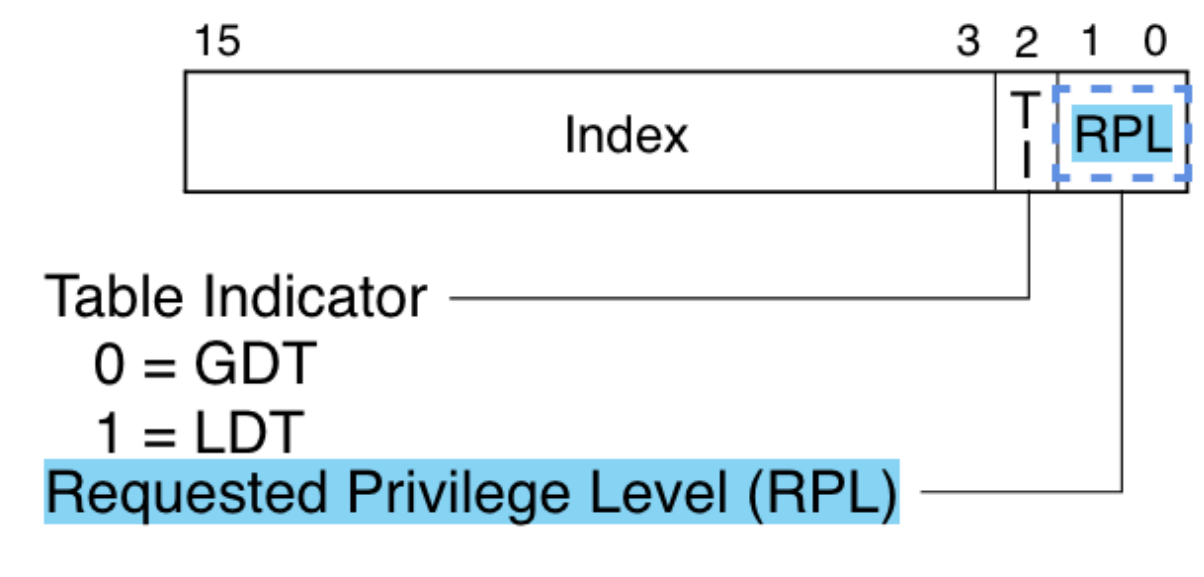


Figure 3-6. Segment Selector

How does hardware enforce privilege levels?

- Privileged instructions can only be called when CPL=0
- Programs cannot lower their CPL directly
- INT instruction causes a software interrupt to set ring = 0
- For data access:
 $CPL \leq DPL$ and $RPL \leq DPL$

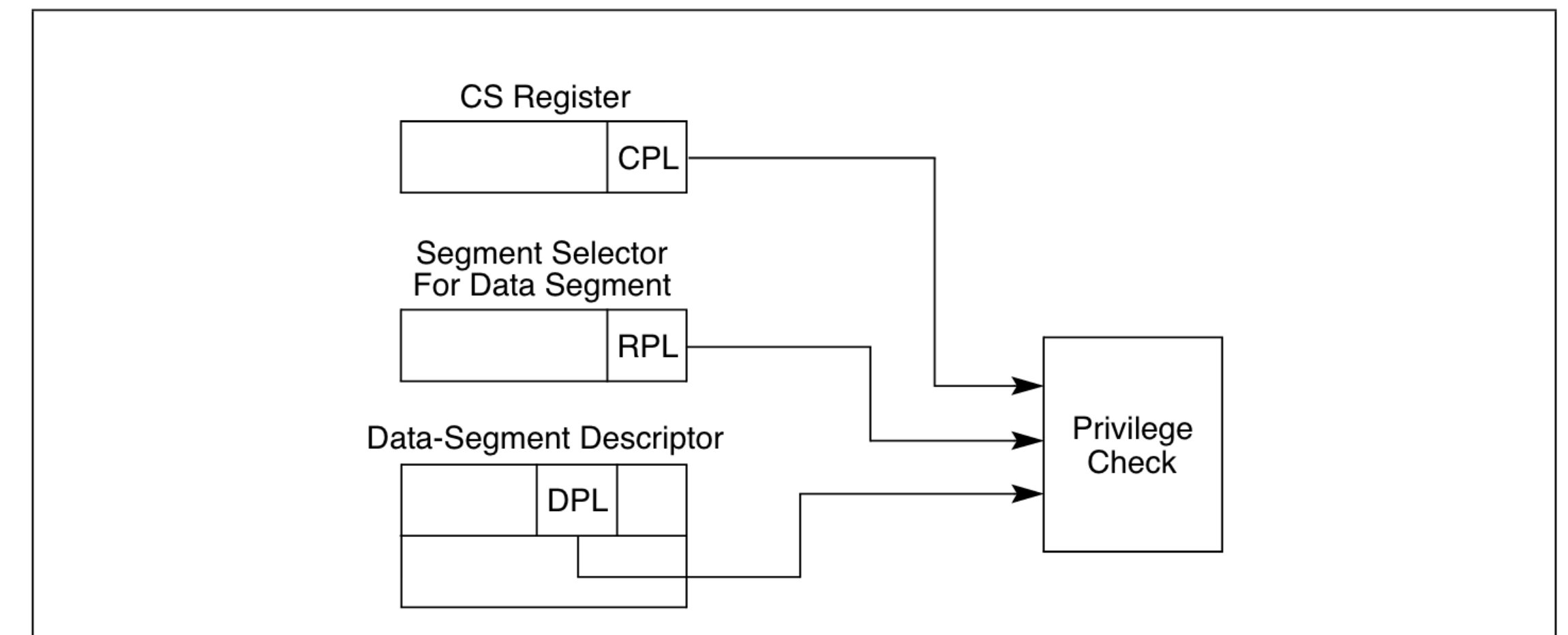


Figure 5-4. Privilege Check for Data Access

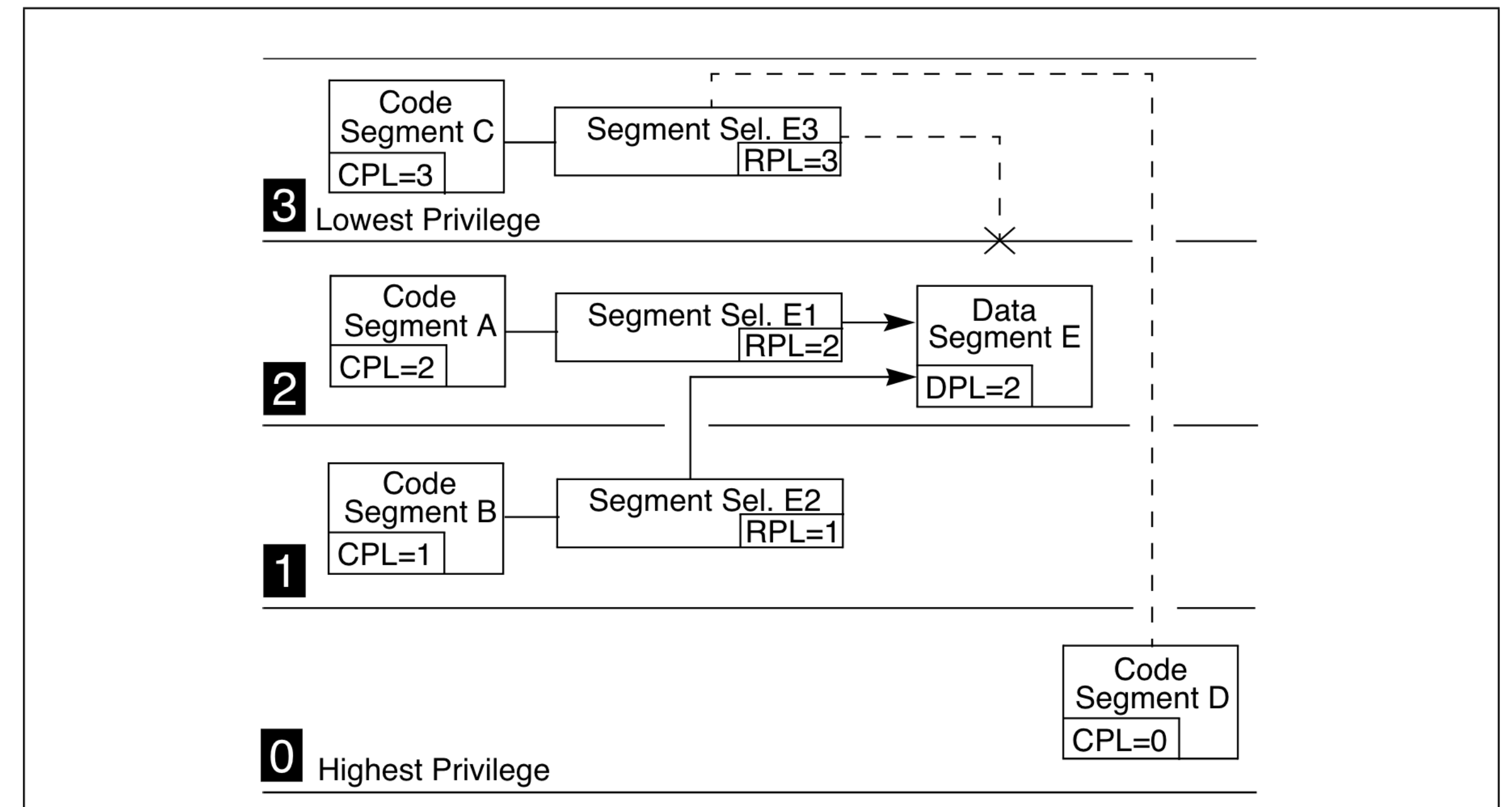
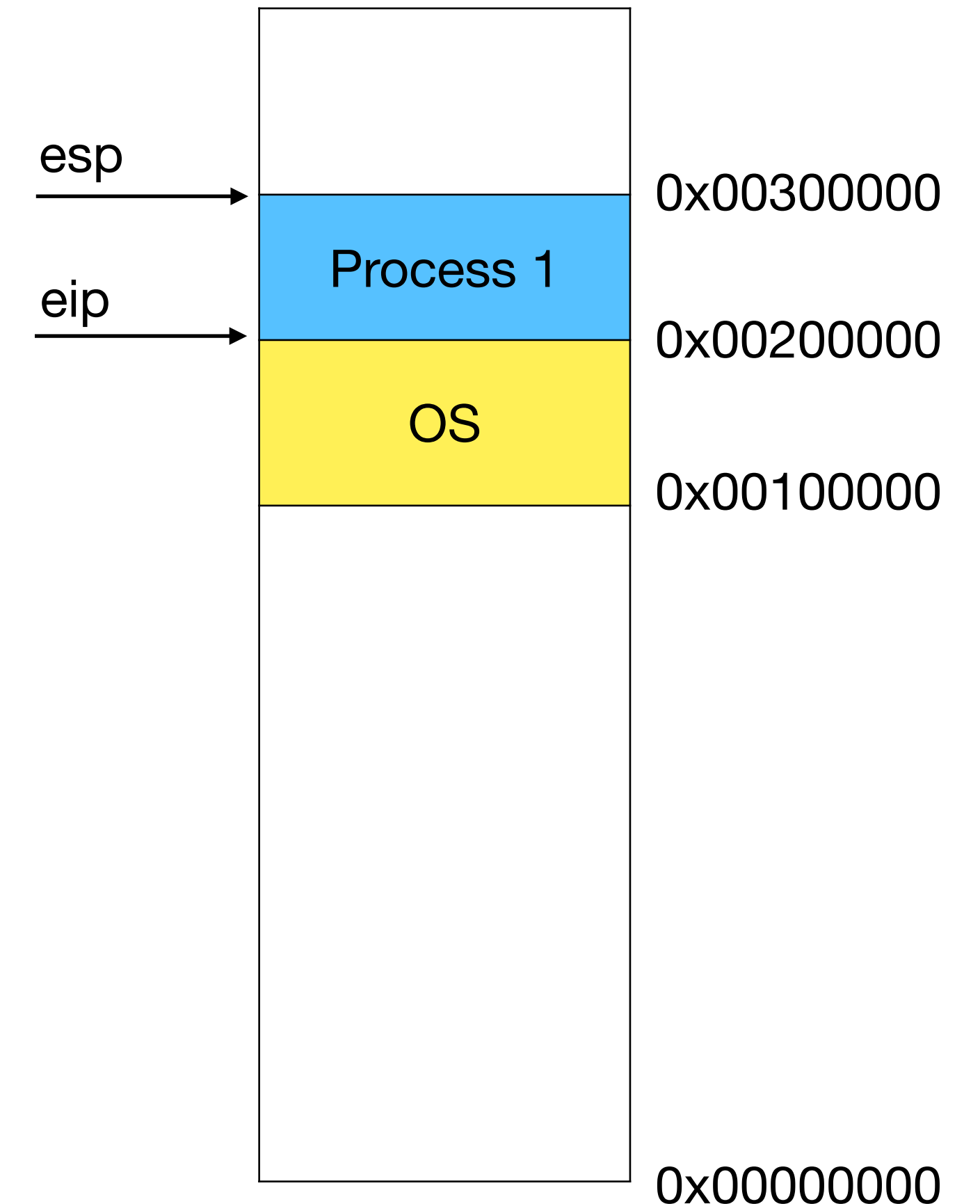


Figure 5-5. Examples of Accessing Data Segments From Various Privilege Levels

Setting up our first process in xv6!

p15-initproc

- Link initcode binary inside the kernel image
- main.c calls seginit, then pinit, then scheduler.
- seginit in vm.c creates code (UCODE) and data segments (UDATA) from STARTPROC (2MB) to 3MB. Flat memory model!
- pinit in proc.c copies program binary to STARTPROC and sets cs,ds,ss,es to the program's segments. Last two bits are set to DPL_USER. Sets eip=0, esp=1MB. Enables interrupt flag in eflags. proc.c maintains list of processes in ptable. We are just starting one process for now.
- scheduler in proc.c selects RUNNABLE process and switches to it.
- Process cannot change GDT entries, IDT entries since they are not in address space of the process! Process cannot call lgdt, lidt since it will run in ring 3.



Link initcode binary inside the OS image

- Make the **entry point as start**, and link the text region **at address 0**
- Copy out the binary (drop ELF header etc.), **into the kernel**, i.e., not as ELF
- Linker generates **_binary_initcode_start**, etc. for OS code to locate the binary

```
.globl start initcode.S
start:
    add $1, %eax
    movl %eax, 0x10001
    jmp start
```

```
initcode: initcode.S Makefile
$(CC) $(CFLAGS) -nostdinc -I. -c initcode.S
$(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
$(OBJCOPY) -S -O binary initcode.out initcode

kernel: $(OBJS) entry.o kernel.ld initcode
$(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary initcode
```

```
kernel.sym
00105418 _binary_initcode_start
0000000a _binary_initcode_size
00105422 _binary_initcode_end
```

```
void pinit(void) { proc.c
    extern char _binary_initcode_start[], _binary_initcode_size[];
    ..
}
```


Setup segments for our process

- Initialize GDT entries for user **code** and **data** segments from **2MB-3MB** with **privilege level 3**

```
int main(void) { // main.c
    ...
    seginit();
    pinit();
    scheduler();
}
```

```
#define DPL_USER    0x3           // User DPL // vm.c
#define STARTPROC  0x200000      // Start allocating process from here
#define PROCSIZE    0x100        // 1MB is the size of each process

void seginit(void) {
    struct cpu *c;
    c->gdt[SEG_KCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, 0);
    c->gdt[SEG_KDATA] = SEG(STA_W, 0, 0xffffffff, 0);
    c->gdt[SEG_UCODE] = SEG(STA_X|STA_R, STARTPROC, (PROCSIZE << 12), DPL_USER);
    c->gdt[SEG_UDATA] = SEG(STA_W, STARTPROC, (PROCSIZE << 12), DPL_USER);
    lgdt(c->gdt, sizeof(c->gdt));
}
```

Understanding swtch

```
pinit(){
    p = allocproc();
    memmove(p->offset, _binary_initcode_start,);
    p->tf->ds,es,ss = (SEG_UDATA<<3) | DPL_USR;
    p->tf->cs = (SEG_UCODE<<3) | DPL_USR;
    p->tf->eflags = FL_IF;
    p->tf->eip = 0;
}
```

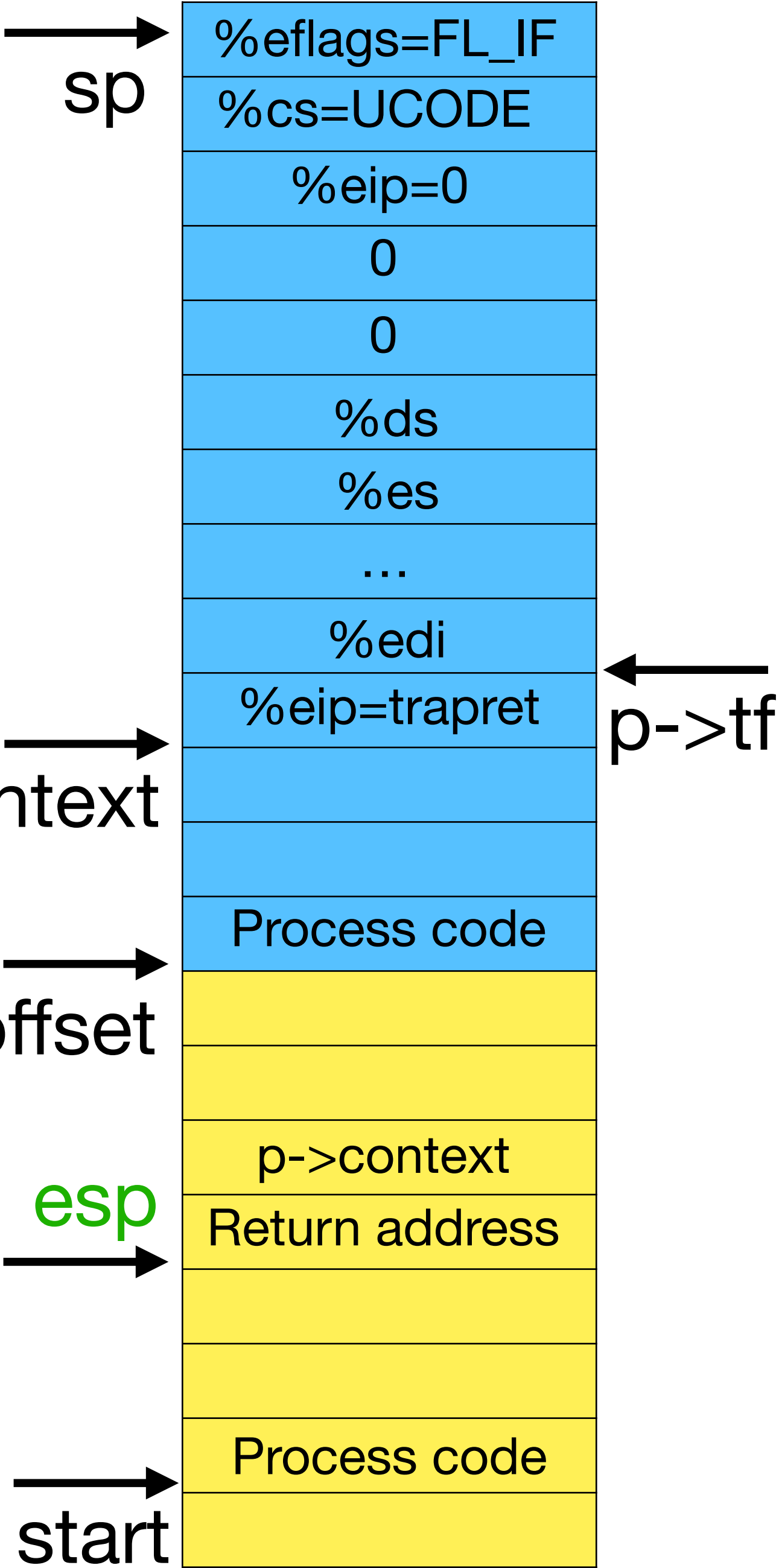
```
allocproc() {
    eip → sp = (char*)(STARTPROC + (PROCSIZE<<12));
    sp -= sizeof *p->tf;
    p->tf = (struct trapframe*)sp;
    sp -= sizeof *p->context;
    p->context = (struct context*)sp;
    p->context->eip = (uint)trapret;
    return p;
}
```

```
scheduler() {
    ...
    swtch(p->context);
}
```

```
swtch:
    movl 4(%esp), %eax
    movl %eax, %esp
    movl $0, %eax
    ret
```

```
trapret:
    popal
    popl %gs
    popl %fs
    popl %es
    popl %ds
    addl $0x8, %esp
    iret
```

_binary_initcode_start



Interrupt handling revisited

eip → for(;;)
;

trap.c

```
void
trap(struct trapframe *tf)
{
    switch(tf->trapno){
    case T_IRQ0 + IRQ_TIMER:
        ticks++;
        cprintf("Tick! %d\n\0", ticks);
        lapiceoi();
    ..
    }
    return
}
```

vectors.S

```
.globl vector0
vector0:
    pushl $0
    pushl $0
    jmp alltraps
```

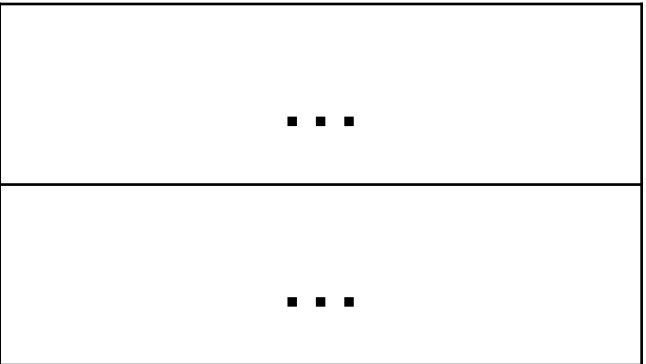
trapasm.S

```
alltraps:
    pushal
    pushl %esp
    call trap
    addl $4, %esp
    popal
    addl $0x8, %esp
    iret
```

IDT

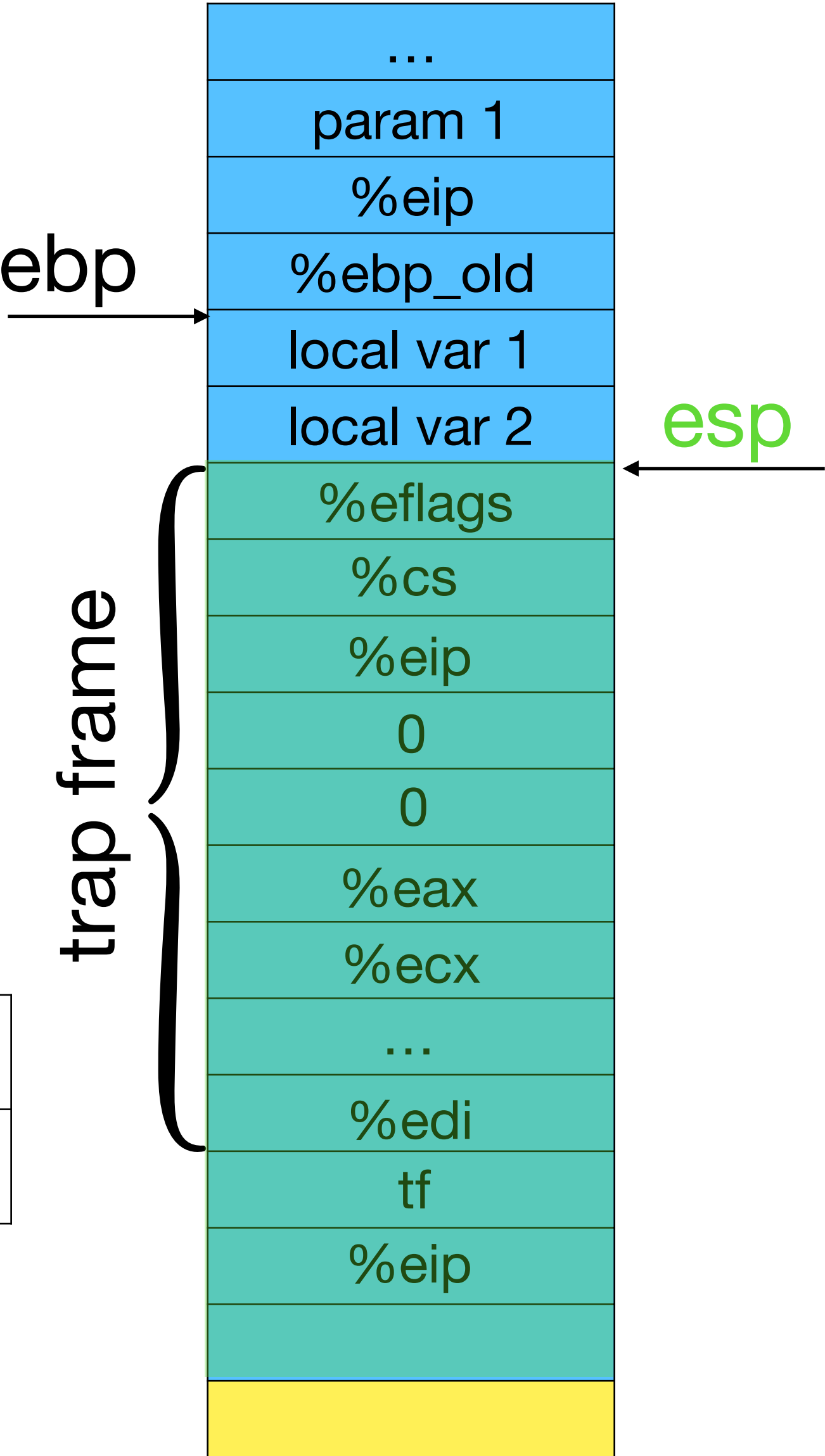


GDT



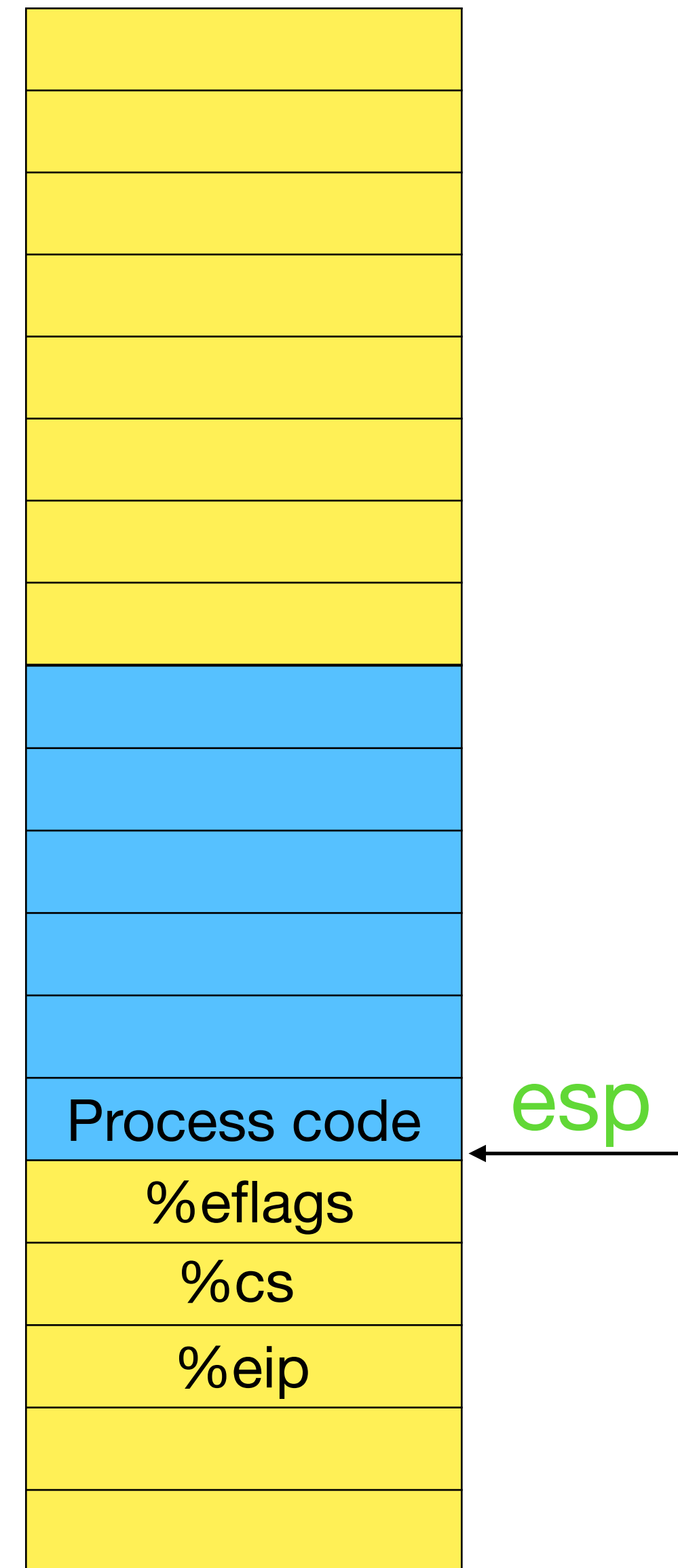
CS →

Stack



Interrupt handling problem

- We cannot trust %esp of the process. Hardware might write (%eflags, %cs, %eip) into another process' address space or into OS memory.
- Use a separate stack set up by the OS!



OS tells hardware where the kernel stack is

- In task-state segment (TSS), OS sets up
 - Stack segment for transition to ring 0 in SS0 and
 - Stack pointer for transition to ring 0 in ESP0

31	15	0	
I/O Map Base Address		Reserved	T 100
Reserved		LDT Segment Selector 96	
Reserved		GS 92	
Reserved		FS 88	
Reserved		DS 84	
Reserved		SS 80	
Reserved		CS 76	
Reserved		ES 72	
EDI			68
ESI			64
EBP			60
ESP			56
EBX			52
EDX			48
ECX			44
EAX			40
EFLAGS			36
EIP			32
CR3 (PDBR)			28
Reserved		SS2 24	
ESP2			20
Reserved		SS1 16	
ESP1			12
Reserved		SS0 8	
ESP0			4
Reserved		Previous Task Link 0	

Reserved bits. Set to 0.

Figure 7-2. 32-Bit Task-State Segment (TSS)

OS tells hardware where the kernel stack is (2)

- OS sets up an entry in GDT that contains location of TSS
- LTR instruction changes task register

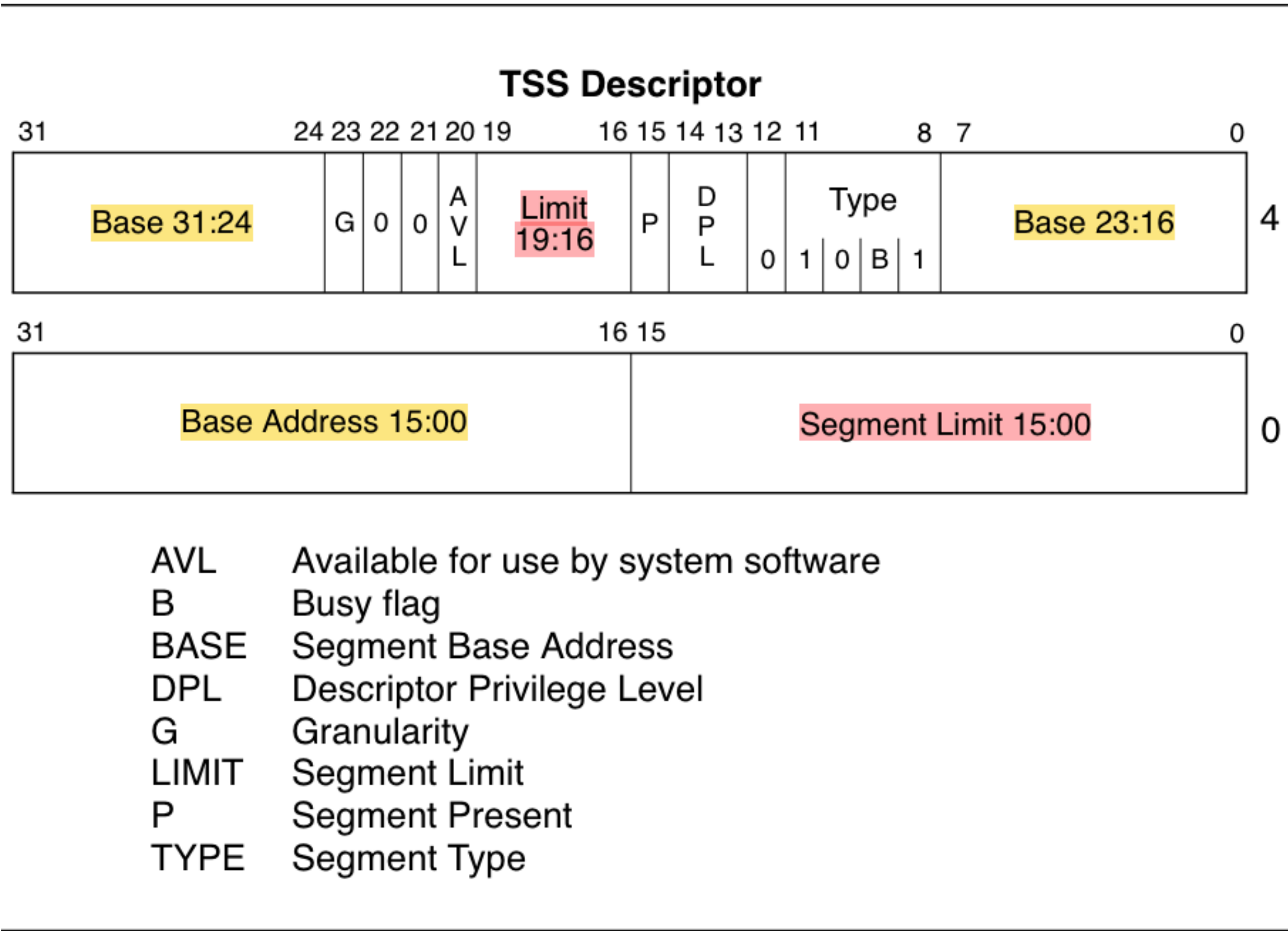


Figure 7-3. TSS Descriptor

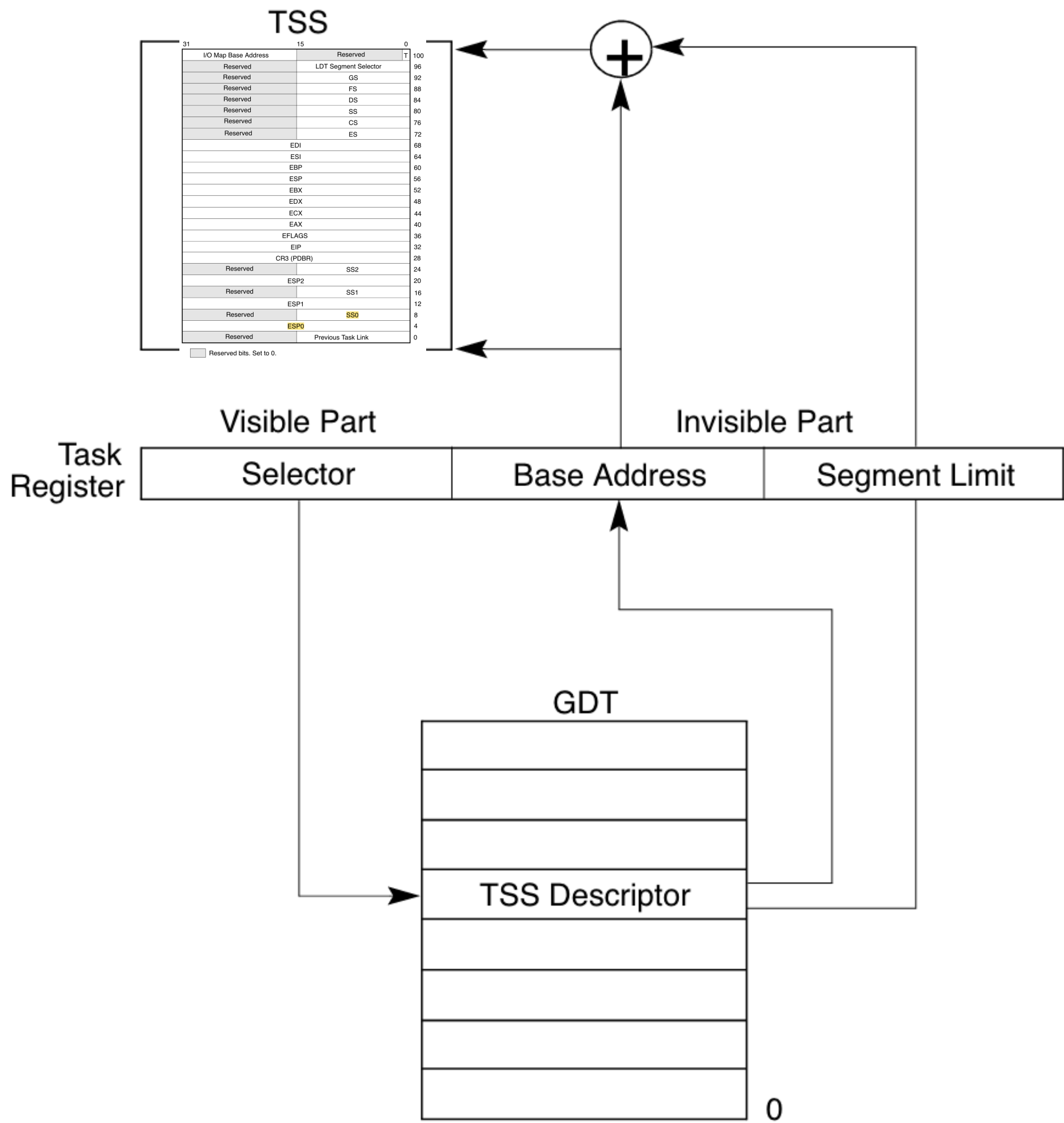


Figure 7-5. Task Register

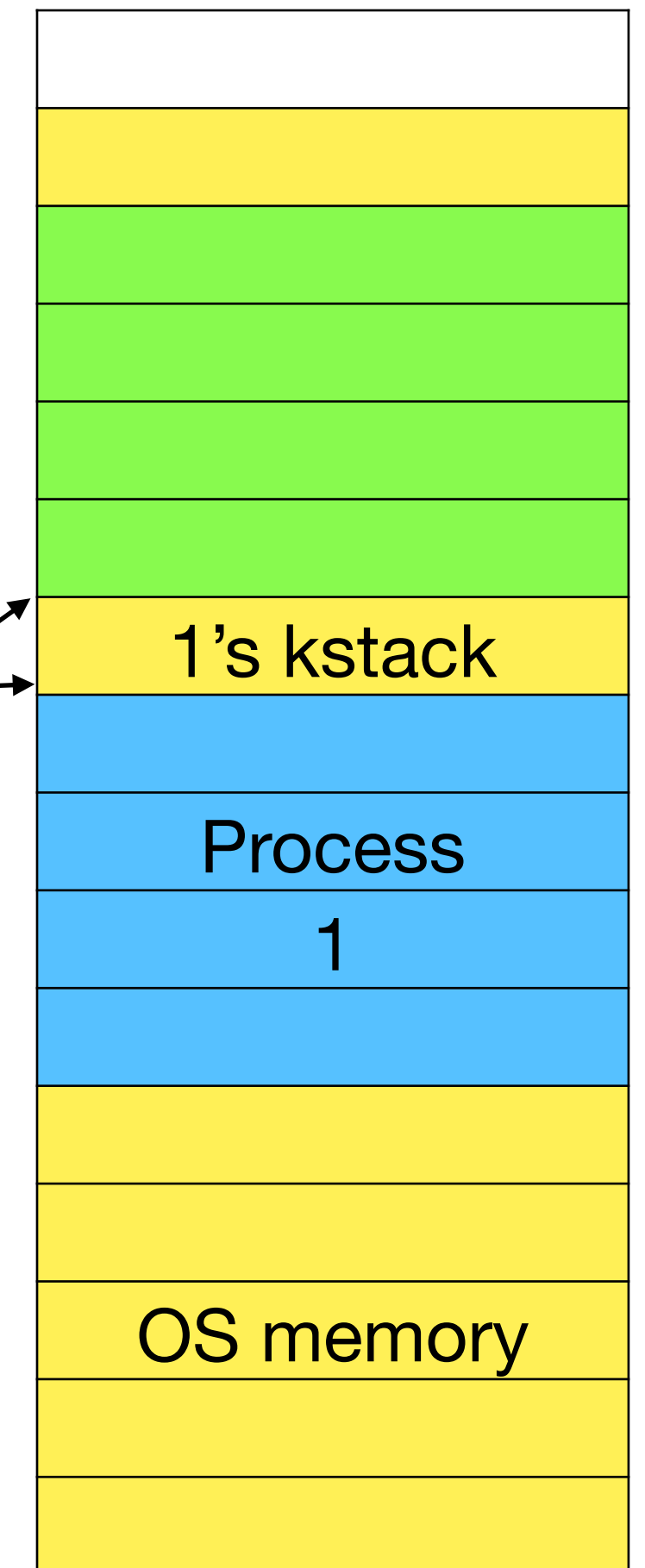
Code walkthrough: setting up kernel stack p16-tss

```
void seginit(void) {
    c->gdt[SEG_UCODE] = SEG(.., STARTPROC, (PROCSIZE-1) << 12, DPL_USER);
    c->gdt[SEG_UDATA] = SEG(.., STARTPROC, (PROCSIZE-1) << 12, DPL_USER);
}

static struct proc* allocproc(void) {
    sp = (char*)(STARTPROC + (PROCSIZE<<12));
    p->kstack = sp - KSTACKSIZE;
}

void scheduler(void) {
    // pick RUNNABLE process p
    switchvm(p);
    swtch(p->context);
}

void switchvm(struct proc *p) {
    mycpu()->gdt[SEG_TSS] = SEG16(STS_T32A, &mycpu()->ts,
                                sizeof(mycpu()->ts)-1, 0);
    mycpu()->ts.ss0 = SEG_KDATA << 3;
    mycpu()->ts.esp0 = (uint)p->kstack + KSTACKSIZE;
    ltr(SEG_TSS << 3);
}
```



Interrupt handling

- Each IDT entry is 64-bits. Contains **code segment** and **eip**
- When interrupt appears,
 - Hardware changes **SS and ESP** according to the TSS
 - Hardware changes CS and EIP to the one pointed by IDT entry.

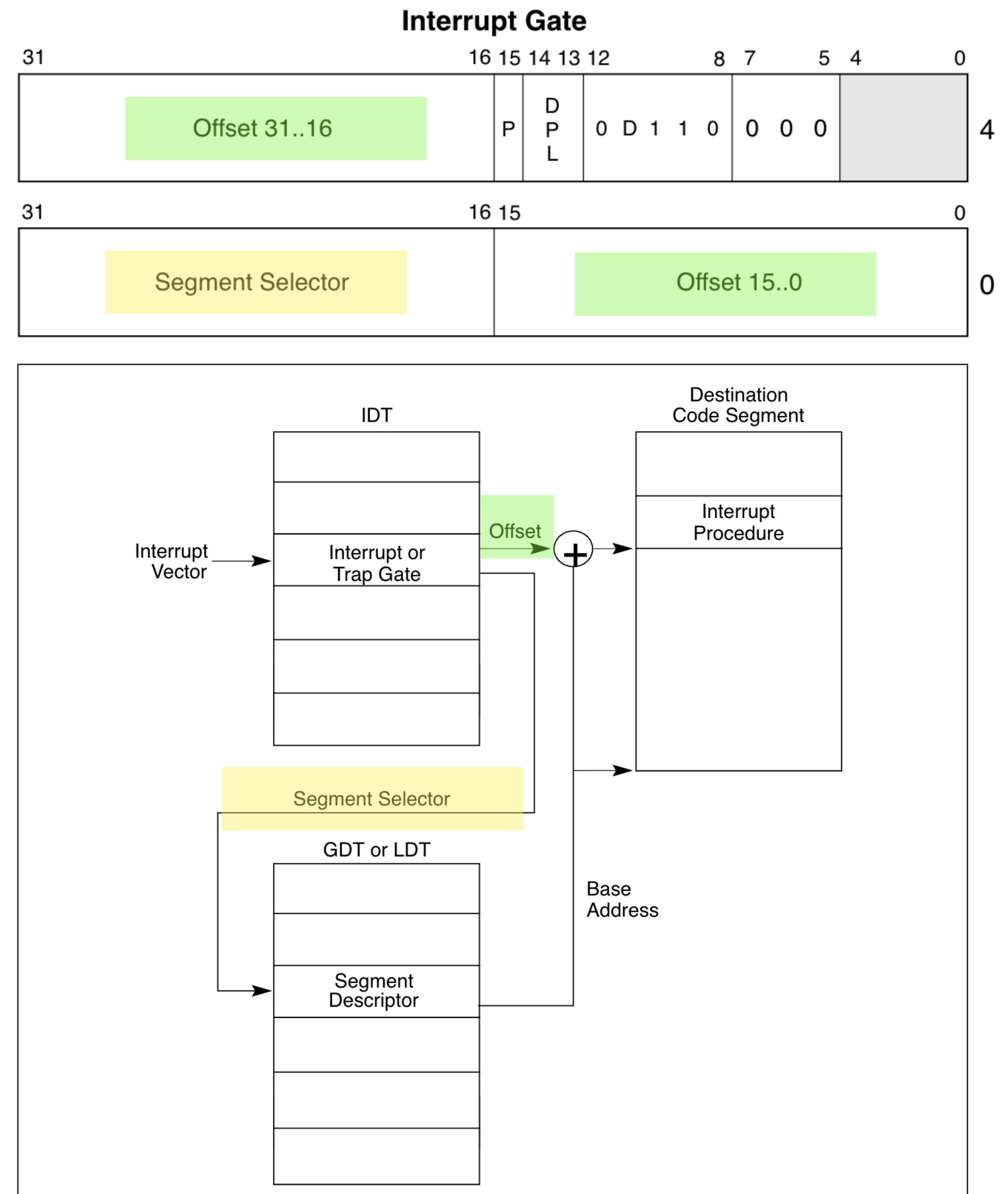


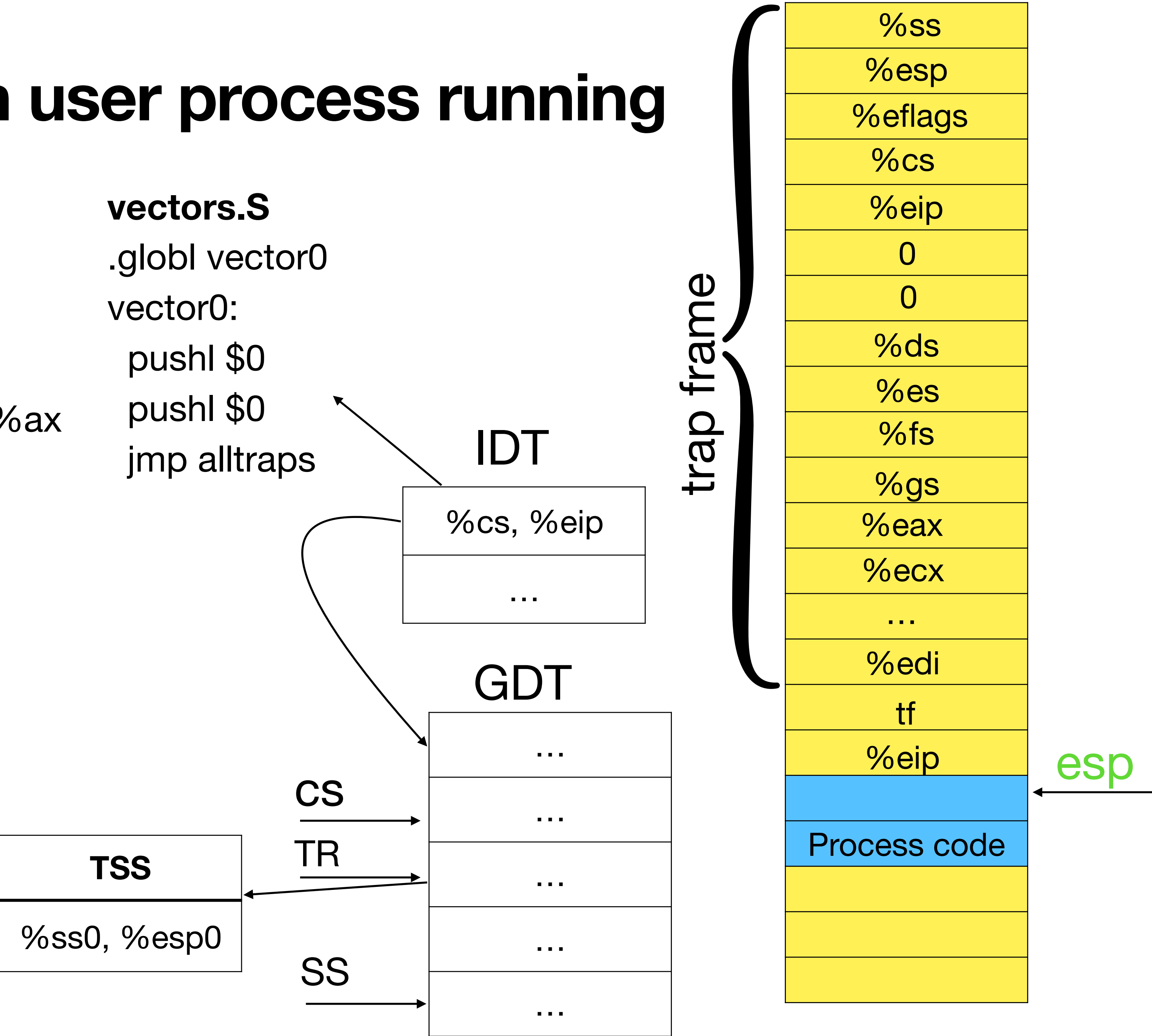
Figure 6-3. Interrupt Procedure Call

Interrupt handling with user process running

eip → for(;;)
;

```
trapasm.S
alltraps:
    pushl %ds..
    pushal
    movw $(SEG_KDATA<<3), %ax
    movw %ax, %ds..
    pushl %esp
    call trap
    addl $4, %esp
    popal
    popl %ds..
    addl $0x8, %esp
    iret
```

```
vectors.S
.globl vector0
vector0:
    pushl $0
    pushl $0
    jmp alltraps
```

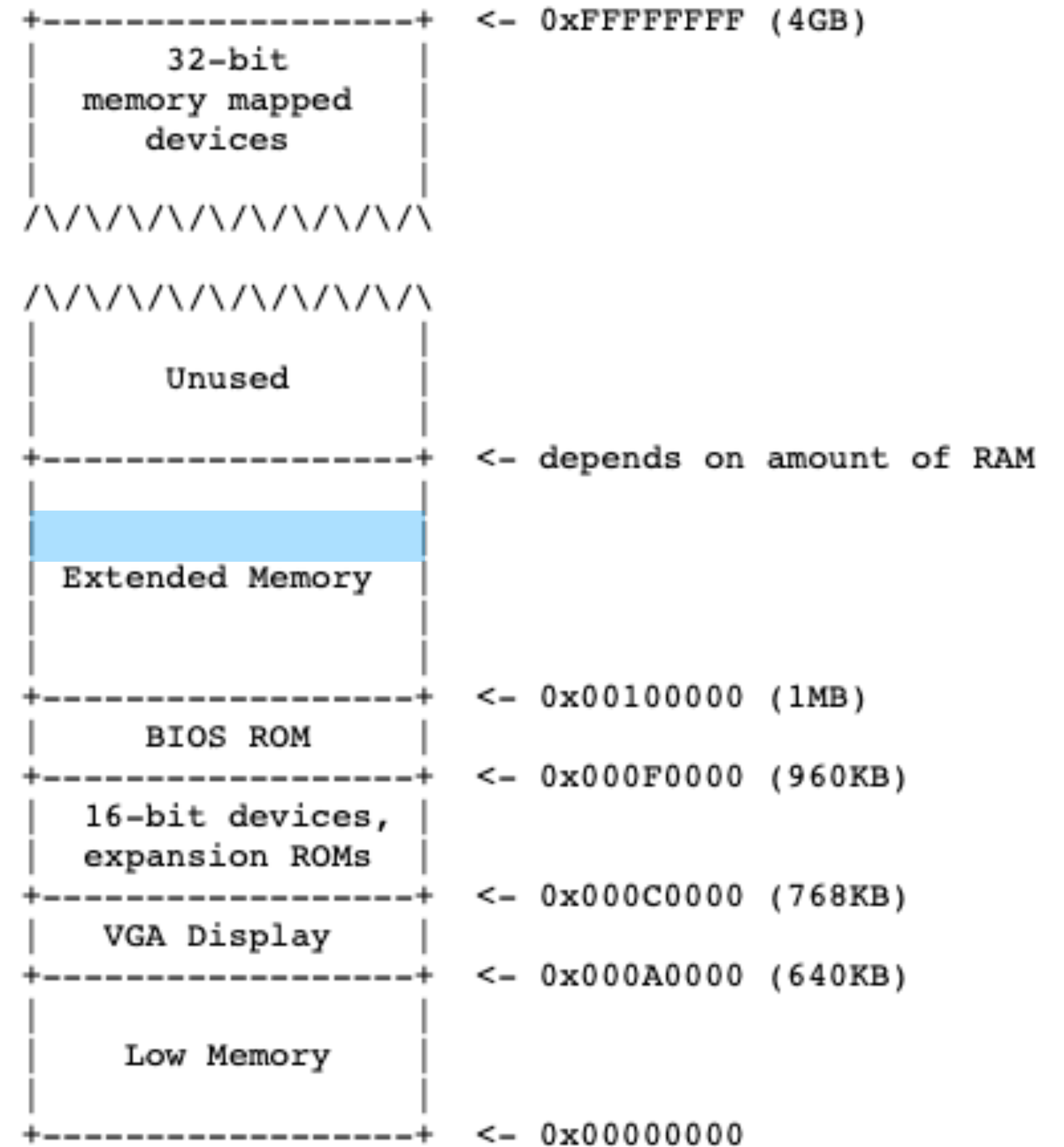


Protection so far

- Can processes read/write each other's memory?
 - No! The other process' memory is not *addressable* by (not in the *address space* of) the process
- Can process take over IDT, GDT, TSS?
 - No! IDT, GDT, TSS are not in process' address space.
 - Process cannot run LIDT, LGDT, LTR instructions since they are *privileged instructions* (only runnable from ring 0)
- Can process change its privilege to ring 0?
 - Not directly. It can change only via trap handling mechanism. OS code runs upon a trap. Trap handling writes to kernel stack.
- Can process run away with the CPU?
 - CPU will be snatched after timer interrupt. Process cannot disable interrupts

IO protection

- Memory mapped IO
 - Can directly read from / write to IO devices if OS keeps it in *process address space*



Port-mapped IO protection

```
inb(0x1F7), outb(0x1F2, 1), ..
```

- Option 1: Make in and out instruction privileged. Processes must do IO via kernel.
- Expensive user mode-kernel mode transitions for IO intensive processes.
- Option 2: Set EFLAGS.IOPL = 3
 - Processes (CPL=3) cannot modify EFLAGS.IOPL
 - Gives permission to access all IO ports

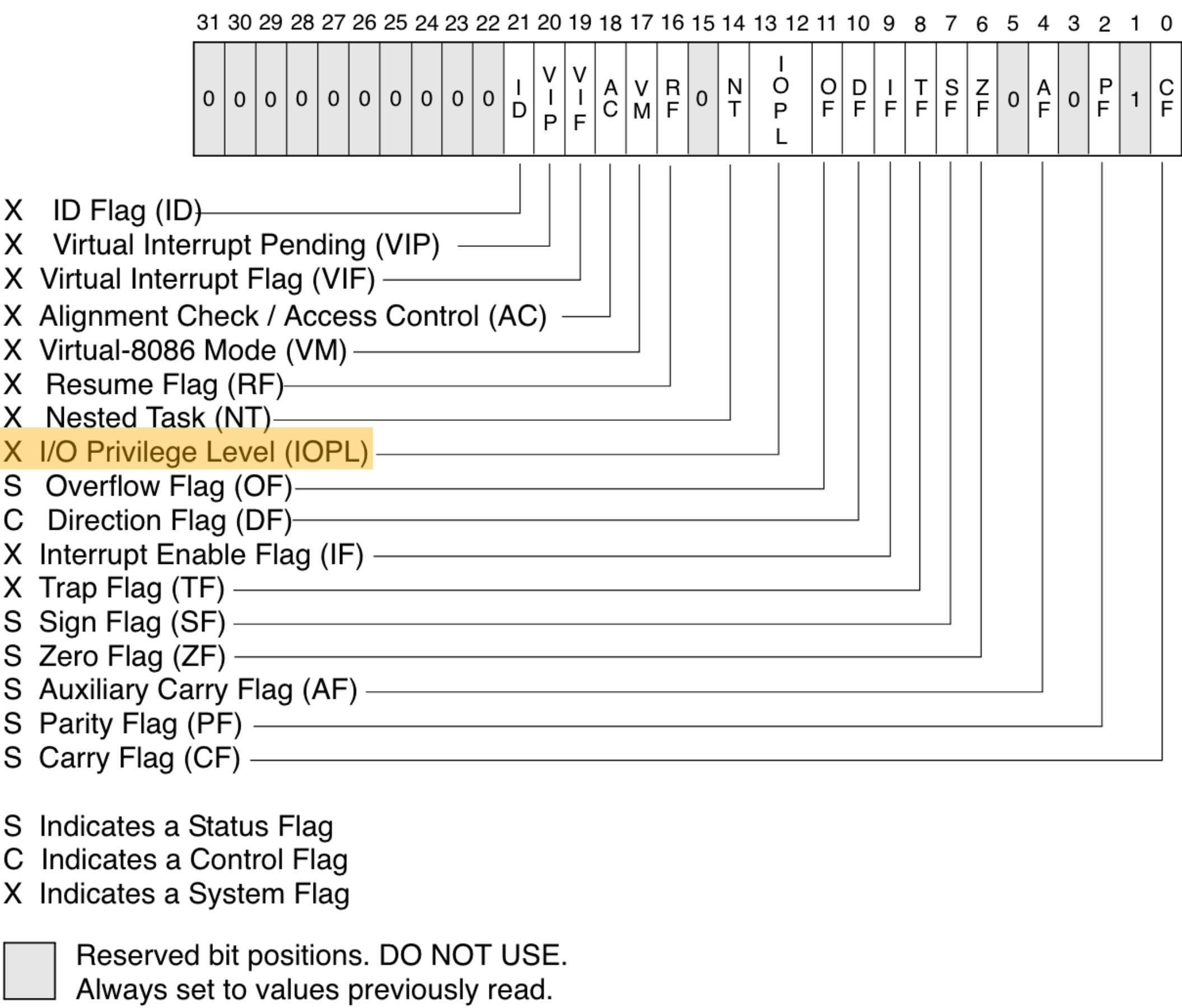


Figure 3-8. EFLAGS Register

Port-mapped IO protection (2)

- Option 3: If $CPL > EFLAGS.IOPL$, use a bitmap to decide if in, out instruction should generate a trap
- Example: `outb(41, 1)` is allowed if 41st bit is set to 0

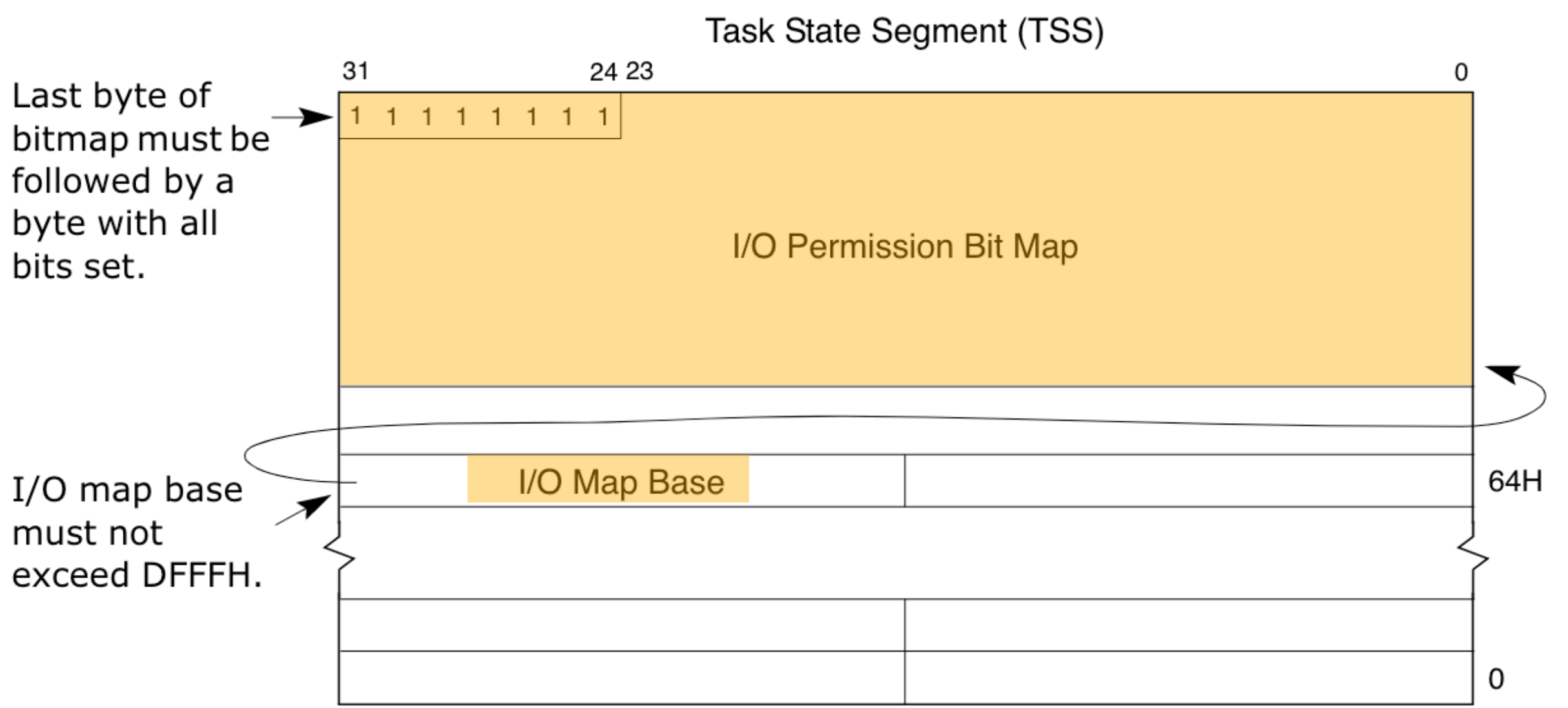


Figure 18-2. I/O Permission Bit Map

31	15	0	
I/O Map Base Address		Reserved	T 100
Reserved		LDT Segment Selector	96
Reserved		GS	92
Reserved		FS	88
Reserved		DS	84
Reserved		SS	80
Reserved		CS	76
Reserved		ES	72
EDI			68
ESI			64
EBP			60
ESP			56
EBX			52
EDX			48
ECX			44
EAX			40
EFLAGS			36
EIP			32
CR3 (PDBR)			28
Reserved		SS2	24
ESP2			20
Reserved		SS1	16
ESP1			12
Reserved		SS0	8
ESP0			4
Reserved		Previous Task Link	0

Reserved bits. Set to 0.

Figure 7-2. 32-Bit Task-State Segment (TSS)

IO protection in xv6

```
void pinit(void) {
```

```
    ..
```

```
    p->tf->eflags = FL_IF;
```

```
}
```

```
void switchvm(struct proc *p) {
```

```
    mycpu()->gdt[SEG_TSS] = SEG16(STS_T32A, &mycpu()->ts,  
                                  sizeof(mycpu()->ts)-1, 0);
```

```
    mycpu()->ts.ss0 = SEG_KDATA << 3;
```

```
    mycpu()->ts.esp0 = (uint)p->kstack + KSTACKSIZE;
```

```
    // setting IOPL=0 in eflags *and* iomb beyond the tss segment limit
```

```
    // forbids I/O instructions (e.g., inb and outb) from user space
```

```
    mycpu()->ts.iomb = (ushort) 0xFFFF;
```

```
    ltr(SEG_TSS << 3);
```

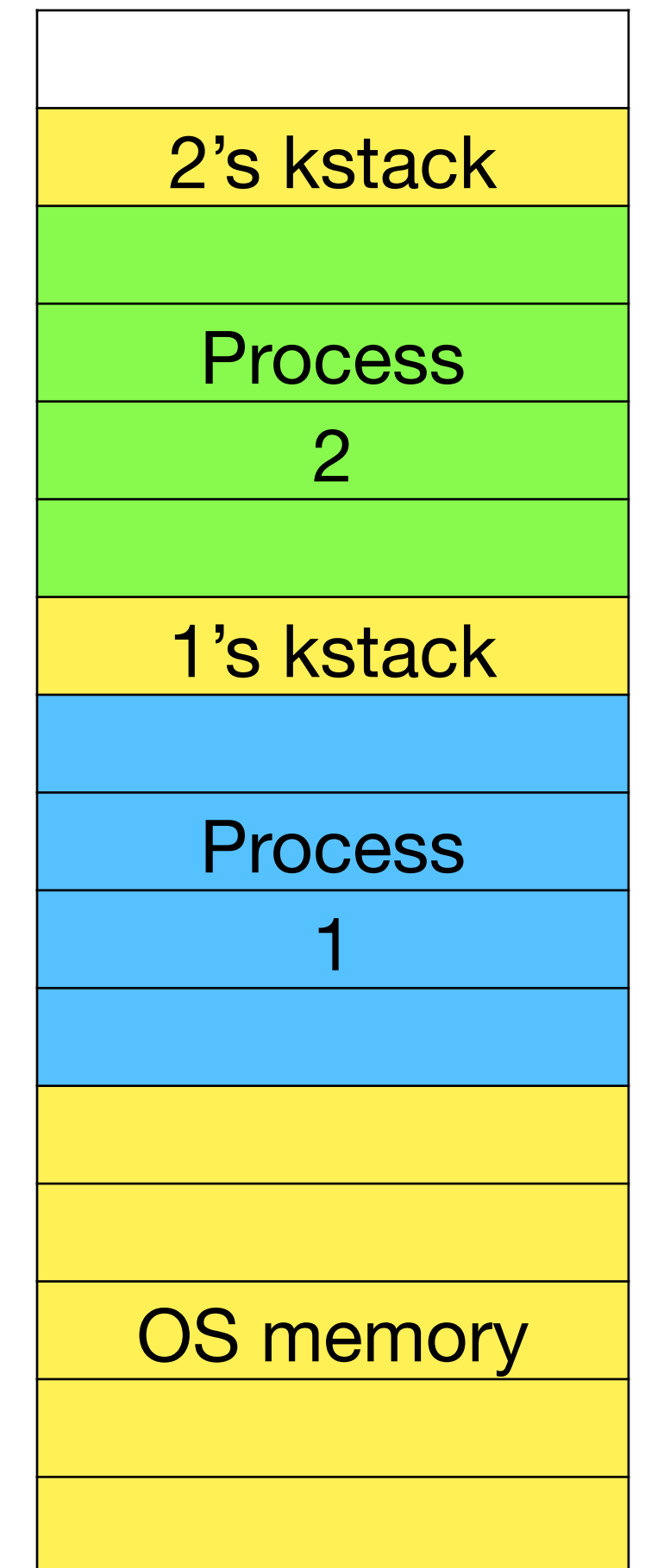
```
}
```

- Processes **cannot** directly do IO
- MMIO: Do not map IO addresses in process address space
- PIO: Running in and out instructions will generate a trap (jump to OS)

Code walkthrough: Setting up multiple processes

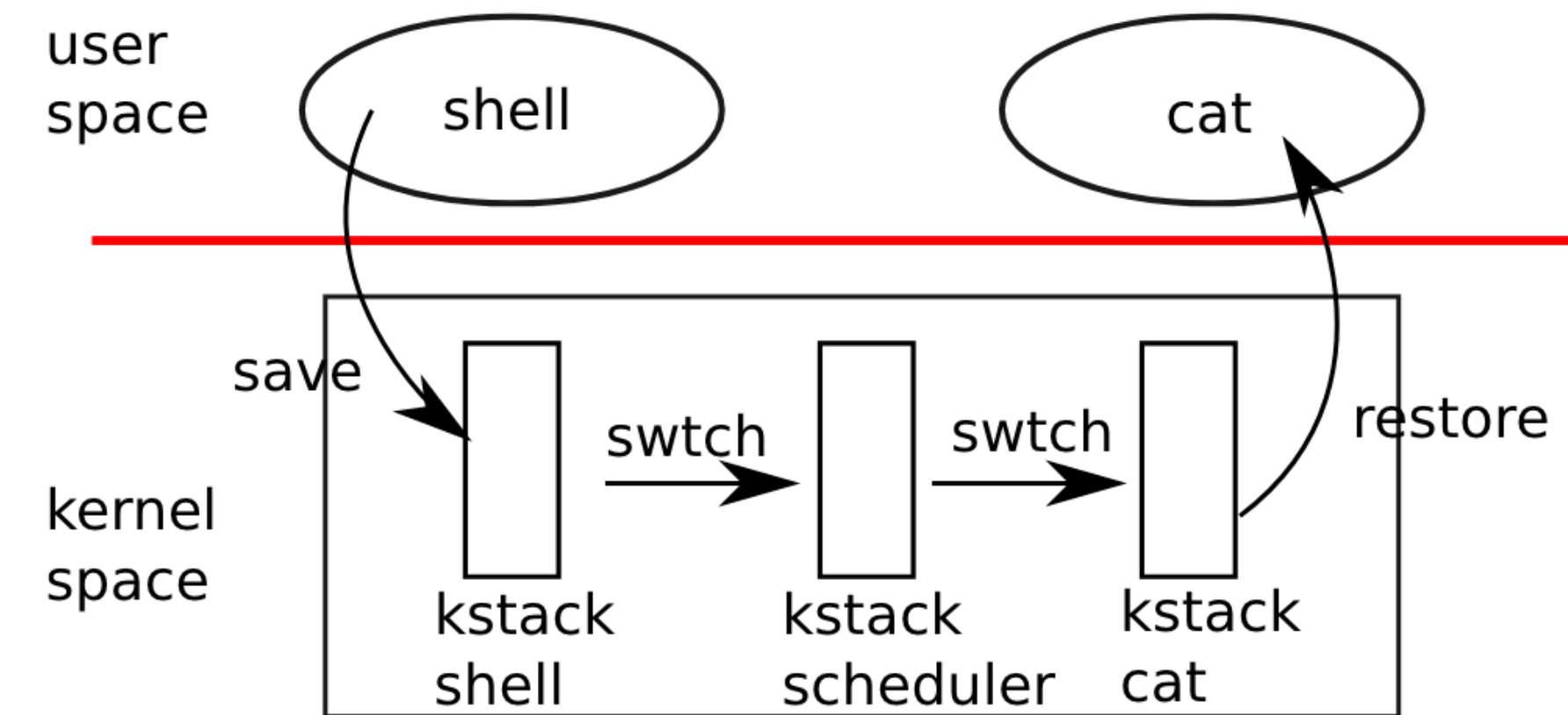
p17-free-space

- main.c
 - Calls pinit twice to start two processes and then calls scheduler
- proc.c
 - pinit calls allocproc which sets up processes like earlier with the added difference that it calls kalloc to find empty 1MB of space
- kalloc.c
 - Maintains free list in chunks of 1MB. kalloc returns the first element from free list. There is no coalescing and splitting since our OS always asks for 1MB.
- No support for context switching. Ctrl+P shows process 1 running forever



Code walkthrough: context switching

p18-sched



- main.c
 - Calls scheduler to run the first RUNNABLE process. When giving control, we *also remember “scheduler context” to come back to scheduler*. Earlier, we were only running one process and *were never coming back to the scheduler*.
- trap.c
 - Calls *yield* when timer interrupt happens
- proc.c
 - yield calls sched which switches control from the process to the scheduler
 - scheduler looks for the next RUNNABLE process and switches to it
- Ctrl+P now shows that processes are switching

Context switching in action: giving control

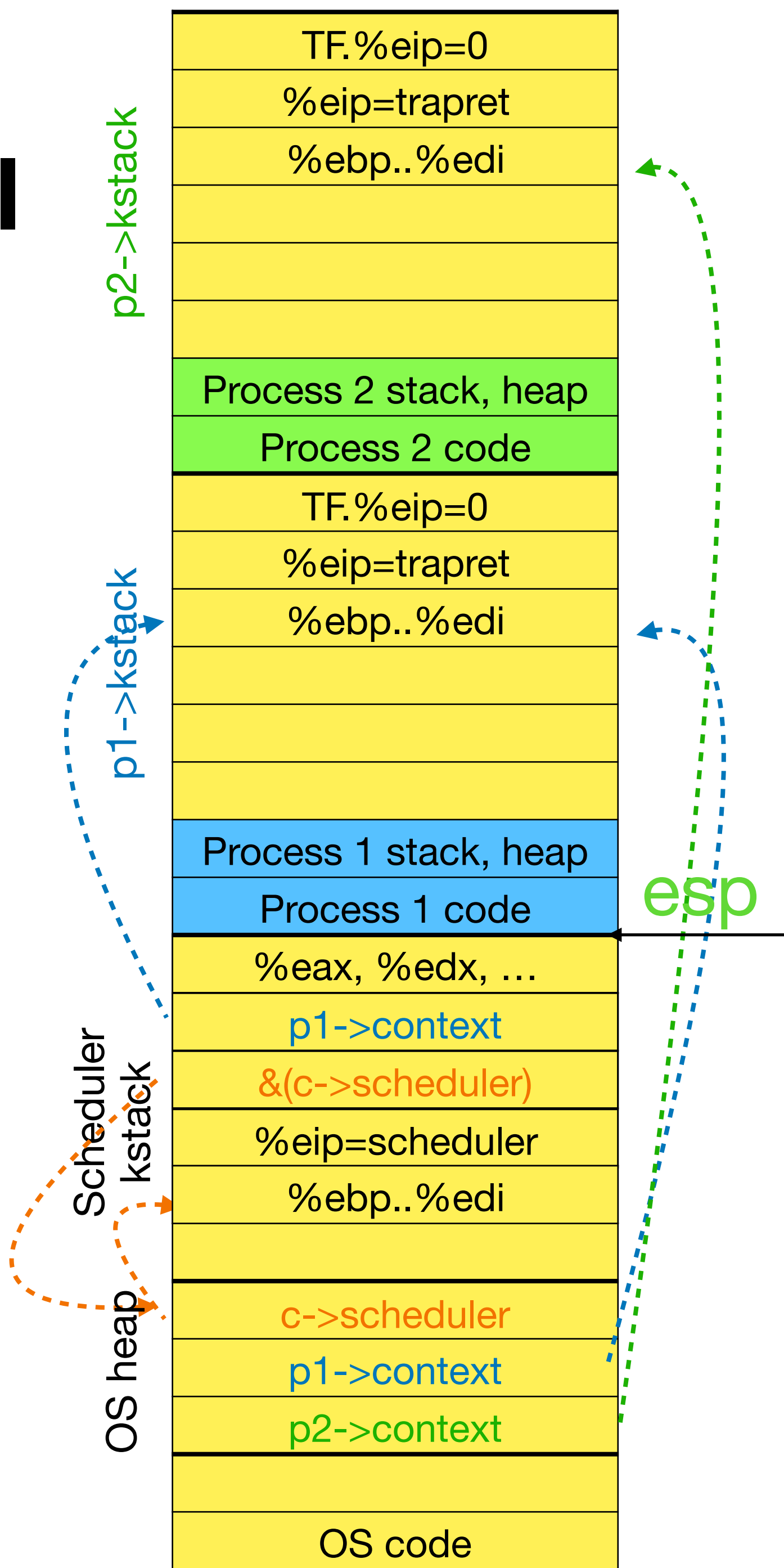
p18-sched

```
.globl swtch
swtch:
    movl 4(%esp), %eax
    movl 8(%esp), %edx
    pushl %ebp
    pushl %ebx
    pushl %esi
    pushl %edi
    movl %esp, (%eax)
    movl %edx, %esp
    popl %edi
    popl %esi
    popl %ebx
    popl %ebp
    ret

void scheduler(void) {
    struct proc *p; struct cpu *c = mycpu();
    for(;;){
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;
            p->state = RUNNING;
            switchvm(p);
            swtch(&(c->scheduler), p->context);
        }
    }
}

void yield(void) {
    struct proc *p = myproc();
    p->state = RUNNABLE;
    swtch(&p->context, c->scheduler);
}

void trap(struct trapframe *tf) {
    ..
    if(tf->trapno == T_IRQ0+IRQ_TIMER)
        yield();
}
```



Context switching in action: taking control

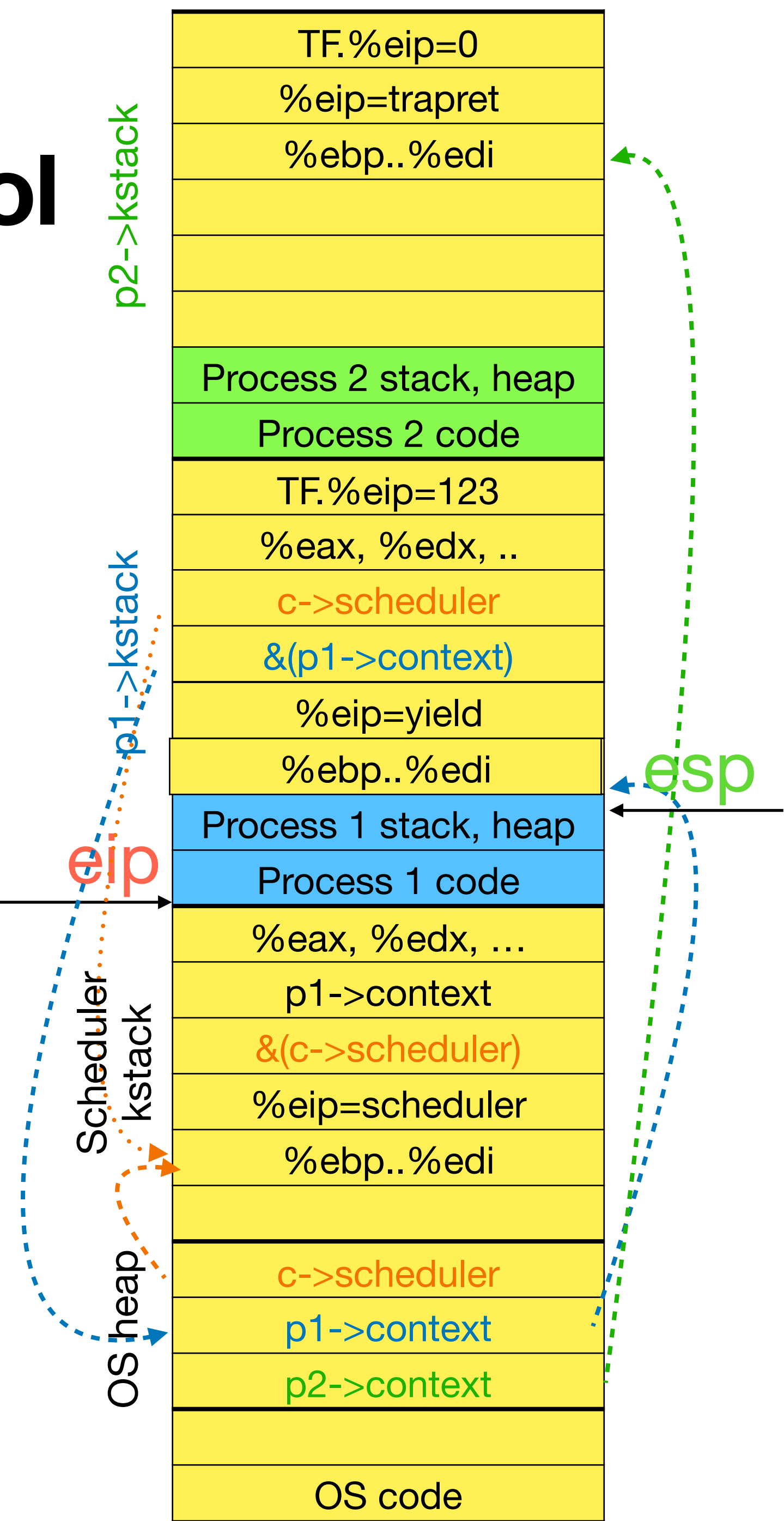
p18-sched

```
.globl swtch
swtch:
    movl 4(%esp), %eax
    movl 8(%esp), %edx
    pushl %ebp
    pushl %ebx
    pushl %esi
    pushl %edi
    movl %esp, (%eax)
    movl %edx, %esp
    popl %edi
    popl %esi
    popl %ebx
    popl %ebp
    ret

void scheduler(void) {
    struct proc *p; struct cpu *c = mycpu();
    for(;;){
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;
            p->state = RUNNING;
            switchvm(p);
            swtch(&(c->scheduler), p->context);
        }
    }
}

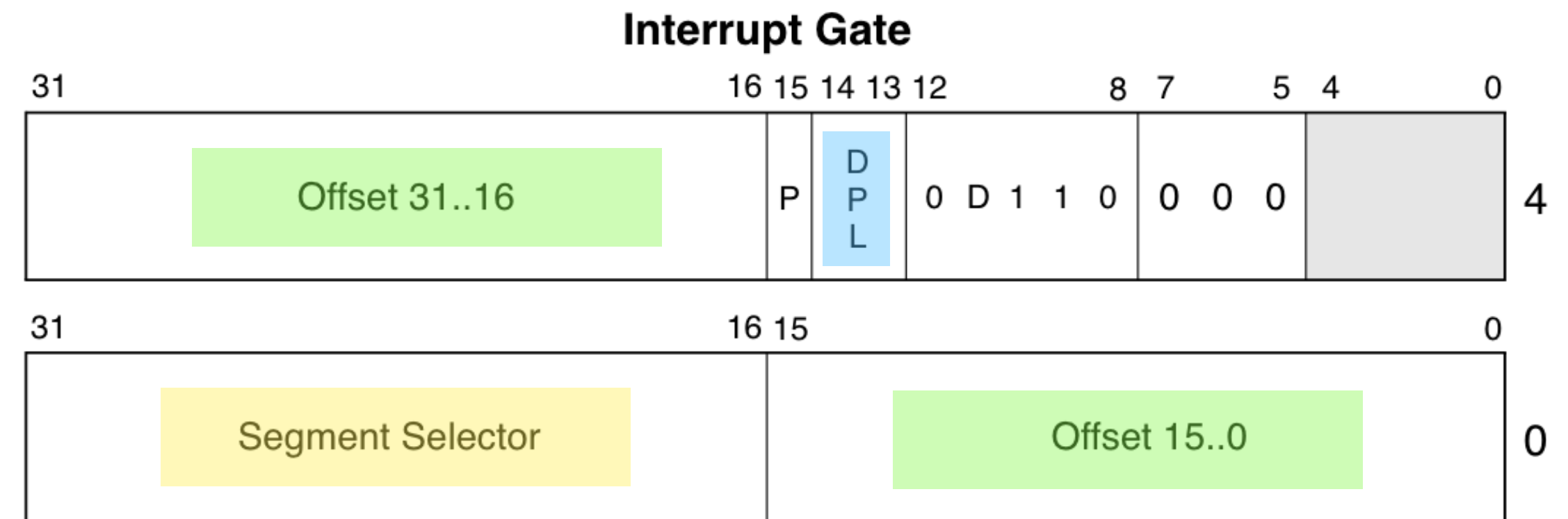
void yield(void) {
    struct proc *p = myproc();
    p->state = RUNNABLE;
    swtch(&p->context, c->scheduler);
}

void trap(struct trapframe *tf) {
    ..
    if(tf->trapno == T_IRQ0+IRQ_TIMER)
        yield();
}
```



System calls

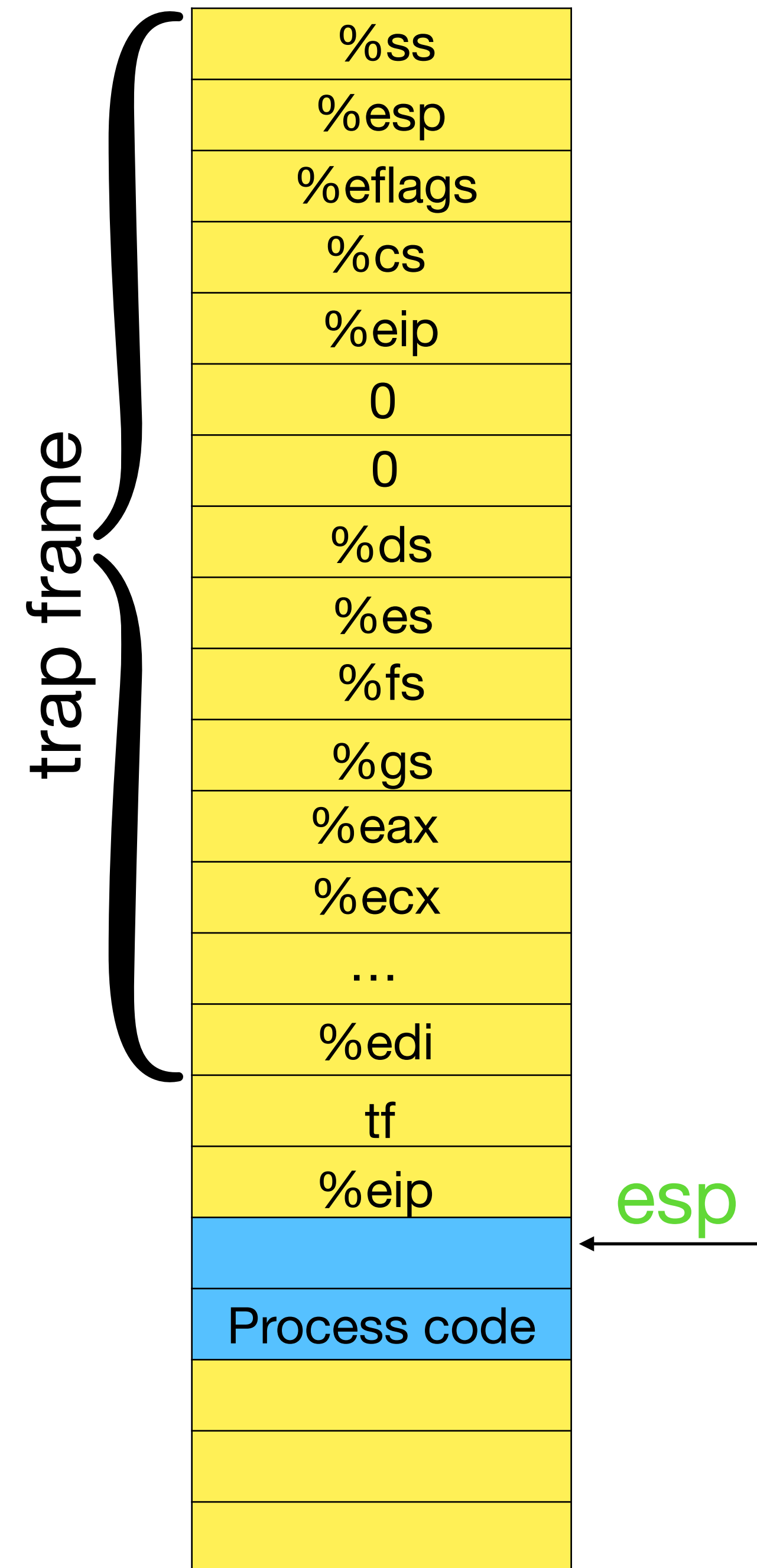
- Process wants OS to work on its behalf
 - Run INT xx instruction to trigger interrupt number xx
 - INT xx is allowed if $CPL \leq DPL$ for the xx interrupt vector



Code walkthrough

p19-syscall

- trap.c
 - tvinit sets DPL of IDT entry for T_SYSCALL to 3
 - trap calls syscall if interrupt vector is T_SYSCALL
- syscall.c
 - syscall reads eax from trap frame to find which syscall is made and calls that particular call
 - Return value of sys call is set in trap frame's eax
- initproc.S
 - Calls three system calls: open “console” file, write “hello world”, close



Syscall handling

```
void tvinit(void) {  
    int i;  
    for(i = 0; i < 256; i++)  
        SETGATE(idt[i], 0, SEG_KCODE<<3, vectors[i], 0);  
    SETGATE(idt[T_SYSCALL], 0, SEG_KCODE<<3, vectors[T_SYSCALL], DPL_USER);  
}
```

trap.c

```
void trap(struct trapframe *tf) {  
    if(tf->trapno == T_SYSCALL){  
        myproc()->tf = tf;  
        syscall();  
        return;  
    }  
    ...  
}
```

```
void syscall(void) {  
    int num;  
    struct proc *curproc = myproc();  
  
    num = curproc->tf->eax;  
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {  
        curproc->tf->eax = syscalls[num]();  
    } ...  
}
```

syscall.c

Visualising syscall handling p19-syscall

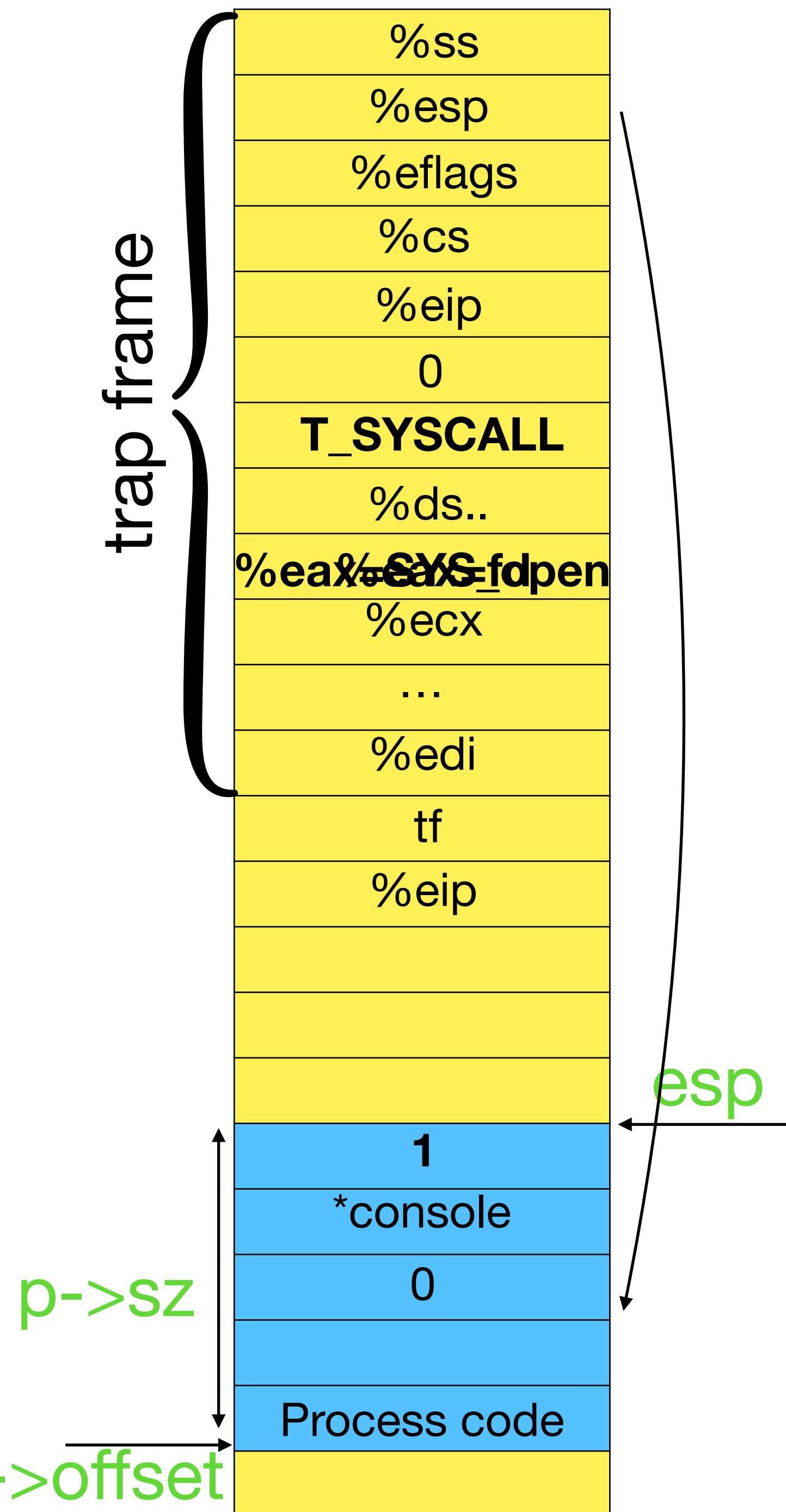
```
# fd=open("console", O_WRONLY)
  eip→ pushl $1
    pushl $console
    pushl $0
    movl $SYS_open, %eax
    int $T_SYSCALL
    # fd = %eax
```

```
int sys_open(void) {
    int fd, omode;
    if(argint(1, &omode) < 0) {
        return -1;
    }
    ..
    return fd;
}
```

```
int fetchint(uint addr, int *ip) {
    if(addr >= p->sz || addr+4 > p->sz)
        return -1;
    *ip = *(int*)(addr + p->offset);
    return 0;
}
```

```
int argint(int n, int *ip) {
    return fetchint((myproc()->tf->esp)
                    + 4 + 4*n, ip);
}

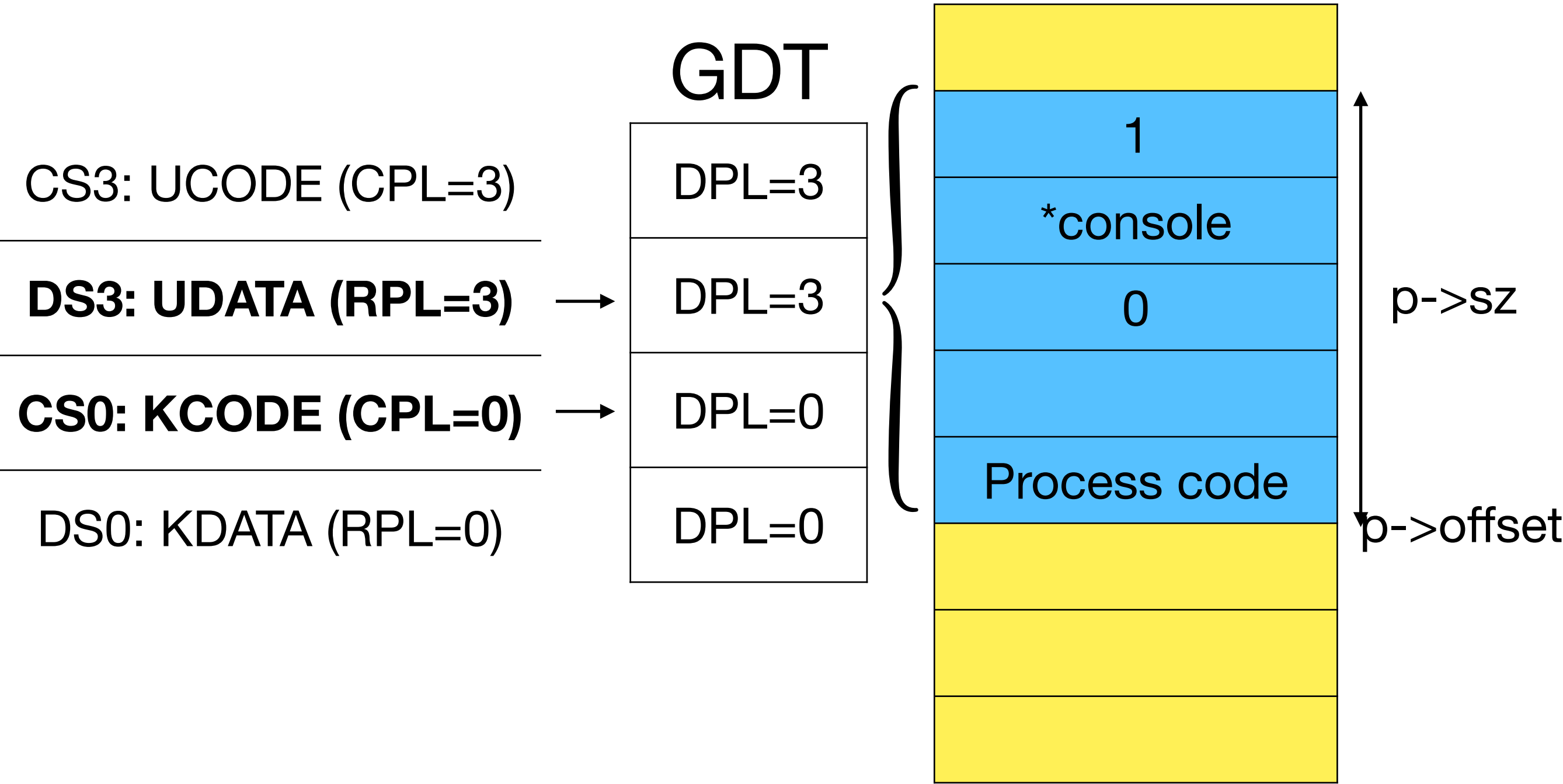
void syscall(void) {
    int num = curproc->tf->eax;
    curproc->tf->eax = syscalls[num]();
}
```



Syscall parameter checking and translation

```
int fetchint(uint addr, int *ip) {
    if(addr >= p->sz || addr+4 > p->sz)
        return -1;
    *ip = *(int*)(addr + p->offset);
}
```

- Syscall parameters must be in process' address space:
 - Check virtual address (VA) is within limit
 - Add base to VA to get physical address (PA)
- Alternative design: use process' DS to read syscall parameters
 - $CPL \leq DPL$ and $RPL \leq DPL$
- Linux has `copy\_from\_user`. Initial Linux versions had security vulnerabilities due to missing checks

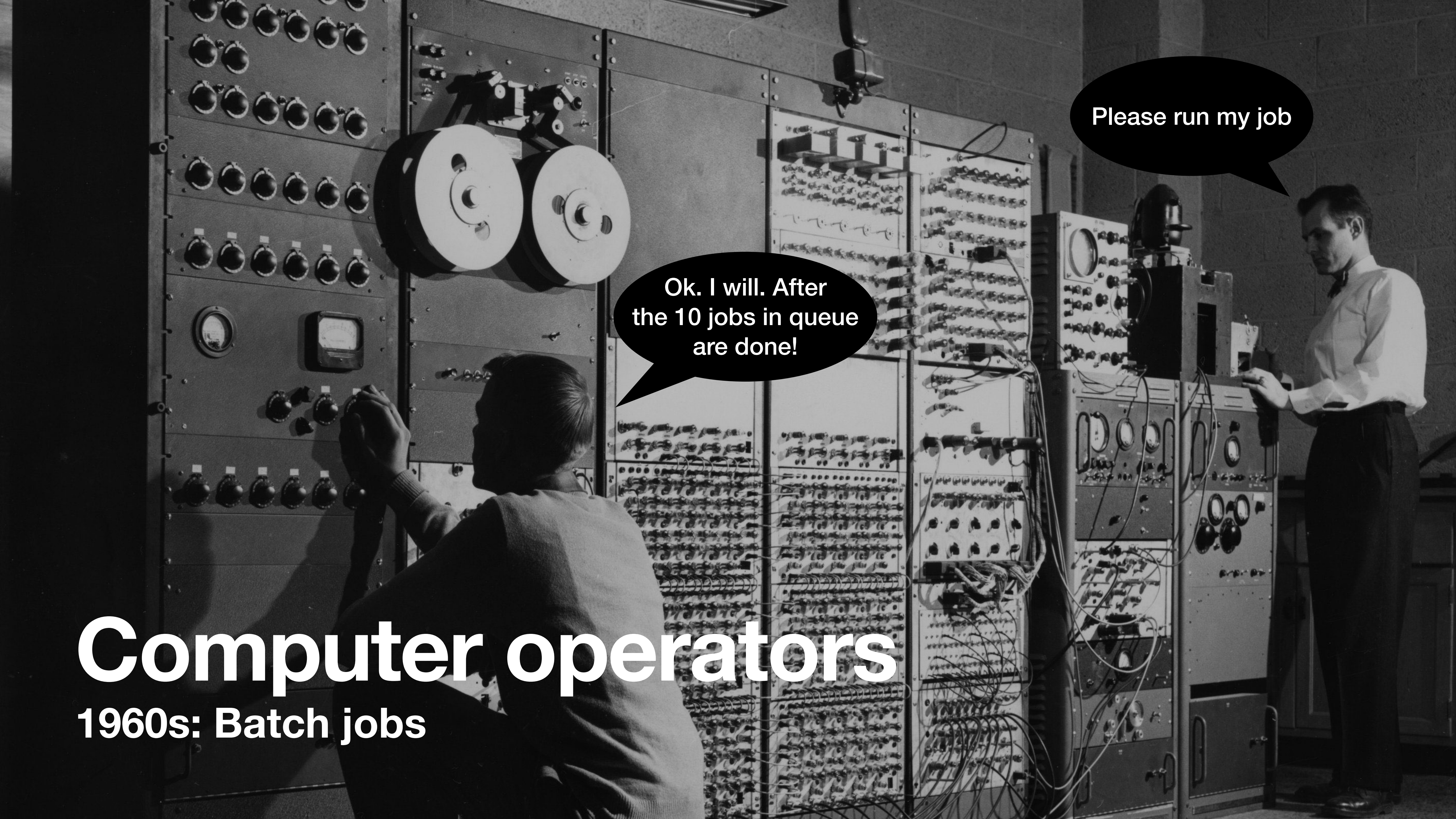


Scheduling policies

OSTEP Ch. 7-9

Scheduling problem

- A number of processes can be runnable.
 - Which process to run next?
 - How long to run each process for?
- What should scheduler try to optimise?
 - What do we know about the behaviour of each process?



Please run my job

Ok. I will. After
the 10 jobs in queue
are done!

Computer operators

1960s: Batch jobs

Batch job characteristics

- Once started, each job runs to completion
- Run-time of each job is known
- Main metric: turnaround time = (completion time - arrival time)

FIFO

- Avg turnaround time = $(10+20+30)/3 = 20$
- Avg turnaround time = $(100+110+120)/3 = 110$
- Convoy effect: fast jobs are stuck behind slow jobs

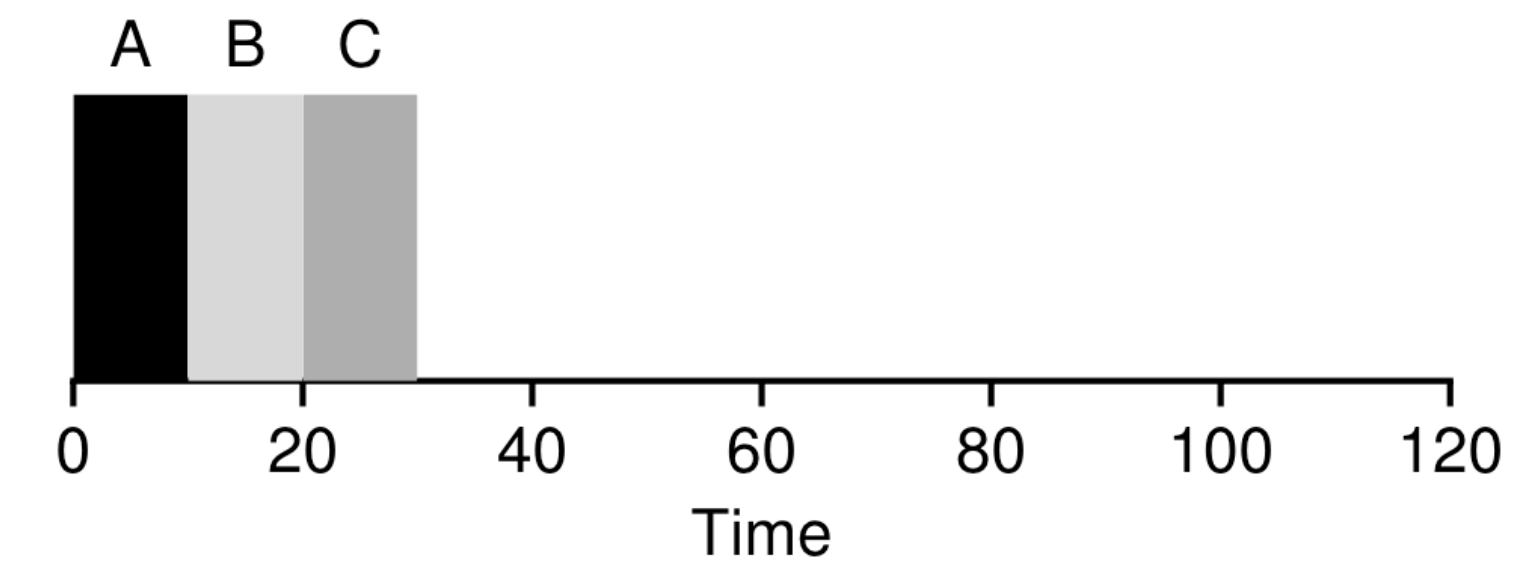
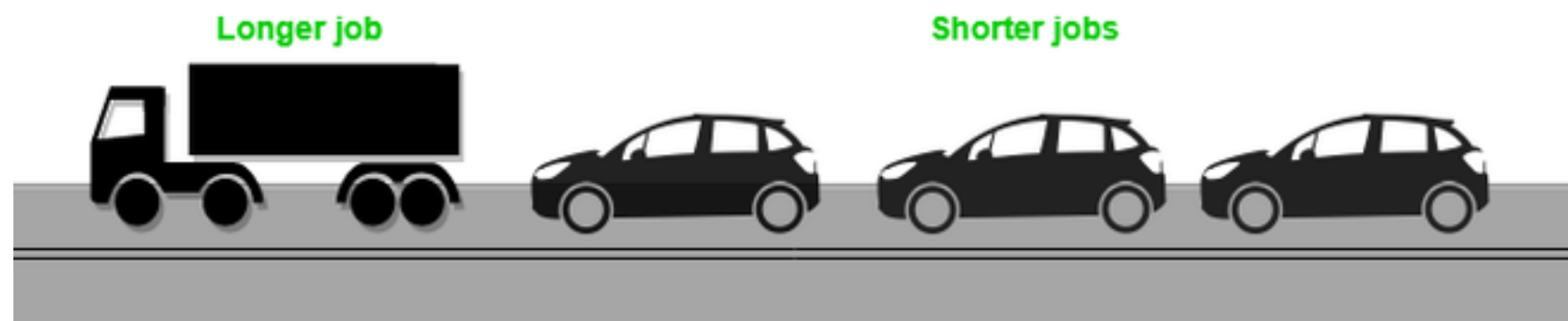


Figure 7.1: FIFO Simple Example

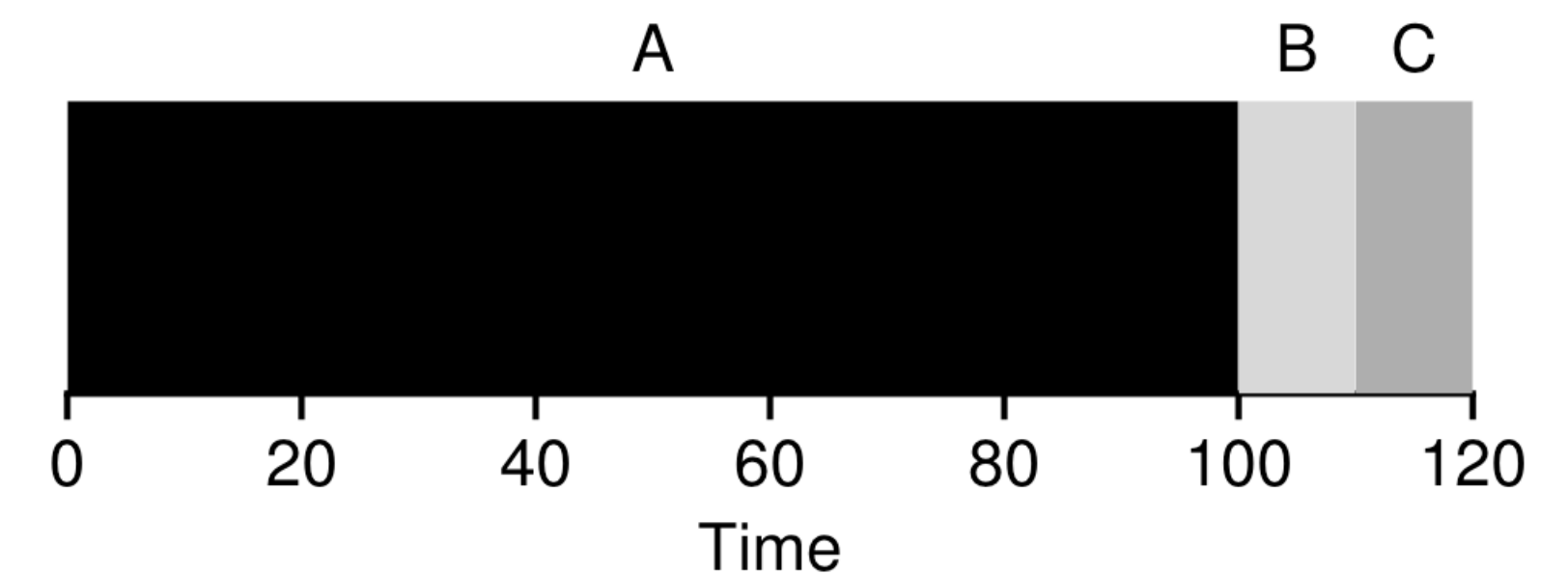


Figure 7.2: Why FIFO Is Not That Great

Shortest job first

- Avg turnaround time = $(10+20+120)/3 = 50$
- Optimal scheduling algorithm
- Jobs may not arrive at the same time.
- Avg turnaround time = $(100+100+110)/3 = 103.33$
- Again leads to convoy effect: fast jobs are stuck behind slow jobs

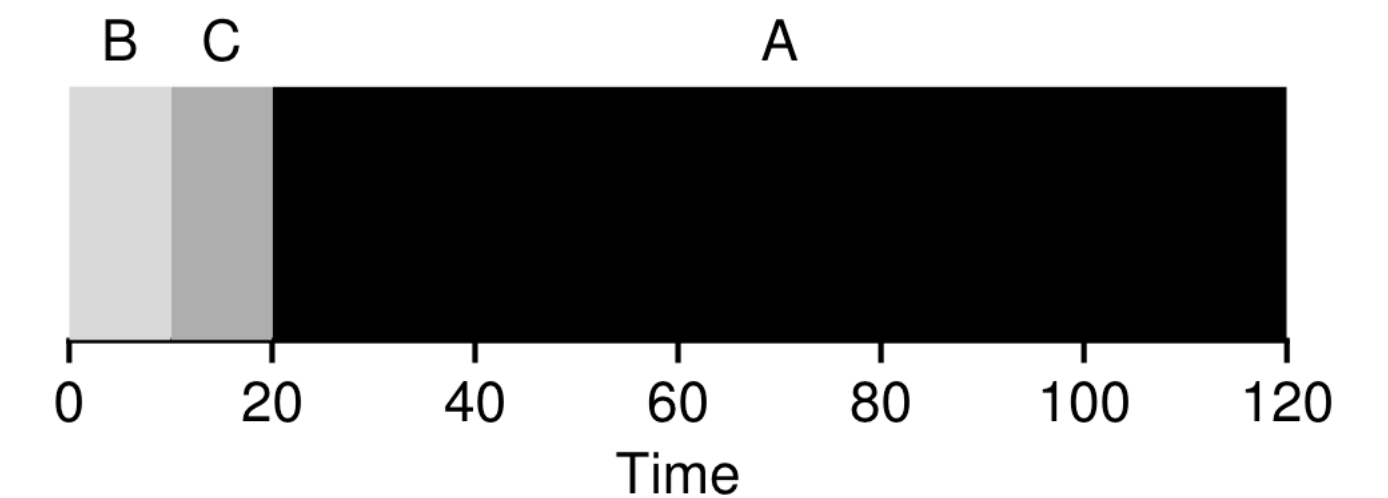
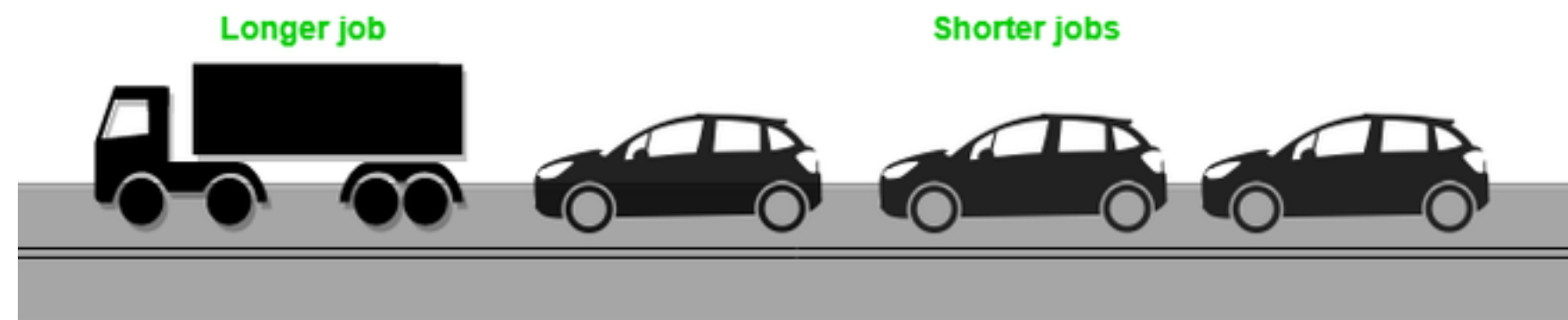


Figure 7.3: SJF Simple Example

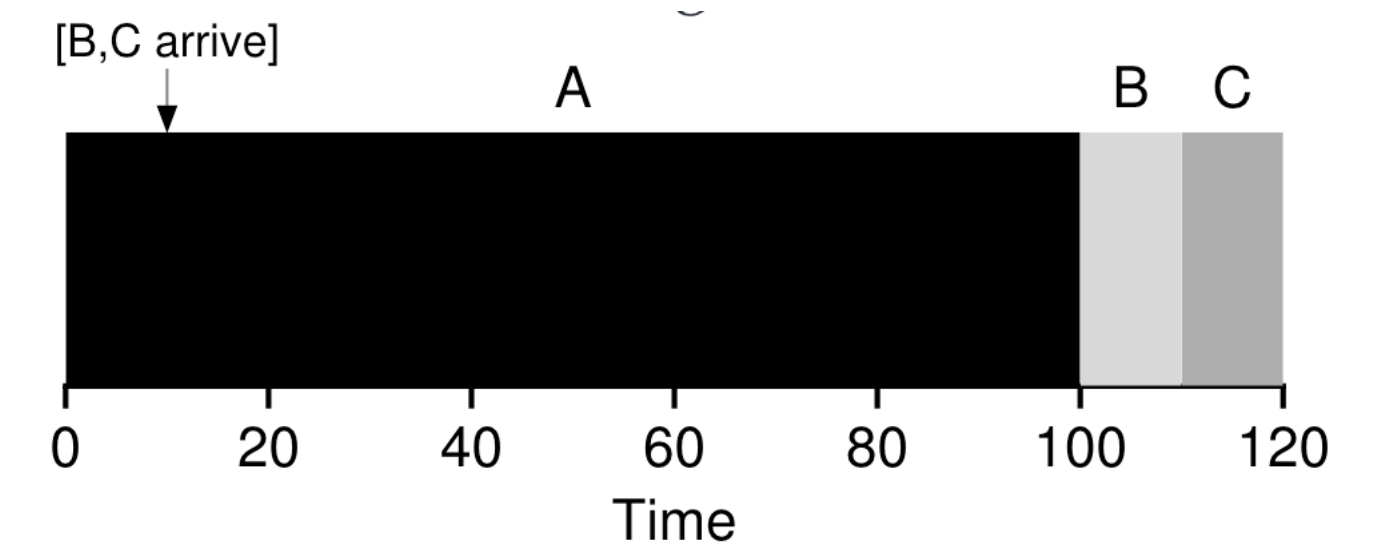


Figure 7.4: SJF With Late Arrivals From B and C

Batch job characteristics

- ~~Once started, each job runs to completion~~
- Preemptive schedulers switch to other processes before a job finishes
- Run-time of each job is known
- Main metric: turnaround time = (completion time - arrival time)

Shortest time to completion first

- Avg turnaround time = $(10+20+120)/3 = 50$
- Optimal scheduling algorithm

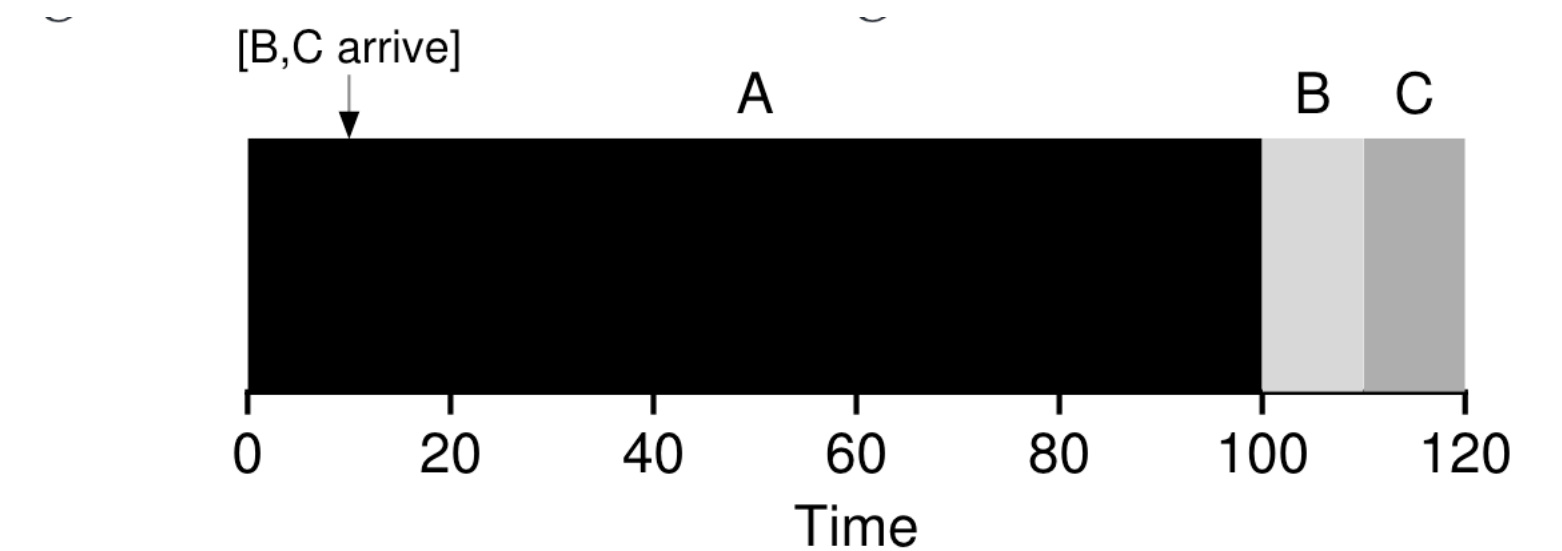


Figure 7.4: SJF With Late Arrivals From B and C

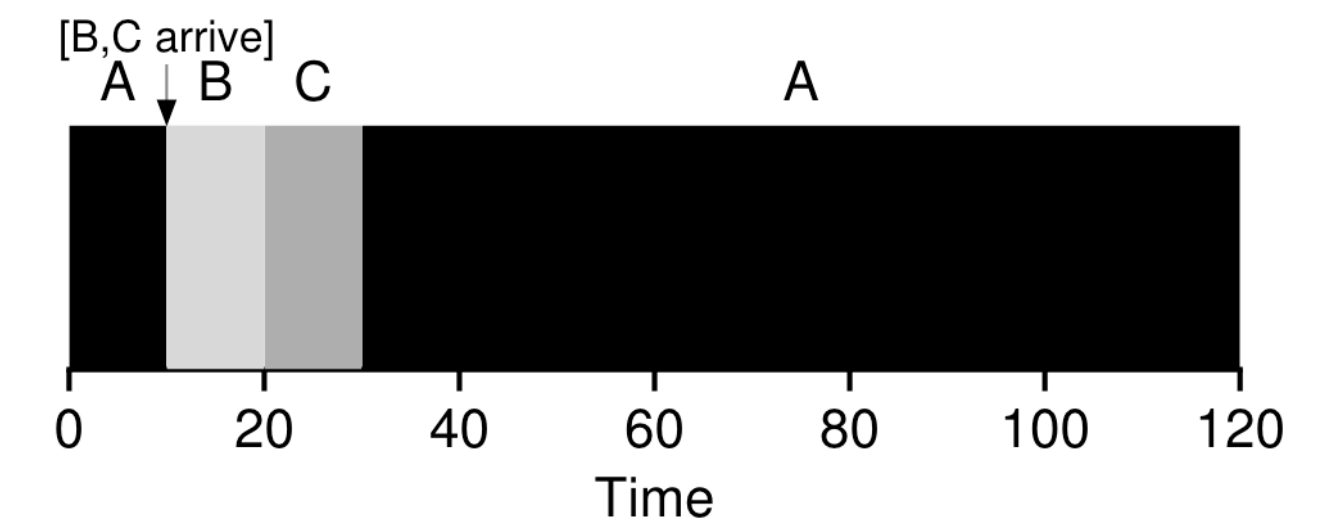
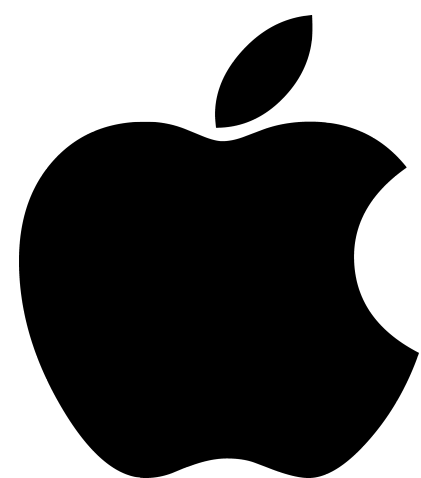
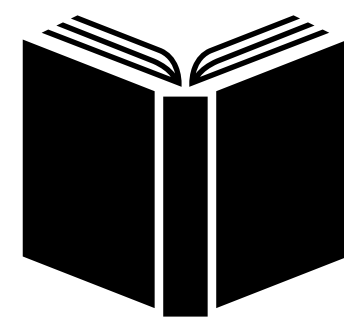
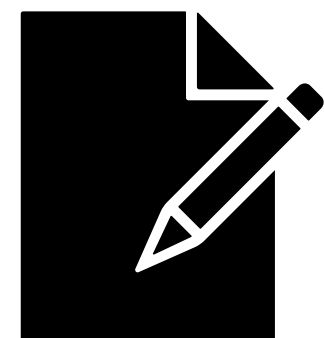
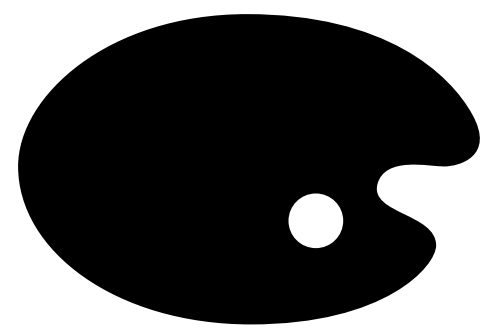


Figure 7.5: STCF Simple Example

Personal computers

1980s: Interactive jobs!



UNIX®



Interactive job characteristics

- Preemptive schedulers switch to other processes before a job finishes
- ~~Run-time of each job is known~~
- ~~Main metric: turnaround time = (completion time - arrival time)~~
- Main metric: response time = (first run - arrival time)
- Avg response time in SJF = $(0+5+10)/3 = 5$

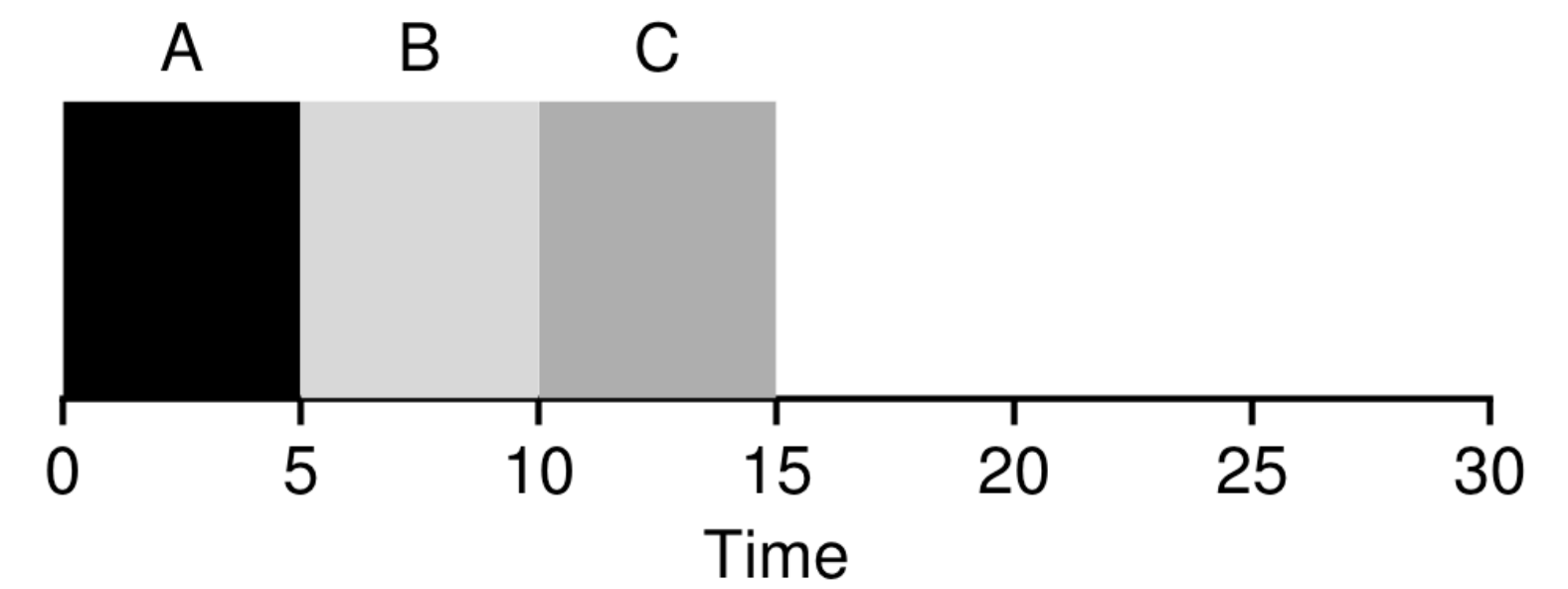


Figure 7.6: SJF Again (Bad for Response Time)

Round robin

- Run for a time slice T (scheduling quantum)
- Avg response time = $(0+1+2)/3 = 1$
- How long should the time slice T be?
 - $T \gg$ context switch time (~ 1 us)
 - $T \sim$ human response time (few ms)
 - $T \sim$ Used to be ~ 10 ms in 1970s. Still ~ 10 ms. CPU speeds: 60MHz $\rightarrow$ 3GHz.
- Worst policy for turnaround time: elongates completion times of all jobs
- Gives “fair share” to all jobs within small amount of time

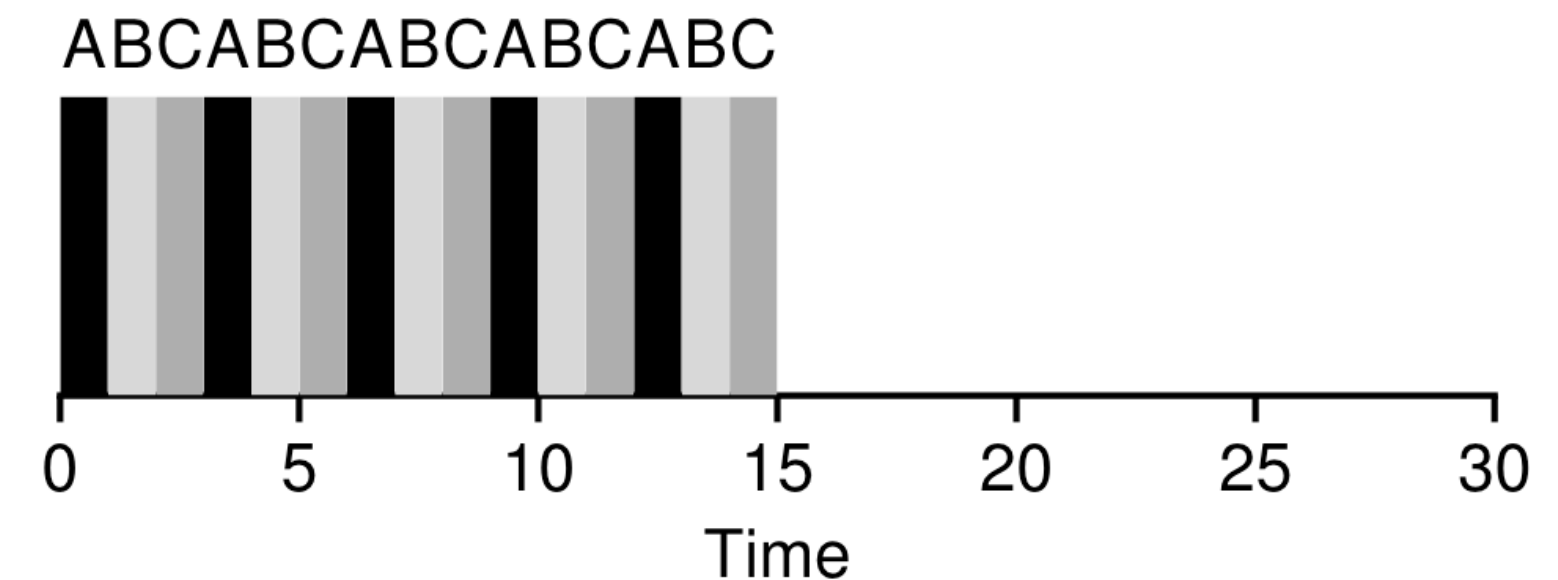


Figure 7.7: Round Robin (Good For Response Time)

Proportional share

- All jobs are not equally important
 - Dropbox syncing should get lesser CPU time than music playback and gameplay
 - Give “proportional share” to jobs based on their importance

Stride scheduling

```
current = remove_min(queue); // pick client with minimum pass
schedule(current);           // use resource for quantum
current->pass += current->stride; // compute next pass using stride
insert(queue, current);      // put back into the queue
```

- Higher priority processes have low strides
- Example: A:B:C = 2:1:5
- Deterministic scheduler

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

Figure 9.3: Stride Scheduling: A Trace

Lottery scheduling

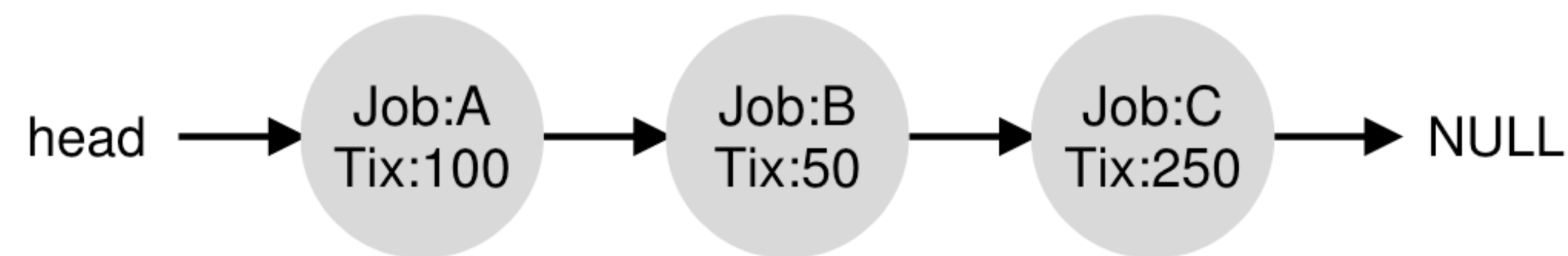
- Probabilistic scheduler
 - A:B ~ 3:1
 - Generate a random number 0-99 in each time quantum
 - 0-74: run A, 75-99: run B

63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 49

A A A A A A A A A A A A A A A A
 B B B B

Lottery scheduling implementation

- Each process holds its own number of tickets
- Generate a random number (e.g. 300) between 0 and total number of tickets (e.g. 400) in each time quantum
- Traverse list while reducing by tickets to find the process



300	200	150	C
125	25		B

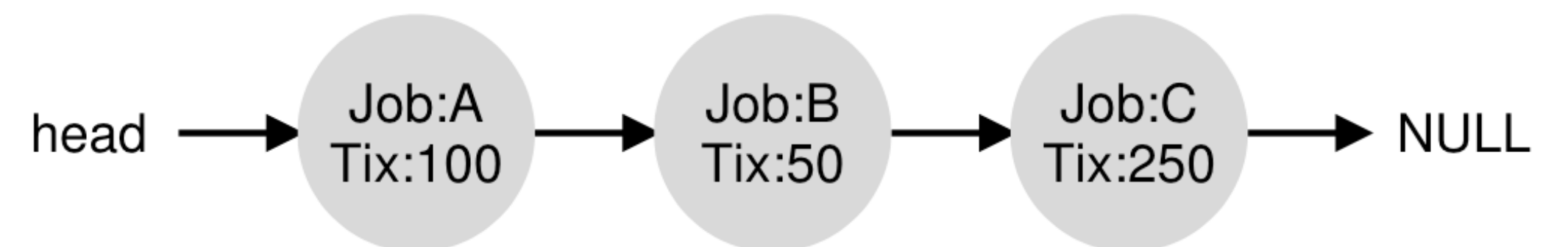
Setting priority

- When processes are trusted, they can raise or lower their priority
 - Example: music player lowers its priority when user pauses the song
 - e.g, B changes its tickets to 250

- Otherwise, users can manually lower priority:

```
time taskset -c 1 ./nicedemo
```

```
time nice -n 19 taskset -c 1 ./nicedemo
```



Process states

- Processes can be blocked upon doing IO

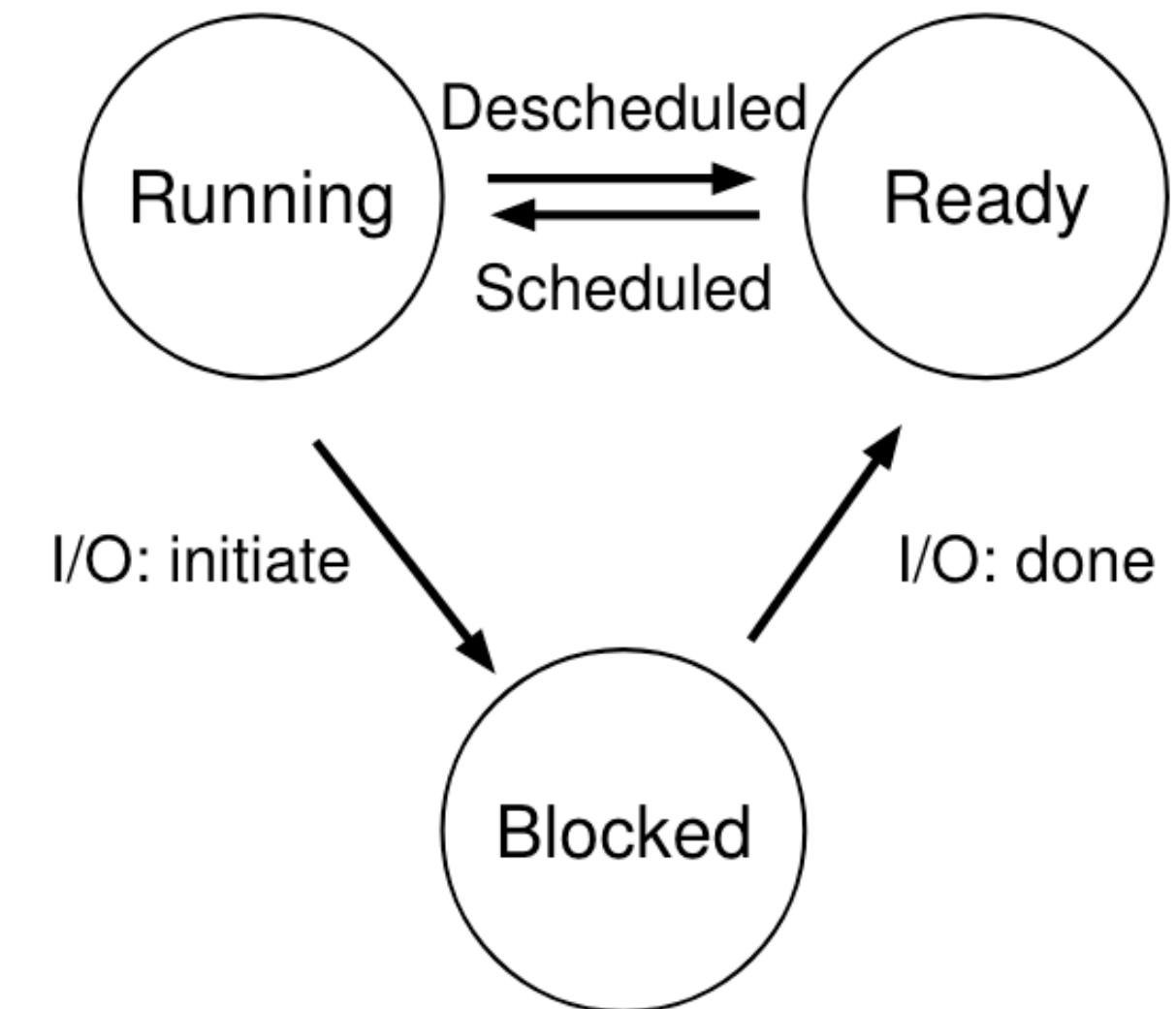


Figure 4.2: Process: State Transitions

Sleep in xv6 disk driver

```
void sleep(void *chan) {                                proc.c
    struct proc *p = myproc();
    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    // Tidy up.
    p->chan = 0;
}

void wakeup(void *chan) {
    struct proc *p;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == SLEEPING && p->chan == chan)
            p->state = RUNNABLE;
}
```

```
// Per-process state                                proc.h
struct proc {
    ...
    void *chan;      // If non-zero, sleeping on chan
}
```

```
void ideintr(void) {                                    ide.c
    struct buf *b = idequeue;
    if(!(b->flags & B_DIRTY) && idewait(1) >= 0)
        insl(0x1f0, b->data, BSIZE/4);
    b->flags |= B_VALID;
    b->flags &= ~B_DIRTY;
    wakeup(b);
}

void iderw(struct buf *b){
    if(idequeue == b)
        idestart(b);
    while((b->flags & (B_VALID|B_DIRTY)) != B_VALID){
        noop();
        sleep(b);
    }
}
```

IO aware scheduling

- Mixture of CPU intensive and IO bound processes
 - Turnaround time is important for CPU intensive processes, e.g., training an ML model
 - Response time is important for interactive IO bound processes, e.g., editor
- Challenge: OS does not know which process is CPU intensive / IO bound
- Round robin (fair share) and proportional share algorithms assume all processes are CPU intensive
- Example: Round robin scheduler (xv6) gives poor response time to interactive processes

CPU	1	1	1	2	2	2	3	3	3	1	1	1	2	2	2	3	3	3	1
Disk																			

CPU	1	1	2	3	3	3	2	2	2	3	3	3	1	1	2	3	3	3	2
Disk			1	1	1	1	1								1	1	1	1	1

Multi-level feedback queue (MLFQ)

- Strict priority using multiple queues
 - $(A > C) \Rightarrow A$ runs
 - $(A = B) \Rightarrow$ Round robin
- Goal: IO bound (interactive) processes shall take higher priority for CPU over CPU-intensive processes
- Challenge: OS does not know nature of the process

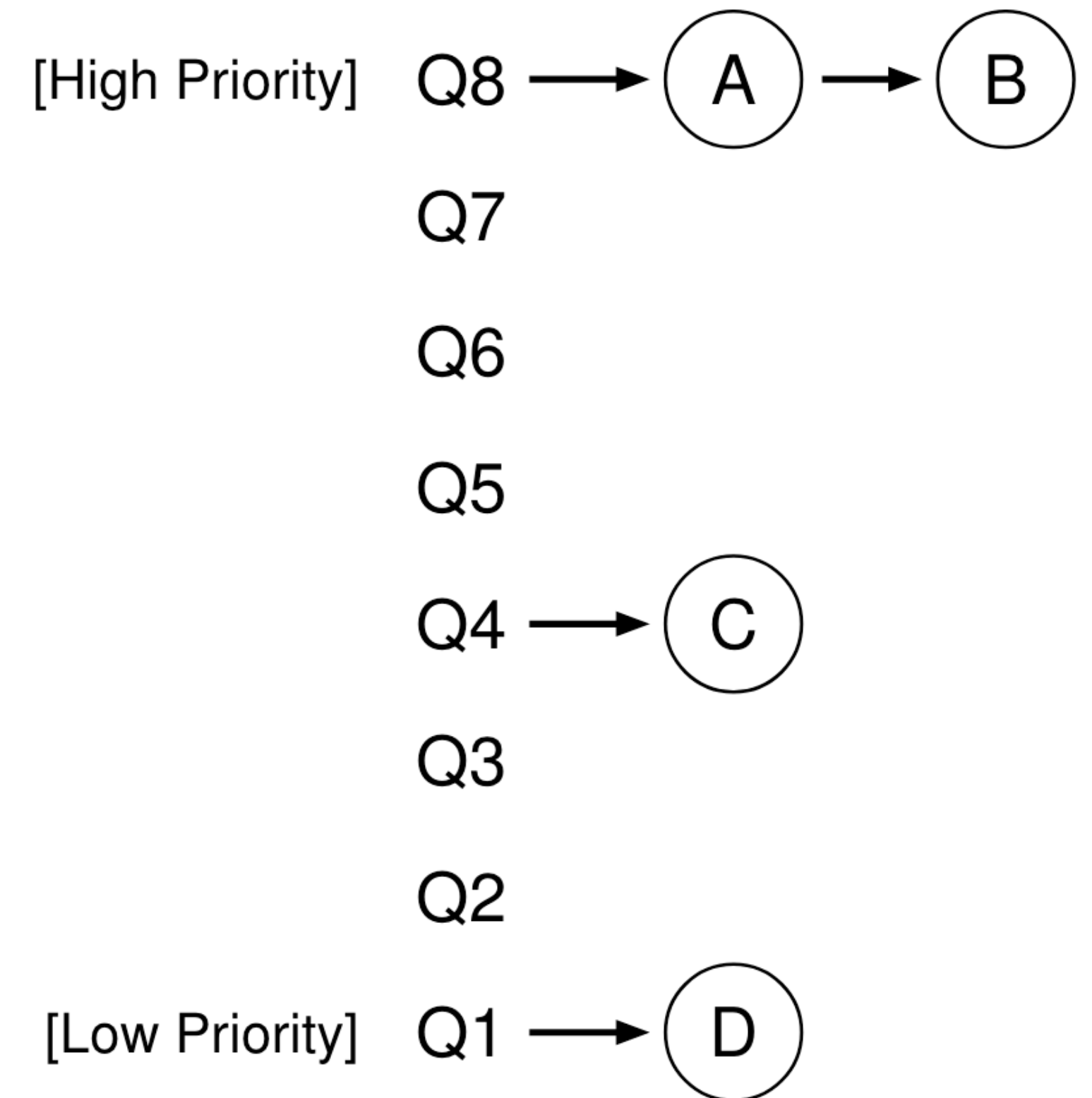


Figure 8.1: MLFQ Example

MLFQ level management

- Observation: A CPU intensive process completely uses up its time slice.
e.g, process 2

CPU	1	1	2	3	3	3	2	2	2	3	3	3	1	1	2	3	3	3	2
Disk			1	1	1	1	1								1	1	1	1	1

- Goal: IO bound (interactive) processes shall take higher priority for CPU over CPU-intensive processes
 - If a process completely uses up its time slice, lower its priority

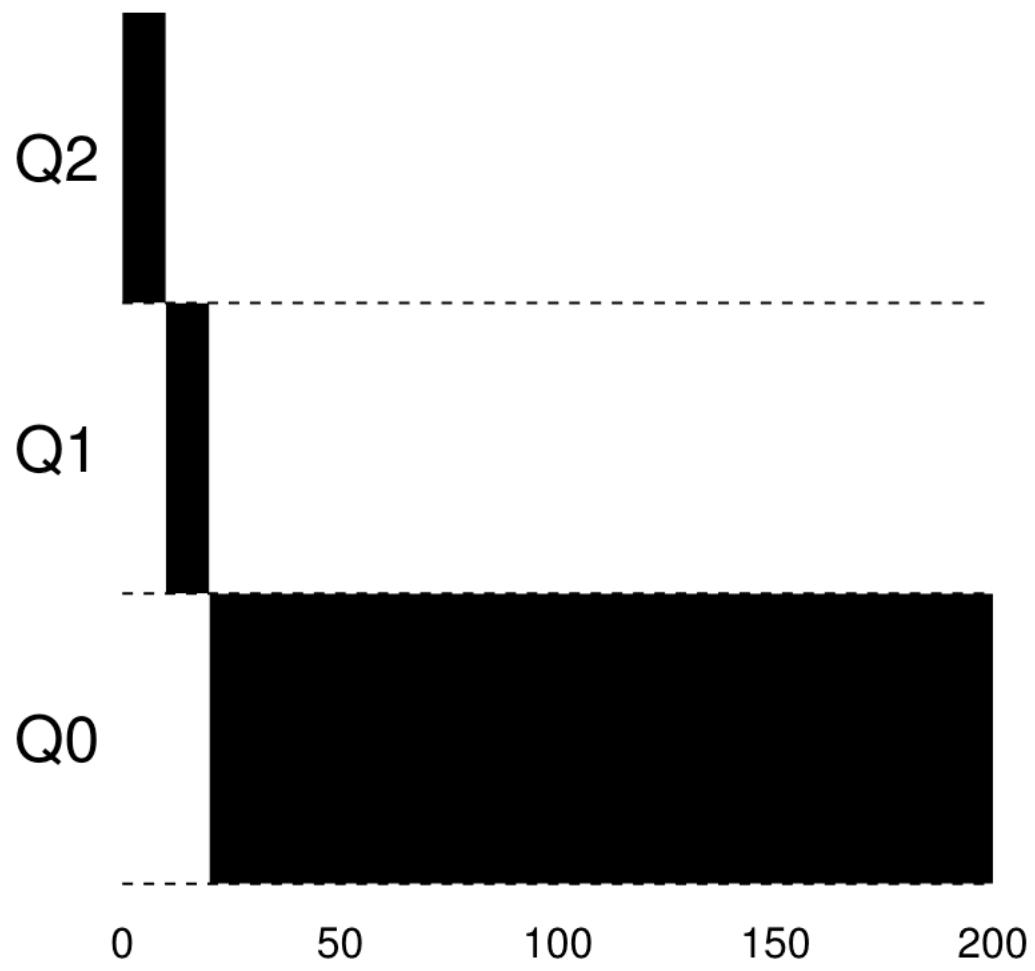


Figure 8.2: Long-running Job Over Time

MLFQ approximates STCF

- Short jobs get priority over long-running jobs
- IO intensive jobs are prioritised over CPU intensive processes => better response time for interactive jobs

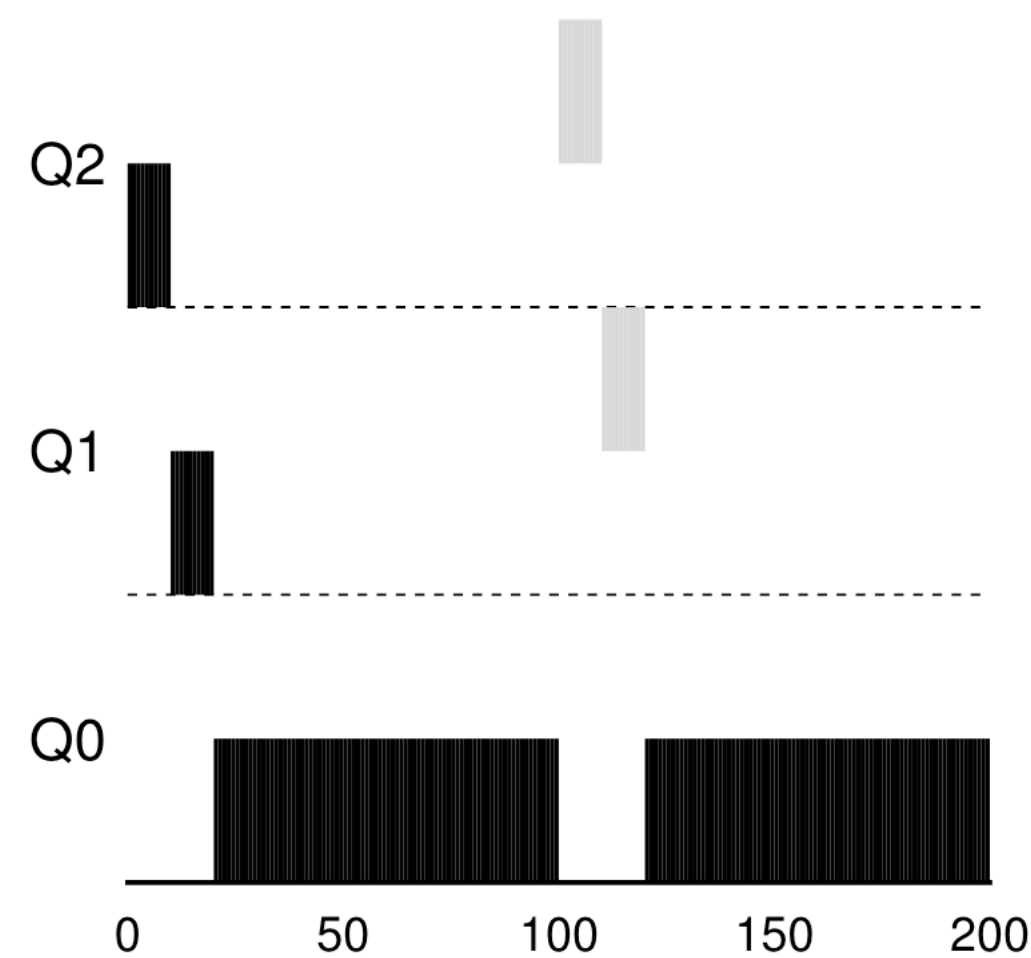


Figure 8.3: Along Came An Interactive Job

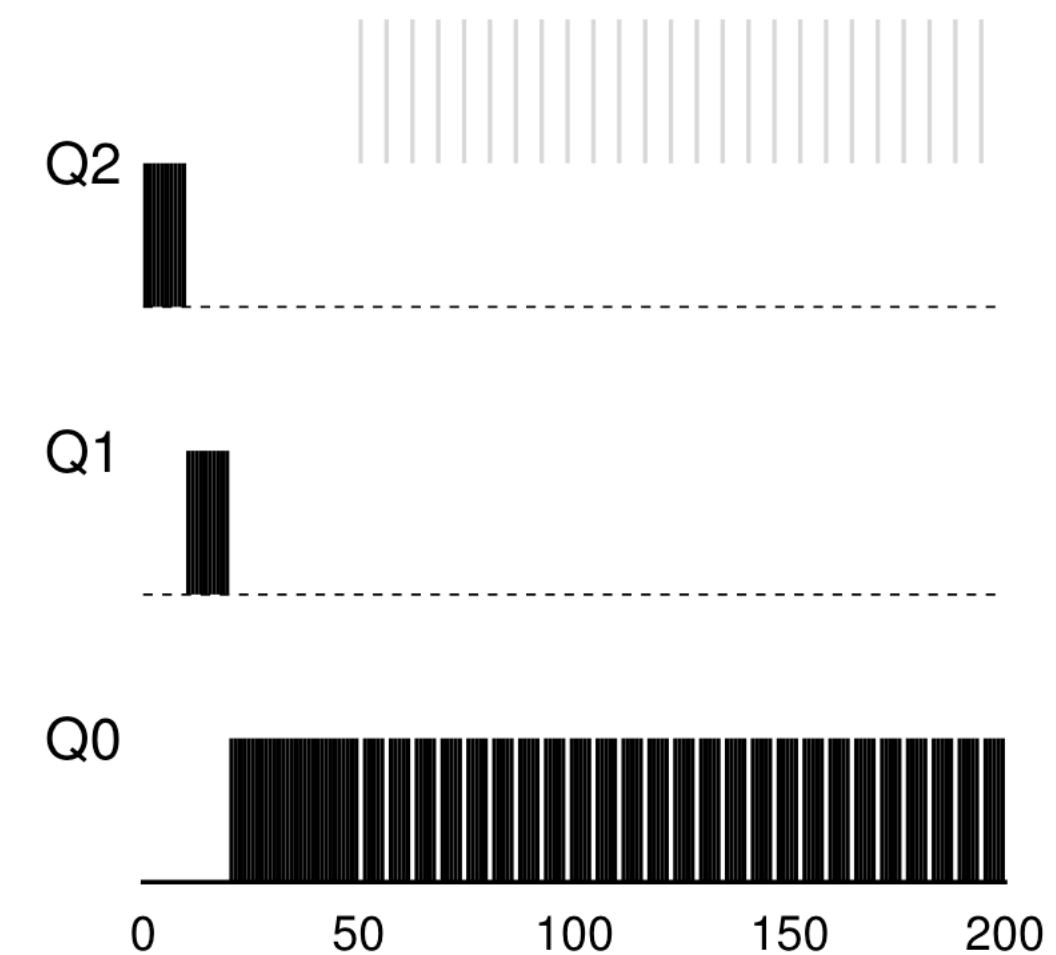
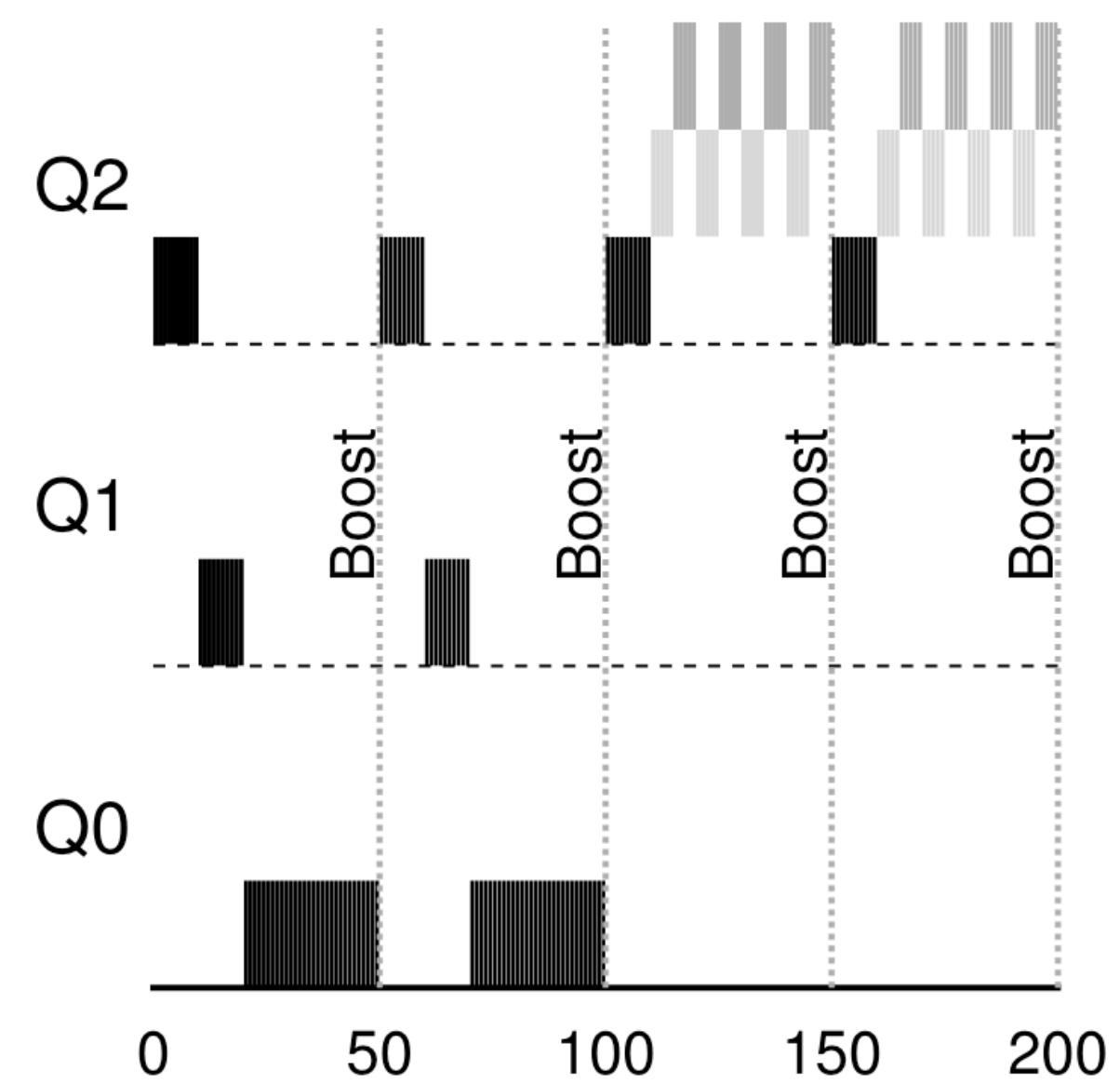
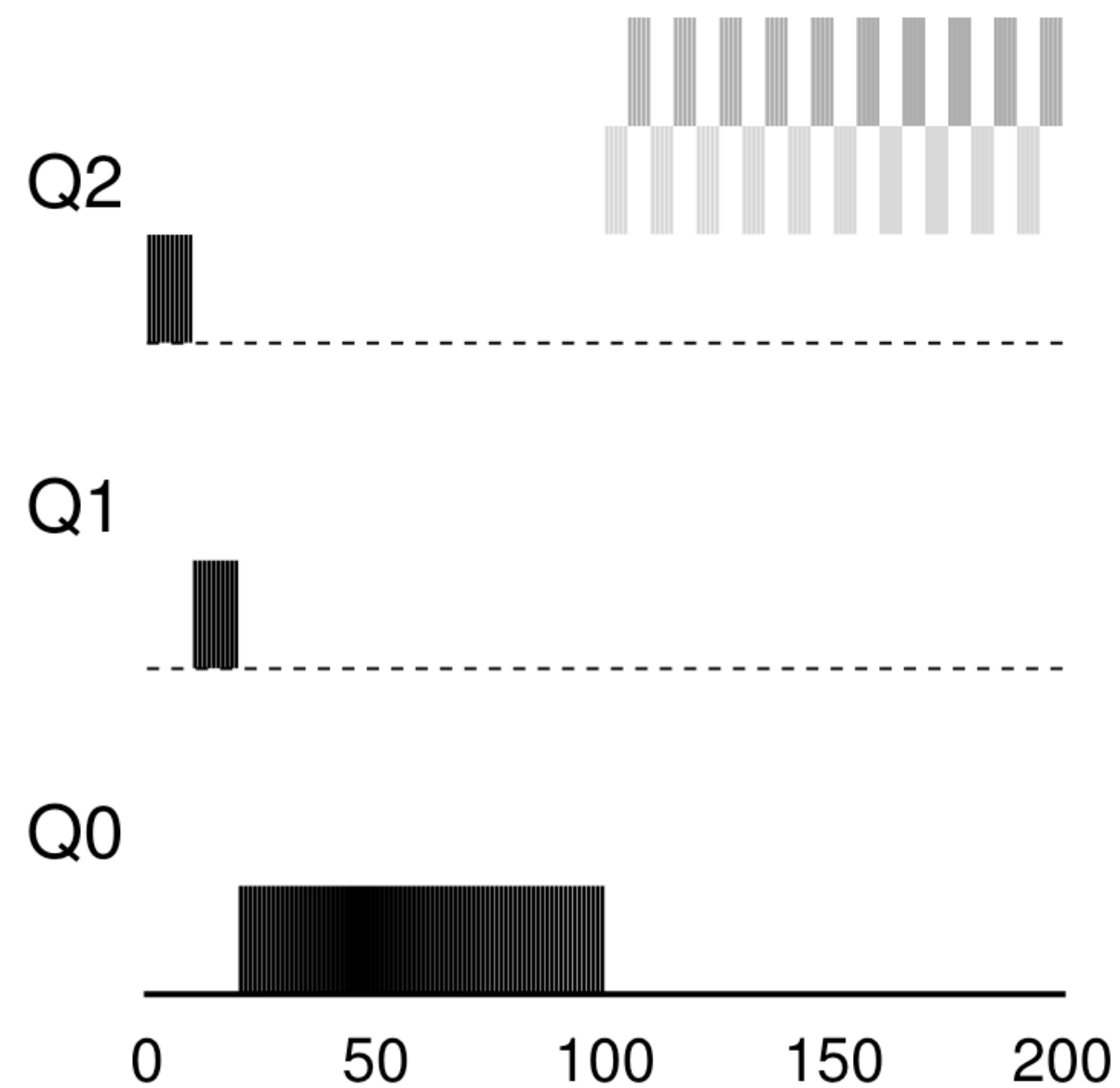


Figure 8.4: A Mixed I/O-intensive and CPU-intensive Workload

Starvation freedom

- Many IO bound jobs can starve CPU intensive jobs
- Boost all jobs to highest priority queue after some time quantum
- Also helps if CPU intensive job has become IO intensive job



Reduce scheduling overhead for CPU intensive jobs

- If there are no IO bound jobs, CPU intensive jobs run without preemption for a longer time slice

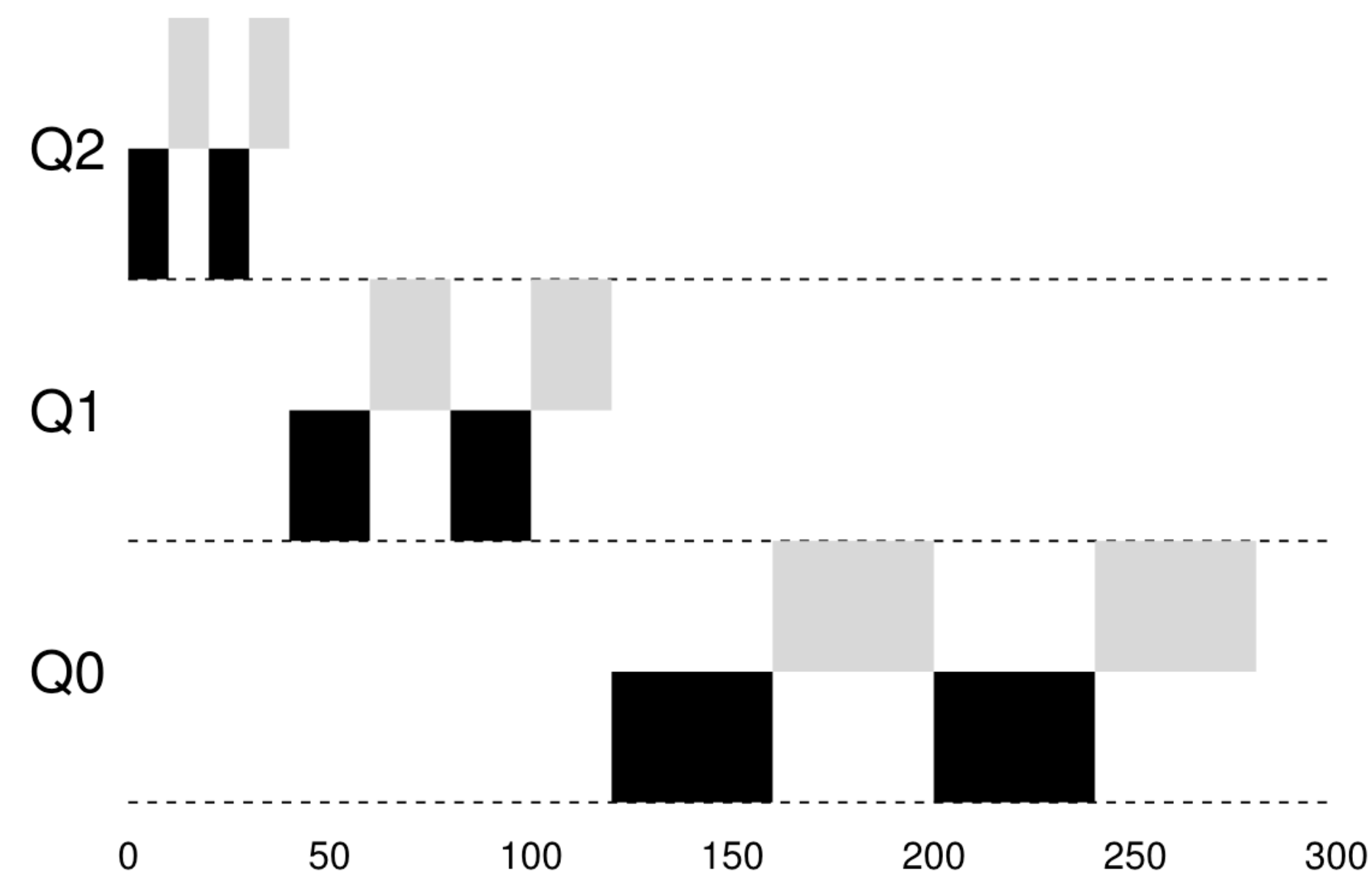
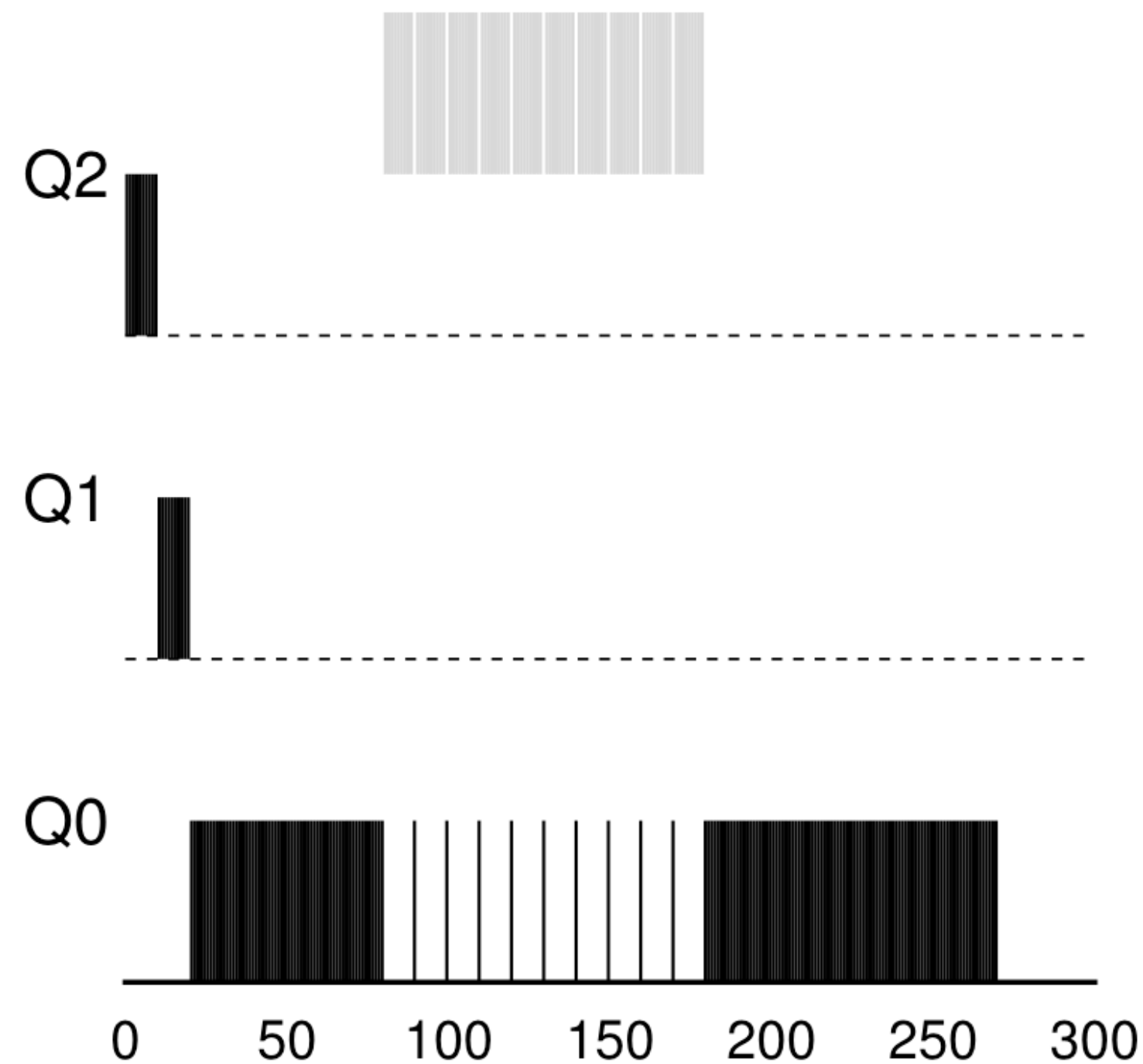


Figure 8.7: Lower Priority, Longer Quanta

Scheduler attacks

- Example: yield CPU right before time quanta expires to stay in highest priority
- In Linux 2.4 ~2003. Freezes up desktops!



Linux Completely Fair Scheduler (CFS)

- Priority scheduler:
 - High priority processes have low nice values. They get longer time slices
 - $\text{weight}[\text{nice}] \approx 1024 * 0.8^{\text{nice}}$

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{n=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency}$$

```
static const int prio_to_weight[40] = {  
    /* -20 */      88761,      71755,      56483,      46273,      36291,  
    /* -15 */      29154,      23254,      18705,      14949,      11916,  
    /* -10 */       9548,       7620,       6100,       4904,       3906,  
    /*  -5 */      3121,      2501,      1991,      1586,      1277,  
    /*   0 */     1024,       820,       655,       526,       423,  
    /*   5 */      335,       272,       215,       172,       137,  
    /*  10 */     110,        87,        70,        56,        45,  
    /*  15 */      36,        29,        23,        18,        15,  
};
```


Completely Fair Scheduler (CFS)

- Each process has vruntime. Update vruntime after each time process is run.
- Maintain runnable processes in a red-black tree (sorted by vruntime). Run the process with lowest vruntime.
- When a new process is run:
 - Assign it lowest vruntime (instead of assigning it zero). Otherwise the new process monopolises the CPU

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

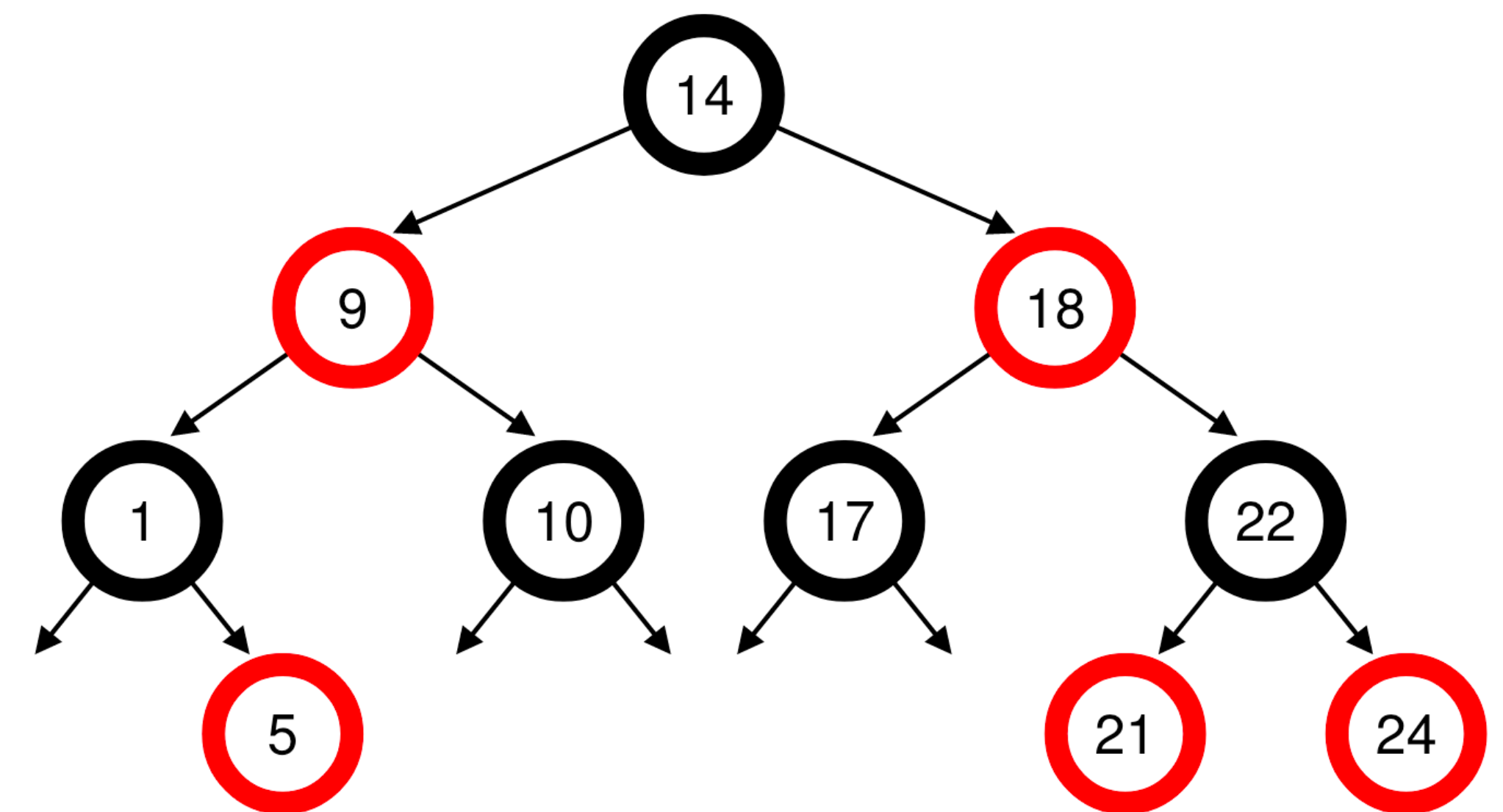
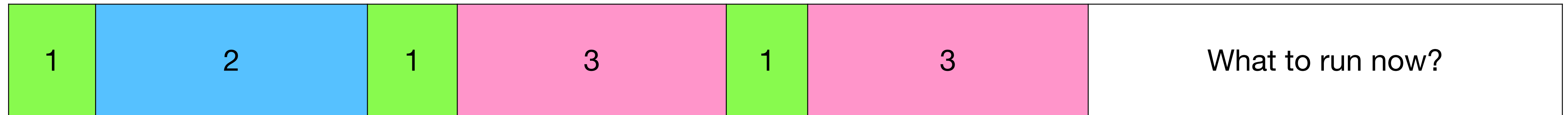


Figure 9.5: CFS Red-Black Tree

CFS: fair but not responsive

Eligible Earliest Virtual Deadline First (EEVDF) from Linux 6.6 (2024)



- $\text{vruntime}(1) = \text{vruntime}(2)$
- CFS: schedule either. Latency-sensitive process 1 misses deadlines!
- EEVDF: since latency-sensitive process 1 is asking for “*smaller time slices*”, it has “*earlier virtual deadline*”. Run 1. Improves responsiveness.

1. Latency-sensitive process
2. Batch process
3. Batch process

Scheduler attacks

Before Linux 2.6 (2003)

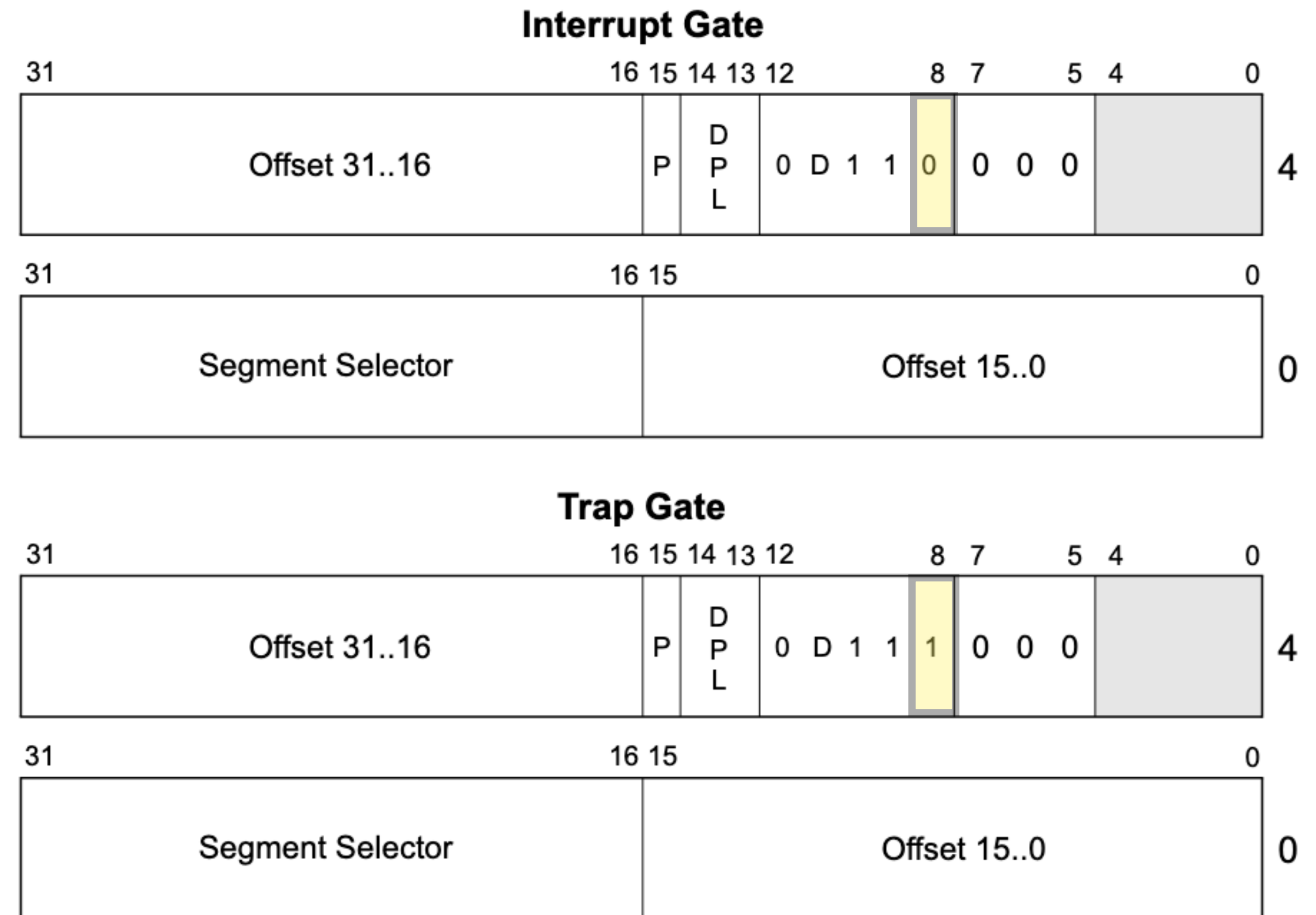
- Kernel code was *non-preemptible*.
- Syscalls also disable interrupts like hardware interrupts
- If process makes large system calls (e.g., large writes), it could exceed its time slice
- From Linux 2.6: kernel code is preemptible, e.g., keep interrupts enabled during syscalls

Keeping interrupts enabled during syscalls

- Set “trap gates” in Interrupt Descriptor Table
- Hardware keeps interrupts enabled during traps

```
void tvinit(void) {  
    int i;  
    for(i = 0; i < 256; i++)  
        SETGATE(idt[i], 0, SEG_KCODE<<3, vectors[i], 0);  
    SETGATE(idt[T_SYSCALL], 1, SEG_KCODE<<3, vectors[T_SYSCALL], DPL_USER);  
}
```

trap.c



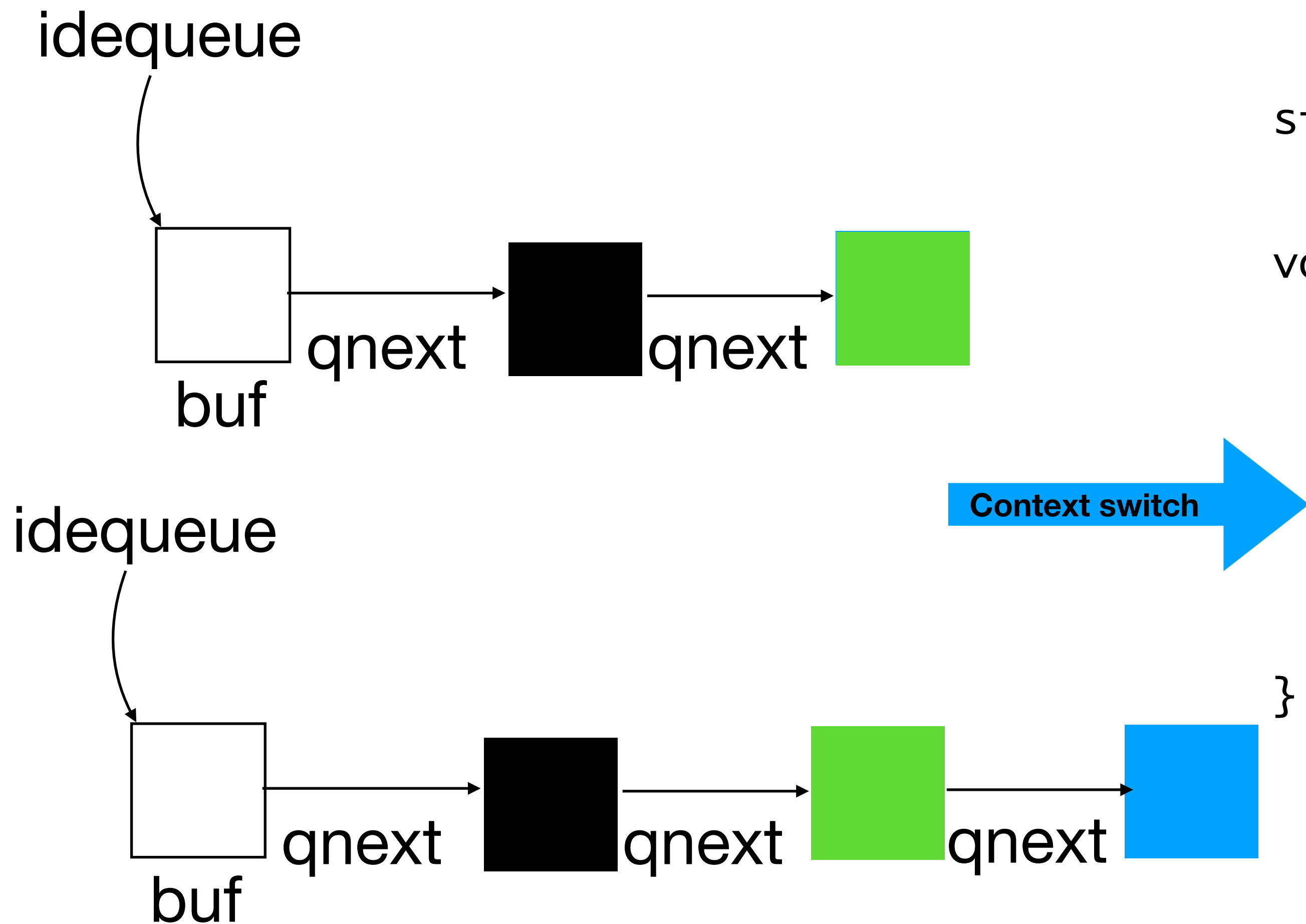
Concurrency (single processor)

Uniprocessor concurrency

- Software interrupts like system calls do not disable interrupts.
- If the kernel was updating its data structures, it needs to be careful to not run into races

Race conditions

Example: Processes reading a block in ide.c



```
struct buf {
```

```
..
```

```
    struct buf *qnext; // disk queue
```

```
    uchar data[BSIZE];
```

```
};
```

```
static struct buf *idequeue;
```

```
void iderw(struct buf *b) {
```

```
    struct buf **pp;
```

```
    b->qnext = 0;
```

```
    for(pp=&idequeue; *pp; pp=&(*pp)->qnext);
```

```
    *pp = b;
```

```
..
```

```
}
```

Protect critical sections!

```
void iderw(struct buf *b) {  
    struct buf **pp;  
    pushcli();  
    for(pp=&idequeue; *pp; pp=&(*pp)->qnext);  
    *pp = b;  
    ..  
    popcli();  
}
```

- Timer interrupt and hence context switch cannot happen

```
void pushcli(void) {  
    int eflags = readeflags();  
    cli();  
    if(cpu->ncli == 0)  
        cpu->intena = eflags & FL_IF;  
    cpu->ncli += 1;  
}  
  
void popcli(void) {  
    cpu->ncli--;  
    if(cpu->ncli == 0 && cpu->intena)  
        sti();  
}
```

Summary

- Memory isolation: How to isolate memory into address spaces?
- Processes in action: Process control block, user stack<>kernel stack, sys call handling, protection
- Scheduling: response time, turnaround time, fairness, scheduler attacks
- Uniprocessor concurrency: disable interrupts around critical sections